Contents

English | 日本語 | 繁體中文

Base64 Function Reference

Base64 encoding and decoding. All functions require explicit import from `base64`.

```
from base64 import encode, decode, encode_url_safe, decode_url_safe
```

Function List

Function	Signature	Description
`encode`	`(str) -> str`	Encodes a string to standard base64
`decode`	`(str) -> Result<str, Error>`	Decodes a standard base64 string
`encode_url_safe`	`(str) -> str`	Encodes a string to URL-safe base64 (no padding)
`decode_url_safe`	`(str) -> Result<str, Error>`	Decodes a URL-safe base64 string

Examples

Basic Encoding and Decoding

```
from base64 import encode, decode

encoded = encode("Hello, World!")
print(encoded)  # SGVsbG8sIFdvcmxkIQ==

case decode(encoded):
    Ok(s):
        print(s)  # Hello, World!
    Err(e):
        print(e.message)
```

URL-safe Base64

URL-safe base64 uses `-` and `_` instead of `+` and `/`, and omits padding (`=`). Useful for URLs, filenames, and tokens.

```
from base64 import encode_url_safe, decode_url_safe

encoded = encode_url_safe("data with special chars: ?&=")
# No + / or = in the output

case decode_url_safe(encoded):
    Ok(s):
        print(s)
    Err(e):
        print(e.message)
```

Working with Byte Data

To encode/decode byte data, combine with `to_bytes` / `bytes_to_str` from `io`.

```
from base64 import encode, decode
from io import to_bytes, bytes_to_str

bytes = to_bytes("binary data")
encoded = encode(bytes_to_str(bytes)?)
```

Error Handling

`decode` and `decode_url_safe` return `Result<str, Error>`. Decoding fails if the input contains invalid base64 characters.

```
case decode("!!!not-valid!!!"):
    Ok(s):
        print(s)
    Err(e):
        print(e.message)  # "invalid base64 character at position 0"
```

With the `?` operator:

```
function process(input: str) -> Result<str, Error>:
    decoded = decode(input)?
    return Ok(decoded)
```

English | 日本語 | 繁體中文

String Operation Function Reference

A list of operation functions for strings (`str`). All functions support UFCS notation.

> **Note:** All string operations are UTF-8 aware. `length()`, `char_at()`, `substring()`, `find()`, and `reverse()` operate on Unicode code points, not bytes. Use `byte_len()` if you need the byte length.

Function List

Search and Check

Function	Signature	Description
`contains`	`(str, str, bool = false) -> bool`	Returns whether a substring is contained
`starts_with`	`(str, str, bool = false) -> bool`	Returns whether it starts with a prefix
`ends_with`	`(str, str, bool = false) -> bool`	Returns whether it ends with a suffix
`find`	`(str, str) -> Option<int>`	Returns the character position of a substring (`None` if not found)

Extraction and Transformation

Function	Signature	Description
`substring`	`(str, int, int) -> str`	Extract a substring (character indices)
`char_at`	`(str, int) -> str`	Get the UTF-8 character at a specified position
`replace`	`(str, str, str) -> str`	Replace all occurrences of a substring

Case Conversion

Function	Signature	Description
`to_upper`	`str -> str`	Convert to ASCII uppercase
`to_lower`	`str -> str`	Convert to ASCII lowercase

Whitespace Removal

Function	Signature	Description
`trim`	`str -> str`	Remove leading and trailing whitespace
`trim_start`	`str -> str`	Remove leading whitespace
`trim_end`	`str -> str`	Remove trailing whitespace

Generation and Processing

Function	Signature	Description
`repeat`	`(str, int) -> str`	Repeat a string n times
`reverse`	`str -> str`	Reverse a string (UTF-8 aware)
`byte_len`	`str -> int`	Returns the byte length of a string

Split and Join

Function	Signature	Description
`split`	`(str, str) -> List<str>`	Split by delimiter
`join`	`(List<str>, str) -> str`	Join with separator

Type Conversion

Function	Signature	Description
`to_int`	`str -> Result<int, Error>`	Convert string to integer
`to_float`	`str -> Result<float, Error>`	Convert string to floating-point number
`to_str`	`int/float/bool/str/enum/record -> str`	Convert value to string

contains

Signature: `contains(string: str, substring: str, ignore_case: bool = false) -> bool`

Returns whether string `string` contains the substring `substring`. When `ignore_case` is `true`, the comparison is case-insensitive (ASCII only).

```
print(contains("hello", "ell"))              # true
print("hello".contains("xyz"))               # false (UFCS)
print(contains("Hello World", "hello", true))  # true (case-insensitive)
```

---

starts_with

Signature: `starts_with(string: str, prefix: str, ignore_case: bool = false) -> bool`

Returns whether string `string` starts with `prefix`. When `ignore_case` is `true`, the comparison is case-insensitive (ASCII only).

```
print(starts_with("hello", "hel"))              # true
print("hello".starts_with("world"))              # false (UFCS)
print(starts_with("Hello", "hello", true))  # true (case-insensitive)
```

---

ends_with

Signature: `ends_with(string: str, suffix: str, ignore_case: bool = false) -> bool`

Returns whether string `string` ends with `suffix`. When `ignore_case` is `true`, the comparison is case-insensitive (ASCII only).

```
print(ends_with("hello", "llo"))              # true
print("hello".ends_with("world"))              # false (UFCS)
print(ends_with("Hello World", "WORLD", true))  # true (case-insensitive)
```

---

find

Signature: `find(string: str, substring: str) -> Option<int>`

Returns the character position of the first occurrence of substring `substring` in string `string`. Returns `None` if not found.

```
print(find("hello world", "world"))   # Some(6)
print(find("hello", "xyz"))           # None
print("abcdef".find("cd"))            # Some(2) (UFCS)
```

---

substring

Signature: `substring(string: str, start: int, end: int) -> str`

Returns the substring of `string` from `start` to `end` (exclusive). Indices are character positions (UTF-8 aware).

Out-of-range indices are clamped to `[0, length]`. If `end < start` after clamping, returns an empty string.

```
print(substring("hello world", 0, 5))   # hello
print(substring("hello world", 6, 11))  # world
```

```
print("abcdef".substring(1, 4))          # bcd (UFCS)
print(substring("hello", -1, 100))       # hello (clamped)
```

---

char_at

Signature: `char_at(string: str, i: int) -> str`

Returns the UTF-8 character at position `i` in string `string` as a string. Raises a runtime error if the index is out of bounds.

Negative indices wrap around from the end (Python-style): `-1` refers to the last character, `-2` to the second-to-last, and so on.

```
print(char_at("hello", 0))     # h
print(char_at("hello", -1))    # o (last character)
print("abc".char_at(2))        # c (UFCS)
```

---

replace

Signature: `replace(string: str, old: str, new: str) -> str`

Returns a new string with all occurrences of `old` in `string` replaced with `new`.

If `old` is an empty string, the input is returned unchanged (as a fresh copy).

```
print(replace("hello world", "world", "ry"))   # hello ry
print(replace("aaa", "a", "bb"))                # bbbbbb
print("foo bar foo".replace("foo", "baz"))      # baz bar baz (UFCS)
print(replace("hello", "", "X"))                # hello (empty pattern is a no-op)
```

---

to_upper

Signature: `to_upper(string: str) -> str`

Returns a new string with ASCII lowercase letters (a-z) converted to uppercase.

```
print(to_upper("hello"))         # HELLO
print("Hello World".to_upper())  # HELLO WORLD (UFCS)
```

---

to_lower

Signature: `to_lower(string: str) -> str`

Returns a new string with ASCII uppercase letters (A-Z) converted to lowercase.

```
print(to_lower("HELLO"))         # hello
print("Hello World".to_lower())  # hello world (UFCS)
```

---

trim

Signature: `trim(string: str) -> str`

Returns a new string with leading and trailing whitespace characters (spaces, tabs, newlines, carriage returns) removed.

```
print(trim("  hello  "))   # hello
print("  hi  ".trim())     # hi (UFCS)
```

---

trim_start

Signature: `trim_start(string: str) -> str`

Returns a new string with leading whitespace characters removed.

```
print(trim_start("  hello  "))   # hello
print("  hi".trim_start())       # hi (UFCS)
```

---

trim_end

Signature: `trim_end(string: str) -> str`

Returns a new string with trailing whitespace characters removed.

```
print(trim_end("  hello  "))   #   hello
print("hi  ".trim_end())       # hi (UFCS)
```

---

repeat

Signature: `repeat(string: str, count: int) -> str`

Returns a new string with `string` repeated `count` times.

```
print(repeat("ab", 3))     # ababab
print("ha".repeat(3))      # hahaha (UFCS)
```

---

reverse

Signature: `reverse(string: str) -> str`

Returns a new string with the characters reversed (UTF-8 aware).

```
print(reverse("hello"))    # olleh
print("abc".reverse())     # cba (UFCS)
```

---

byte_len

Signature: `byte_len(string: str) -> int`

Returns the byte length of string `string`. Unlike `length()`, which returns the number of UTF-8 characters, `byte_len()` returns the number of bytes.

```
print(byte_len("hello"))   # 5
print(byte_len("あいう"))   # 9
print(length("あいう"))        # 3 (characters)
```

---

split

Signature: `split(string: str, delimiter: str) -> List<str>`

Splits string `string` by delimiter `delimiter` and returns a `List<str>`.

When the delimiter is an empty string `""`, the string is split into individual characters (UTF-8 aware).

```
parts = split("a,b,c", ",")
print(parts[0])   # a
print(parts[1])   # b
print(parts[2])   # c

for word in "hello world".split(" "):
    print(word)
# hello
# world

# Split into characters
chars = split("hello", "")
print(chars)   # [h, e, l, l, o]

# UTF-8 characters
chars = split("あいう", "")
print(chars)   # [あ, い, う]
```

> **Tip:** To iterate a string character by character, you can use a `for` loop directly without calling `split`: `for c in s:` yields each UTF-8 code point as a single-character `str`. See control-flow.md.

---

join

Signature: `join(values: List<str>, sep: str) -> str`

Joins the elements of a string list with separator `sep` and returns a string.

```
parts = ["a", "b", "c"]
print(join(parts, ","))        # a,b,c
print(parts.join("-"))         # a-b-c (UFCS)
print(",".join(parts))         # a,b,c (UFCS, Python-style)
```

---

to_int

Signature: `to_int(string: str) -> Result<int, Error>`

Converts a string to an integer. Leading whitespace is allowed. Returns `Err` if the string is empty, contains invalid characters, or overflows.

```
case to_int("42"):
    Ok(v):
        print(v)              # 42
```

```
    Err(e):
        print(e.message)

case "123".to_int():
    Ok(v):
        print(v)              # 123
    Err(e):
        print(e.message)

# Invalid input returns Err
print(to_int("abc"))          # Err(Error("to_int: invalid character in 'abc'"))
print(to_int(""))             # Err(Error("to_int: empty string"))
```

---

to_float

Signature: `to_float(string: str) -> Result<float, Error>`

Converts a string to a floating-point number. Returns `Err` if the string is empty, contains invalid characters, or is out of range for `float`.

```
case to_float("3.14"):
    Ok(v):
        print(v)              # 3.14
    Err(e):
        print(e.message)

case "2.5".to_float():
    Ok(v):
        print(v)              # 2.5
    Err(e):
        print(e.message)

# Invalid input returns Err
print(to_float("abc"))         # Err(Error("to_float: invalid character in 'abc'"))
print(to_float(""))            # Err(Error("to_float: empty string"))
print(to_float("1e400"))       # Err(Error("to_float: out of range in '1e400'"))
```

---

to_str

Signature: `to_str(v: int | float | bool | str | enum | record) -> str`

Converts a value to a string.

Type	Conversion Format
`int`	`%ld`
`float`	`%g`, with trailing `.0` for whole-number values (e.g. `"3.0"`, `"0.0"`)
`bool`	`"true"` / `"false"`
`str`	Returned as-is
enum	Variant name (e.g. `"Red"`)
record	`TypeName(field1: val1, field2: val2)`
`List` / `Map` / `Set`	Recursively formatted, nested containers (e.g. `Map<str, List<int>>`) are supported

Type	Conversion Format
union	Formatted as the active variant; `List`, `Map`, `Set`, and function variants are all supported
function value (closure / lambda)	`"<closure>"`

Whole-number `float` values are formatted with a trailing `.0` (e.g. `to_str(3.0) == "3.0"`) so they are visually distinguishable from `int`. Record types automatically generate a `to_str` representation. If a user-defined `function to_str(v: MyRecord) -> str` is provided, it takes precedence over the auto-generated version. This also works with `print()` and f-string interpolation.

```
print(to_str(42))          # 42
print(to_str(3.14))        # 3.14
print(to_str(3.0))         # 3.0          (whole-number float keeps .0)
print(to_str(true))        # true
print(99.to_str())         # 99 (UFCS)

enum Color:
    Red
    Green

print(to_str(Color::Red))   # Red

record Point:
    x: int
    y: int

p = Point(10, 20)
print(to_str(p))           # Point(x: 10, y: 20)
print(p)                   # Point(x: 10, y: 20)
print(f"pos={p}")          # pos=Point(x: 10, y: 20)
```

English | 日本語 | 繁體中文

Built-in Function Reference

Function List

Core

Function	Description
`print()` / `print(expr1, expr2, ...)`	Prints values to standard output (space-separated)
`length(value)`	Returns the number of elements in a list, map, or set, or the number of UTF-8 characters in a string
`range(n)` / `range(start, end)` / `range(start, end, step)`	Generates a list of integers
`exit(code)`	Terminates the process with the given exit code
`arguments()`	Returns command-line arguments as `List<str>`
`available_parallelism()`	Returns the runtime worker count as `int`
`sleep(duration_ms)`	Suspends execution for the specified number of milliseconds
`env(key)`	Returns the environment variable as `Option<str>`

Function	Description
`env(key, default)`	Returns the environment variable, or `default` if not set
`send(stream, data)`	Sends `List<u8>` through `TcpStream` or `TlsStream`, returns `Result<int, Error>`
`receive(stream, max)`	Receives up to `max` bytes from `TcpStream` or `TlsStream` as `Result<List<u8>, Error>`
`close(handle)`	Closes a `TcpStream`, `TlsStream`, or `TcpListener`
`block_on(task)`	Blocks the current thread until a `Task<T>` completes and returns its result
`to_str(value)`	Converts a value to its string representation. Supports `int`, `float` (whole numbers print with trailing `.0`), `bool`, `str`, record, enum, tuple, `List`, `Map`, `Set` (nested containers like `Map<str, List<int>>` are recursively formatted), `Result`, `Option`, union types (formatted as the active variant), and function values (printed as `<closure>`). String elements inside collections are wrapped in double quotes (e.g., `["hello", "world"]`)
`type_of(expr)`	Returns the type of `expr` as a `Type` value. See type_of
`fail()` / `fail(message)`	Marks the current test as failed (only available in `ry test` mode)

Option

Function	Description
`Some(expr)`	Constructs the value-present variant of an Option type

Result / Error

Function	Description
`Ok(value)`	Constructs the success variant of a `Result<T, Error>`
`Err(error)`	Constructs the error variant of a `Result<T, Error>`
`Error(message)`	Creates an `Error` value with a message
`Error(message, code)`	Creates an `Error` value with a message and error code
`result.and_then(closure)`	If `Ok`, calls `closure` (which returns `Result<U, E>`); if `Err`, propagates the error
`result.map(closure)`	If `Ok`, applies `closure` to the value and wraps the return in `Ok`; if `Err`, propagates the error

Checked Arithmetic

Function	Description
`checked_add(a, b)`	Returns `Ok(a + b)` if no overflow, otherwise `Err(Error("arithmetic overflow"))`
`checked_sub(a, b)`	Returns `Ok(a - b)` if no overflow, otherwise `Err(Error("arithmetic overflow"))`
`checked_mul(a, b)`	Returns `Ok(a * b)` if no overflow, otherwise `Err(Error("arithmetic overflow"))`
`saturating_add(a, b)`	Returns `a + b`, clamped to `int` range on overflow
`saturating_sub(a, b)`	Returns `a - b`, clamped to `int` range on overflow
`saturating_mul(a, b)`	Returns `a * b`, clamped to `int` range on overflow
`wrapping_add(a, b)`	Returns `a + b` with wrapping on overflow
`wrapping_sub(a, b)`	Returns `a - b` with wrapping on overflow
`wrapping_mul(a, b)`	Returns `a * b` with wrapping on overflow

Collection Operations

Function	Description
`has_key(map, key)`	Returns whether a key exists in the map
`add(set, value)`	Adds an element to a set (duplicates are ignored)
`remove(set, value)`	Removes an element from a set
`append(list, value)` / `append!(list, value)`	Adds an element to the end of a list (mutating)
`appended(list, value)`	Returns a new list with the element added (non-mutating)
`pop(list)`	Removes and returns the last element as `Option<T>`
`reverse(list)`	Returns a new reversed list (also works on strings)
`reverse!(list)`	Reverses a list in place (mutating)
`slice(list, start, end)`	Returns a new sub-list from start to end
`take(list, count)`	Returns a new list with the first count elements
`tap(list, function)`	Calls function on each element for side effects, returns the original list
`filter(list, pred)`	Returns a new list with elements matching the predicate
`map(list, function)`	Returns a new list with each element transformed
`sort(list)` / `sort(list, comp)`	Returns a new sorted list (default ascending)
`sort!(list)` / `sort!(list, comp)`	Sorts a list in place (mutating)
`insert(list, i, val)`	Inserts an element at index i
`remove_at(list, i)`	Removes and returns the element at index i
`items(map)`	Returns a list of (key, value) tuples
`remove(map, key)`	Removes the entry with the specified key
`get(map, key)`	Returns the value for key as `Option<V>`
`get(map, key, default)`	Returns the value for key, or default if not found
`union(set, set)`	Returns the union of two sets
`intersection(set, set)`	Returns the intersection of two sets
`difference(set, set)`	Returns the difference of two sets
`symmetric_difference(set, set)`	Returns the symmetric difference of two sets
`is_subset(set, set)`	Returns whether the first set is a subset of the second
`is_superset(set, set)`	Returns whether the first set is a superset of the second
`first(list)`	Returns the first element as `Option<T>`, or `None` if empty
`last(list)`	Returns the last element as `Option<T>`, or `None` if empty

Function	Description
`remove(list, value)`	Removes the first occurrence of value from a list
`is_empty(list / map / set / str)`	Returns whether the collection or string is empty
`distinct(list)`	Returns a new list with duplicates removed
`flatten(list)`	Returns a new list with nested lists flattened
`reduce(list, fn)`	Reduces a list to a single value using the reducer function
`fold(list, init, fn)`	Folds a list with an initial accumulator value
`any(list, pred)`	Returns `true` if any element matches the predicate
`all(list, pred)`	Returns `true` if all elements match the predicate
`sum(list)`	Returns the sum of all elements
`min(list)`	Returns the minimum element
`max(list)`	Returns the maximum element
`enumerate(list)`	Returns a list of `(index, value)` tuples. Also accepts a `str`, yielding `(int, str)` per UTF-8 code point
`zip(list1, list2)`	Returns a list of `(a, b)` tuples pairing elements from two lists. Either (or both) arguments may be a `str`
`keys(map)`	Returns all keys as a `List<K>`
`values(map)`	Returns all values as a `List<V>`
`merge(map1, map2)`	Returns a new map containing entries from both maps

Iterator

Function	Description
`iter(collection)`	Creates a lazy iterator from a List, Set, or Map
`next(iter)`	Returns the next element as `Option<T>`, or `None` if exhausted
`to_list(iter)`	Collects all remaining iterator elements into a `List<T>`
`filter(iter, pred)`	Returns a lazy iterator that yields only elements matching the predicate
`map(iter, function)`	Returns a lazy iterator that transforms each element
`take(iter, count)`	Returns a lazy iterator that yields at most count elements

String Operations

Function	Description
`contains(string, substring)`	Whether a substring is contained
`starts_with(string, prefix)`	Whether it starts with a prefix
`ends_with(string, suffix)`	Whether it ends with a suffix
`find(string, substring)`	Character position of a substring (`Option<int>`)
`byte_len(string)`	Returns the byte length of a string
`substring(string, start, end)`	Extract a substring
`char_at(string, i)`	Get the character at a specified position
`replace(string, old, new)`	Replace all occurrences of a substring
`to_upper(string)` / `to_lower(string)`	Uppercase / lowercase conversion

Function	Description
`trim(string)` / `trim_start(string)` / `trim_end(string)`	Whitespace removal
`repeat(string, count)`	Repeat a string n times
`reverse(string)`	Reverse a string
`split(string, delimiter)`	Split a string into a list
`join(list, sep)`	Join list elements with a separator
`to_int(s)` / `to_float(s)` / `to_str(v)`	Type conversion (`to_int` and `to_float` return `Result<T, Error>`)

-> See **String Operation Function Reference** for details

---

print

Signature: `print()` / `print(expr1, expr2, ...)`

Prints one or more values to standard output, separated by spaces. A newline is appended at the end. When called with no arguments, prints only a newline.

Type	Output Format
`int`	`%ld`
`float`	`%g`, with trailing `.0` for whole-number values (e.g. `3.0`, `0.0`)
`bool`	`true` / `false`
`str`	`%s`
`Result` (Ok)	`Ok(value)`
`Result` (Err)	`Err(value)`
`Option` (Some)	`Some(value)`
`Option` (None)	`None`
`list`	`[elem1, elem2, ...]`
`map`	`{key1: val1, key2: val2, ...}`
`set`	`{elem1, elem2, ...}`
`tuple`	`(elem1, elem2, ...)`
`enum`	Variant name (e.g., `Red`)
`record`	`RecordName(field: val, ...)`
function value (closure / lambda)	`<closure>`
union	Formatted as the active variant's type

Whole-number `float` values always print with a trailing `.0` so they are visually distinguishable from `int`. Nested collections (e.g. `Map<str, List<int>>`) are recursively formatted using the inner element's formatter. Union variants whose underlying type is `List`, `Map`, or `Set` format as that collection; variants whose underlying type is a function value format as `<closure>`.

```
print(42)          # 42
print(3.14)        # 3.14
print(3.0)         # 3.0         (whole-number float keeps .0)
print(0.0)         # 0.0
print(true)        # true
print("hello")     # hello
print(Ok(42))      # Ok(42)
print(Err(Error("fail")))  # Err(Error: fail (code: 0))
```

```
print(Some(1))      # Some(1)
print(None)         # None
print([1, 2, 3])    # [1, 2, 3]
print({"a": 1})     # {"a": 1}
print({1, 2, 3})    # {1, 2, 3}
print((1, "hello"))  # (1, "hello")

# Nested collections
m: Map<str, List<int>> = {"a": [1, 2, 3]}
print(m)            # {"a": [1, 2, 3]}

# Collection-typed union variant
x: int | List<int> = [1, 2, 3]
print(x)            # [1, 2, 3]

# Function value
f = (x: int) => x * 2
print(f)            # <closure>

# Multiple arguments (space-separated)
print(1, 2, 3)              # 1 2 3
print("hello", "world")    # hello world
print(1, "hello", true)    # 1 hello true
print()                     # (empty line)
```

Some

Signature: `Some(expr) -> Option<T>`

Constructs the value-present variant of an Option type.

```
x: Option<int> = Some(42)
print(x)    # Some(42)
```

---

length

Signature: `length(x: List<T> | Map<K, V> | Set<T> | str) -> int`

Returns the number of elements in a list, map, or set, or the number of UTF-8 characters in a string. Use `byte_len()` for the byte length.

```
print(length([1, 2, 3]))         # 3
print(length({"a": 1, "b": 2})) # 2
print(length({1, 2, 3}))         # 3
print(length("hello"))           # 5
print(length("あいう"))           # 3 (UTF-8 characters)
```

---

has_key

Signature: `has_key(m: Map<K, V>, key: K) -> bool`

Returns whether a specified key exists in the map. UFCS notation is also available.

```
m = {"a": 1, "b": 2}
print(has_key(m, "a"))     # true
print(m.has_key("z"))      # false (UFCS)
```

---

add

Signature: `add(s: Set<T>, value: T)`

Adds an element to a set. Does nothing if the element already exists. UFCS notation is also available.

```
s = {1, 2, 3}
s.add(4)           # UFCS
add(s, 5)          # Normal call
s.add(1)           # Ignored because it already exists
print(length(s))     # 5
```

---

remove

Signature: `remove(s: Set<T>, value: T)`

Removes an element from a set. UFCS notation is also available.

```
s = {1, 2, 3}
s.remove(2)        # UFCS
print(2 in s)      # false
```

---

range

Signature: `range(n: int) -> List<int>` / `range(start: int, end: int) -> List<int>` / `range(start: int, end: int, step: int) -> List<int>`

Generates a list of integers.

Form	Generated Values
`range(n)`	`[0, 1, ..., n-1]`
`range(start, end)`	`[start, start+1, ..., end-1]`
`range(start, end, step)`	`[start, start+step, start+2*step, ...]` (up to but not including `end`)

- When `step > 0`, generates values from `start` ascending toward `end`.
- When `step < 0`, generates values from `start` descending toward `end`.
- When `step == 0`, a runtime error occurs.
- If the range is empty (e.g., `range(0, 10, -1)`), returns an empty list.

```
print(range(3))            # [0, 1, 2]
print(range(2, 5))         # [2, 3, 4]
print(range(0, 10, 2))     # [0, 2, 4, 6, 8]
print(range(10, 0, -3))    # [10, 7, 4, 1]

for i in range(3):
    print(i)
# 0
```

```
# 1
# 2
```

---

exit

Signature: `exit(code: int)`

Terminates the process immediately with the given exit code. Statements after `exit()` are compiled into an unreachable block that LLVM removes during optimization, so they never run:

```
exit(0)        # normal termination
exit(1)        # error termination

print("a")
exit(0)
print("b")     # never prints — unreachable after exit
```

The same treatment applies to `return`, `break`, and `continue` — code after any diverging control-flow statement is silently elided.

---

arguments

Signature: `arguments() -> List<str>`

Returns the command-line arguments passed to the script as a list of strings. Does not include the interpreter name or the script filename —only the arguments after the script path.

```
# Run: ry script.ry hello world
a = arguments()
print(length(a))    # 2
print(a[0])      # hello
print(a[1])      # world

for x in arguments():
    print(x)
```

---

sleep

Signature: `sleep(duration_ms: int) -> Unit`

Suspends execution of the current thread for the specified number of milliseconds. If `duration_ms` is 0 or negative, the function returns immediately.

```
sleep(1000)    # wait 1 second
sleep(0)       # returns immediately
```

---

env

Signature: `env(key: str) -> Option<str>` / `env(key: str, default: str) -> str`

Returns the value of an environment variable. The one-argument form returns `Option<str>` (`Some(value)` if set, `None` if not). The two-argument form returns the value or `default` if the variable is not set.

If a `.env` file exists in the project root (the directory containing `package.toml`), its entries are automatically loaded into the process environment at startup. Existing environment variables are not overwritten by `.env` values.

> **Security note:** `.env` files typically contain secrets (API keys, database passwords, tokens, etc.). Do **not** commit `.env` to version control (add it to `.gitignore` or equivalent), and treat its contents as sensitive configuration.

```
# One-argument form: returns Option<str>
path = env("PATH")
case path:
    Some(v):
        print(v)
    None:
        print("PATH not set")

# Two-argument form: returns str with default
port = env("PORT", "8080")
print(port)   # "8080" if PORT is not set
```

`.env` file format

```
# Comments start with #
DATABASE_URL=postgres://localhost/mydb
API_KEY="secret-key-123"
EMPTY_VALUE=
QUOTED='single quoted'
```

Environment-specific `.env` files

When `RY_ENV` is set, Ry loads environment-specific `.env` files with the following priority:

- `.env.<env>` is loaded first (e.g., `.env.dev` when `RY_ENV=dev`)
- `.env` is loaded second (values already set by `.env.<env>` are not overwritten)
- When `RY_ENV=prod`, no `.env` files are loaded (security)
- When `RY_ENV` is not set, only `.env` is loaded (backward compatible)

See RY_ENV for details on environment modes.

---

append

Signature: `append(list: List<T>, value: T)`

Adds an element to the end of a list. This is a mutating operation — the list is modified in place. UFCS notation is also available.

```
xs = [1, 2]
xs.append(3)
print(xs)   # [1, 2, 3]
```

---

pop

Signature: `pop(list: List<T>) -> Option<T>`

Removes and returns the last element of a list as `Option<T>`. Returns `None` if the list is empty. UFCS notation is also available.

```
xs = [1, 2, 3]
v = xs.pop()
print(v)    # Some(3)
print(xs)   # [1, 2]
```

---

reverse (list)

Signature: `reverse(list: List<T>) -> List<T>`

Returns a new list with elements in reverse order. The original list is not modified. Also works on strings (see String Operations). UFCS notation is also available.

```
xs = [1, 2, 3]
ys = reverse(xs)
print(ys)   # [3, 2, 1]
print(xs)   # [1, 2, 3] (unchanged)
```

---

slice

Signature: `slice(list: List<T>, start: int, end: int) -> List<T>`

Returns a new sub-list from `start` (inclusive) to `end` (exclusive). Indices are clamped to the valid range (`0` to `length(list)`). UFCS notation is also available.

```
xs = [1, 2, 3, 4, 5]
print(slice(xs, 1, 3))     # [2, 3]
print(slice(xs, 0, 100))   # [1, 2, 3, 4, 5] (clamped)
```

---

take

Signature: `take(list: List<T>, count: int) -> List<T>`

Returns a new list with the first `count` elements. If `count` exceeds the list length, returns a copy of the entire list. If `count <= 0`, returns an empty list. The original list is not modified. UFCS notation is also available.

```
xs = [1, 2, 3, 4, 5]
ys = xs.take(3)
print(ys)   # [1, 2, 3]
print(xs.take(10))   # [1, 2, 3, 4, 5] (clamped)
print(xs.take(0))    # []
```

---

tap

Signature: `tap(list: List<T>, function: function(T) -> R) -> List<T>`

Calls the given function on each element (ignoring any return value), then returns the original list unchanged. Useful for debugging or inserting side effects in a method chain. UFCS notation is also available.

```
xs = [1, 2, 3]
ys = xs.tap((x: int) => print(x)).map((x: int) => x * 2)
# prints 1, 2, 3, then ys = [2, 4, 6]
```

---

filter

Signature: `filter(list: List<T>, pred: function(T) -> bool) -> List<T>`

Returns a new list containing only elements for which the predicate returns `true`. The original list is not modified. UFCS notation is also available.

```
xs = [1, 2, 3, 4, 5]
ys = xs.filter((x: int) => x > 3)
print(ys)   # [4, 5]
print(xs)   # [1, 2, 3, 4, 5]  (unchanged)
```

---

map

Signature: `map(list: List<T>, function: function(T) -> U) -> List<U>`

Returns a new list with each element transformed by the given function. The output element type can differ from the input type. The original list is not modified. UFCS notation is also available.

```
xs = [1, 2, 3]
ys = xs.map((x: int) => x * 2)
print(ys)   # [2, 4, 6]
```

---

sort

Signature: `sort(list: List<T>) -> List<T>` / `sort(list: List<T>, comp: function(T, T) -> bool) -> List<T>`

Returns a new sorted list. Default is ascending order. An optional comparator function can be provided that returns `true` if the first argument should come before the second. The original list is not modified. The sort is **stable** (equal elements preserve their original order). UFCS notation is also available.

```
xs = [3, 1, 2]
print(xs.sort())   # [1, 2, 3]

# Descending order
desc = xs.sort((a: int, b: int) => a > b)
print(desc)   # [3, 2, 1]
```

---

sort!

Signature: `sort!(list: List<T>)` / `sort!(list: List<T>, comp: function(T, T) -> bool)`

Sorts a list in place. Same sorting algorithm as `sort()`, but modifies the original list instead of creating a new one. UFCS notation is also available.

```
xs = [3, 1, 2]
xs.sort!()
print(xs)   # [1, 2, 3]
```

---

reverse!

Signature: `reverse!(list: List<T>)`

Reverses a list in place. UFCS notation is also available.

```
xs = [1, 2, 3]
xs.reverse!()
print(xs)   # [3, 2, 1]
```

---

appended

Signature: `appended(list: List<T>, value: T) -> List<T>`

Returns a new list with the element added at the end. The original list is not modified. UFCS notation is also available.

```
xs = [1, 2]
ys = xs.appended(3)
print(xs)   # [1, 2] (unchanged)
print(ys)   # [1, 2, 3]
```

---

append!

Signature: `append!(list: List<T>, value: T)`

Alias for `append()`. Adds an element to the end of a list in place. Provided for naming consistency with the `!` convention.

---

first

Signature: `first(list: List<T>) -> Option<T>`

Returns the first element of a list as `Option<T>`. Returns `None` if the list is empty.

```
print(first([10, 20, 30]))   # Some(10)
```

---

last

Signature: `last(list: List<T>) -> Option<T>`

Returns the last element of a list as `Option<T>`. Returns `None` if the list is empty.

```
print(last([10, 20, 30]))   # Some(30)
```

---

get (Map)

Signature: `get(map: Map<K, V>, key: K) -> Option<V>` / `get(map: Map<K, V>, key: K, default: V) -> V`

Two-argument form returns the value for key as `Option<V>`. Three-argument form returns the value or the default.

```
m = {"a": 1, "b": 2}
print(get(m, "a"))        # Some(1)
print(get(m, "z"))        # None
print(get(m, "z", 0))    # 0
```

---

iter

Signature: `iter(collection: List<T> | Set<T>) -> Iterator<T> / iter(collection: Map<K, V>) -> Iterator<(K, V)>`

Creates a lazy iterator from a collection. The iterator does not copy data; it references the original collection. UFCS notation is also available.

- For `List<T>` and `Set<T>`, the element type is `T`.
- For `Map<K, V>`, the element type is the tuple `(K, V)`.

```
xs = [1, 2, 3]
it = xs.iter()            # Iterator<int>
ys = it.to_list()         # [1, 2, 3]

m = {"a": 1, "b": 2}
for k, v in m.iter():         # Iterator<(str, int)>
    print(k)
```

---

next

Signature: `next(iter: Iterator<T>) -> Option<T>`

Returns the next element from the iterator as `Option<T>`. Returns `None` when the iterator is exhausted. The iterator advances its internal state on each call. UFCS notation is also available.

```
it = [10, 20].iter()
print(it.next())    # Some(10)
print(it.next())    # Some(20)
print(it.next())    # None
```

---

to_list

Signature: `to_list(iter: Iterator<T>) -> List<T>`

Collects all remaining elements from the iterator into a new list. UFCS notation is also available.

```
xs = [1, 2, 3, 4, 5]
ys = xs.iter().filter((x: int) => x > 2).to_list()
print(ys)    # [3, 4, 5]
```

---

type_of

Signature: `type_of(expr: T) -> Type`

Returns the type of an expression as a `Type` value. Every distinct type definition (primitive, collection, record, enum, `Option`, `Result`, function, etc.) receives a unique identity at compile time, so `type_of` values can be compared by `==` to check whether two expressions share the same type.

- The argument is evaluated for side effects but only its static type is used.
- Printing a `Type` value via `print` or `to_str` yields the human-readable name (for example, `"int"`, `"List"`, `"Point"`).
- Two expressions with the same canonical type return equal `Type` values; different records (or a record and an enum that happen to share a name) are always distinguishable.
- The bare `none` literal reports as `"None"`. A typed `Option<T>` value (whether constructed via `Some(...)` or assigned from `none`) reports as `"Option"`.

```
record Point:
  x: int
  y: int

enum Color:
  Red
  Green
  Blue

print(to_str(type_of(42)))          # int
print(to_str(type_of(3.14)))        # float
print(to_str(type_of("hello")))     # str
print(to_str(type_of([1, 2, 3])))   # List
print(to_str(type_of({"a": 1})))    # Map
print(to_str(type_of({1, 2})))      # Set

p = Point(1, 2)
print(to_str(type_of(p)))           # Point

c = Color::Red
print(to_str(type_of(c)))           # Color

# identity comparison
print(type_of(42) == type_of(100))  # true
print(type_of(42) == type_of(3.14)) # false
print(type_of(p) != type_of(c))     # true

# low-level numeric types are distinguished from `int`
x: i32 = 1
print(to_str(type_of(x)))           # i32
print(type_of(x) == type_of(42))    # false

# type_of is reflective: the type of a Type value is Type
print(to_str(type_of(type_of(42)))) # Type
```

Type categories returned by `type_of`

Input	`to_str(type_of(...))`
`42`	`int`
`3.14`	`float`
`true` / `false`	`bool`
`"hello"`	`str`
`[1, 2]`	`List`
`{"a": 1}`	`Map`
`{1, 2}`	`Set`
`x: i32 = 1`	`i32` (and similarly for `u8`, `i16`, ⋯, `f32`)

Input	`to_str(type_of(...))`
record value	record name (e.g. `Point`)
enum value	enum name (e.g. `Color`)
`none` literal	`None`
`Some(1)`	`Option`
`x: Option<int> = none`	`Option`
`Ok(1)` / `Err(e)`	`Result`
lambda / closure	`function`
`type_of(x)`	`Type`

The bare `none` literal is reported as `"None"` to distinguish it from a typed `Option` value. Any `Option<T>` container — whether constructed via `Some(...)` or assigned from `none` to an `Option<T>`-typed binding — reports as `"Option"`.

English | 日本語 | 繁體中文

Collection Reference (Tuple, List, Map, Set)

Tuple

Overview

A fixed-length combination of heterogeneous values. Implemented as a stack-allocated value type using LLVM literal StructType.

Syntax

```
t = (42,)                       # single-element tuple (trailing comma required)
t = (1, 3.14)
t: (int, float) = (1, 3.14)
```

Type Annotation

```
single: (int,) = (42,)                    # trailing comma required for single-element
pair: (int, str) = (42, "hello")
triple: (int, float, bool) = (1, 2.0, true)
```

Comparison

Tuples support `==` and `!=` via element-wise comparison.

```
print((1, 2) == (1, 2))     # true
print((1, 2) != (3, 4))     # true
print(("a", 1) == ("b", 1))  # false
```

Element Access

Access elements using numeric indices `.0`, `.1`, etc.

```
t = (10, 3.14)
print(t.0)   # 10
print(t.1)   # 3.14
```

Function Return Values

```
function swap(a: int, b: int) -> (int, int):
    return (b, a)

result = swap(1, 2)
print(result.0)   # 2
print(result.1)   # 1
```

Constraints and Errors

Constraint	Details
Out-of-range index	Compile error
Comparison operators	Only `==` and `!=` are supported; `<`, `<=`, `>`, `>=` are not

---

List

Overview

A variable-length sequence of elements of the same type. Allocated on the heap.

Syntax

```
xs = [1, 2, 3]
xs: List<int> = [1, 2, 3]
```

Empty List

An empty list requires a type annotation so the element type can be determined:

```
xs: List<int> = []
xs: List<str> = []
```

Concatenation

Lists can be concatenated with `+` and `+=`:

```
a = [1, 2, 3]
b = [4, 5, 6]
c = a + b       # [1, 2, 3, 4, 5, 6]
a += [7, 8]     # a is now [1, 2, 3, 7, 8]
```

Both operands must have the same element type.

Supported Element Types

`int`, `float`, `bool`, `str`

Equality

Lists support `==` and `!=`. Two lists are equal when they have the same length and all corresponding elements are equal.

```
[1, 2, 3] == [1, 2, 3]   # true
[1, 2, 3] != [1, 2, 4]   # true
[1, 2]    != [1, 2, 3]   # true (different lengths)
```

Index Access

```
xs = [1, 2, 3]
print(xs[0])   # 1
print(xs[2])   # 3
```

Negative indices wrap around from the end (Python-style):

```
xs = [10, 20, 30]
print(xs[-1])   # 30 (last element)
print(xs[-2])   # 20
print(xs[-3])   # 10
```

Out-of-bounds access (including negative indices that exceed the list length) raises a runtime error.

Index Assignment

```
xs = [1, 2, 3]
xs[0] = 99
print(xs[0])   # 99
xs[-1] = 42    # assigns to last element
print(xs[2])   # 42
```

Chained Index and Field Assignment

Index and field assignment compose: the left-hand side of `=` / `+=` / `-=` / `*=` / `/=` / `//=` / `%=` / `**=` / `&=` / `|=` / `^=` / `<<=` / `>>=` can be any postfix chain rooted at a mutable variable.

```
record Point:
  x: int
  y: int

pts = [Point(1, 2), Point(3, 4)]
pts[0].x = 99           # list-of-records field update
pts[0].x += 1
print(pts[0].x)         # 100

grid = [[1, 2], [3, 4]]
grid[0][1] = 99         # nested list element
print(grid[0])          # [1, 99]

m: Map<str, Map<str, int>> = {"a": {"x": 1}}
m["a"]["x"] = 42        # nested map element
```

Compound assignment evaluates each index exactly once, so `xs[f()] += 1` calls `f()` a single time. Compound assignment to a missing map key is a runtime error — insert the key first if you want to accumulate:

```
m = {"a": 1}
m["a"] += 10            # OK  → {"a": 11}
m["b"] += 10            # runtime error: compound assignment to missing map key
```

> **Nested writes are isolated**: chained writes like `grid[i][j] = v` and `r.items[i] = v` (through a record field containing a list) privatize every level on the LHS path whose reference count > 1 before the mutation lands, so aliases of the outer container — or any inner container on the path — observe the pre-write state. See Path Copy-on-Write below for the details.

length

```
xs = [1, 2, 3]
print(length(xs))   # 3
```

print

```
xs = [1, 2, 3]
print(xs)   # [1, 2, 3]
```

for Iteration

```
xs = [10, 20, 30]
for x in xs:
    print(x)
# 10
# 20
# 30
```

append

Adds an element to the end of the list. This is a mutating operation.

```
xs = [1, 2]
xs.append(3)
print(xs)   # [1, 2, 3]
```

pop

Removes and returns the last element. Returns `None` if the list is empty.

```
xs = [1, 2, 3]
v = xs.pop()
print(v)    # Some(3)
print(xs)   # [1, 2]
```

reverse

Returns a new list with elements in reverse order. The original list is not modified. Also works on strings.

```
xs = [1, 2, 3]
print(reverse(xs))   # [3, 2, 1]
print(xs)            # [1, 2, 3] (unchanged)
```

slice

Returns a new sub-list from `start` (inclusive) to `end` (exclusive). Indices are clamped to the valid range.

```
xs = [1, 2, 3, 4, 5]
print(slice(xs, 1, 3))     # [2, 3]
print(slice(xs, 0, 100))   # [1, 2, 3, 4, 5] (clamped)
```

take

Returns a new list with the first `count` elements. If `count` exceeds the list length, returns a copy of the entire list. If `count <= 0`, returns an empty list. The original list is not modified.

```
xs = [1, 2, 3, 4, 5]
ys = xs.take(3)
```

```
print(ys)   # [1, 2, 3]
print(xs.take(10))   # [1, 2, 3, 4, 5] (clamped)
print(xs.take(0))    # []
```

tap

Calls the given function on each element (ignoring any return value), then returns the original list unchanged. Useful for debugging or inserting side effects in a method chain.

```
xs = [1, 2, 3]
ys = xs.tap((x: int) => print(x)).map((x: int) => x * 2)
# prints 1, 2, 3, then ys = [2, 4, 6]
```

filter

Returns a new list containing only elements that satisfy the predicate. The original list is not modified.

```
xs = [1, 2, 3, 4, 5]
ys = xs.filter((x: int) => x > 3)
print(ys)   # [4, 5]
```

map

Returns a new list with each element transformed by the given function. The output element type can differ from the input. The original list is not modified.

```
xs = [1, 2, 3]
ys = xs.map((x: int) => x * 2)
print(ys)   # [2, 4, 6]
```

sort

Returns a new sorted list. Default is ascending order. A custom comparator can be provided. The original list is not modified. The sort is **stable** (equal elements preserve their original order). Internally uses TimSort.

```
xs = [3, 1, 2]
print(xs.sort())   # [1, 2, 3]

# Descending order with comparator
desc = xs.sort((a: int, b: int) => a > b)
print(desc)   # [3, 2, 1]
```

Chaining filter, map, sort

These functions return new lists, so they can be chained via UFCS.

```
xs = [5, 3, 1, 4, 2]
result = xs.filter((x: int) => x > 1).map((x: int) => x * 10).sort()
print(result)   # [20, 30, 40, 50]
```

reduce

Reduces a list to a single value using an accumulator function, starting with the first element.

```
xs = [1, 2, 3, 4, 5]
total = reduce(xs, (a: int, b: int) => a + b)
print(total)   # 15
```

fold

Folds a list to a single value using an accumulator function and an explicit initial value.

```
xs = [1, 2, 3, 4, 5]
total = fold(xs, 0, (a: int, b: int) => a + b)
print(total)   # 15
```

any

Returns `true` if at least one element satisfies the predicate.

```
xs = [1, 2, 3, 4, 5]
print(any(xs, (x: int) => x > 4))   # true
print(any(xs, (x: int) => x > 9))   # false
```

all

Returns `true` if every element satisfies the predicate.

```
xs = [2, 4, 6]
print(all(xs, (x: int) => x > 0))   # true
print(all(xs, (x: int) => x > 3))   # false
```

sum

Returns the sum of all elements.

```
xs = [1, 2, 3, 4, 5]
print(sum(xs))   # 15
```

min

Returns the minimum element.

```
xs = [3, 1, 4, 1, 5]
print(min(xs))   # 1
```

max

Returns the maximum element.

```
xs = [3, 1, 4, 1, 5]
print(max(xs))   # 5
```

first

Returns the first element. Returns `None` if the list is empty.

```
xs = [10, 20, 30]
print(first(xs))   # Some(10)
```

last

Returns the last element. Returns `None` if the list is empty.

```
xs = [10, 20, 30]
print(last(xs))   # Some(30)
```

is_empty

Returns `true` if the container has no elements. Accepts lists, maps, sets, and strings.

```
xs = [1, 2, 3]
print(is_empty(xs))        # false
print(is_empty([]))        # true (requires type annotation in practice)
print(is_empty(""))        # true  (str support, #831)
print(is_empty("hello"))   # false
```

enumerate

Returns a list of `(index, element)` tuples.

```
xs = [10, 20, 30]
pairs = enumerate(xs)
# pairs = [(0, 10), (1, 20), (2, 30)]

# Tuple destructuring in for loop
for i, x in enumerate(xs):
    print(f"{i}: {x}")    # 0: 10, 1: 20, 2: 30
```

zip

Combines two lists into a list of `(elem1, elem2)` tuples. The result length equals the shorter list.

```
xs = [1, 2, 3]
ys = ["a", "b", "c"]
pairs = zip(xs, ys)
# pairs = [(1, "a"), (2, "b"), (3, "c")]

# Tuple destructuring in for loop
for a, b in zip(xs, ys):
    print(f"{a}: {b}")    # 1: a, 2: b, 3: c
```

insert

Inserts an element at the specified index. Elements at and after the index are shifted right.

```
xs = [1, 2, 3]
insert(xs, 1, 99)
print(xs)   # [1, 99, 2, 3]
```

remove_at

Removes and returns the element at the specified index. Elements after the index are shifted left.

```
xs = [1, 2, 3, 4]
v = remove_at(xs, 1)
print(v)    # 2
print(xs)   # [1, 3, 4]
```

remove

Removes the first occurrence of the specified value from the list. Does nothing if the value is not found. This is a mutating operation.

```
xs = [1, 2, 3, 2, 4]
remove(xs, 2)
print(xs)   # [1, 3, 2, 4]
```

distinct

Returns a new list with duplicate elements removed. The original order is preserved (first occurrence kept). The original list is not modified.

```
xs = [1, 2, 3, 2, 1, 4]
print(distinct(xs))   # [1, 2, 3, 4]
print(xs)             # [1, 2, 3, 2, 1, 4] (unchanged)
```

flatten

Flattens a nested list (list of lists) by one level. Returns a new list. The original list is not modified.

```
xs = [[1, 2], [3, 4]]
print(flatten(xs))   # [1, 2, 3, 4]
print(xs)            # [[1, 2], [3, 4]] (unchanged)
```

In-Place Mutating Variants

Some list operations have two forms: a non-mutating version that returns a new list, and an in-place variant whose name ends with `!`. Use the `!` form when you intend to mutate the receiver and want to make that intent explicit at the call site; use the non-mutating form when you want to preserve the original.

In-place (`!`)	Non-mutating equivalent	Difference
`append!(xs, v)` / `append(xs, v)`	`appended(xs, v)`	`append!` / `append` mutate in place and return `Unit`; `appended` returns a new list
`sort!(xs)` / `sort!(xs, cmp)`	`sort(xs)` / `sort(xs, cmp)`	`sort!` mutates `xs` in place; `sort` returns a new sorted list
`reverse!(xs)`	`reverse(xs)`	`reverse!` mutates `xs` in place; `reverse` returns a new reversed list

All `!` variants participate in Copy-on-Write: if `xs` is shared (reference count > 1), the outer buffer is cloned once before the in-place mutation so that aliases are not affected (see Copy-on-Write (CoW) Semantics).

```
xs = [3, 1, 2]
sort!(xs)
print(xs)          # [1, 2, 3]

ys = [1, 2, 3]
reverse!(ys)
print(ys)          # [3, 2, 1]

zs = [1, 2]
append!(zs, 3)
print(zs)          # [1, 2, 3]

# Non-mutating variant: appended returns a new list
a = [1, 2]
b = appended(a, 3)
```

```
print(a)          # [1, 2] (unchanged)
print(b)          # [1, 2, 3]
```

When to use which:

- Prefer `sort`, `reverse`, `appended` when readers benefit from seeing that the original is preserved (e.g. functional pipelines, or when the original is still needed afterwards).
- Prefer `sort!`, `reverse!`, `append!` when the original is genuinely being replaced, to avoid the allocation of an intermediate copy and to make the mutation explicit at the call site.

Operation Complexity

Operation	Complexity
`xs[i]` index access	O(1)
`append` / `append!`	O(1) amortized
`pop`	O(1)
`first`, `last`	O(1)
`insert`, `remove_at`	O(n)
`sort` / `sort!`	O(n log n)
`take`	O(n)
`tap`	O(n)
`filter`, `map`, `reduce`, `fold`	O(n)
`reverse` / `reverse!`	O(n)
`distinct`	O(n)
`length`	O(1)

Constraints and Errors

Constraint	Details
All elements must be the same type	Mixed types cause a compile error
Empty list `[]`	Requires type annotation (e.g., `xs: List<int> = []`)
Out-of-range access	Runtime error (exit(1))

Copy-on-Write (CoW) Semantics

All collection types (List, Map, Set) use **Copy-on-Write** semantics when managed by ARC. This means:

- **Assignment shares data**: `b = a` does not copy the collection — both variables reference the same data. The reference count is incremented.
- **Mutation triggers a path-walking copy**: When a shared collection is mutated, every level on the LHS path whose reference count > 1 is cloned before the mutation lands. A top-level write (`b.append(...)`, `b[i] = v` with `b` shared) clones only the outermost container. A chained write (`b[i][j] = v`, `r.items[i] = v`, `m[k1][k2] = v`) walks from the root down and clones each intervening container that is still shared, so aliases of the outer container — or any inner container on the path — are isolated from the mutation.
- **Unique owners mutate in-place**: When a collection (or the container at some hop) has only one reference (`strong_count == 1`), the CoW check skips the clone for that level and mutates in-place with zero copy overhead.

```
a = [1, 2, 3]        # strong_count = 1
b = a                 # strong_count = 2 (shared)
```

```
append(b, 4)          # strong_count > 1 → copy outer buffer, then mutate
                      # a = [1, 2, 3]  (strong_count = 1)
                      # b = [1, 2, 3, 4]  (strong_count = 1, new allocation)

c = [10, 20]          # strong_count = 1
append(c, 30)         # strong_count == 1 → mutate in-place (no copy)
```

Path Copy-on-Write

Path CoW handles chained writes through nested collections — the writer walks from the LHS root variable (or record field) down to the leaf write site and privatizes each level whose reference count is greater than one. This gives strict aliasing isolation between aliases sharing the outer container and any inner container on the write path.

```
a = [[1, 2], [3, 4]]
b = a                       # outer list shared: strong_count = 2
b[0][0] = 99                # walks: clone outer (a and b now have their own
                            # outer); clone b's inner[0] (a's inner[0] keeps [1, 2]);
                            # write 99 into the clone
# a = [[1, 2], [3, 4]]
# b = [[99, 2], [3, 4]]
```

Record fields with ARC-managed collection values participate in path CoW the same way as direct index hops. Record-to-record assignment (`r2 = r1`) retains each ARC field so a subsequent `r2.items[i] = v` observes the refcount > 1 at the field slot and clones before mutating:

```
record Box:
  items: List<int>

r1 = Box([1, 2, 3])
r2 = r1
r2.items[0] = 99
# r1.items[0] == 1
# r2.items[0] == 99
```

Not supported as the root of a path-CoW chain: method-call lvalues (`f().x[i] = v`) and chains that interleave an index hop inside a record field walk (`rec.arr[0].items[i] = v`). These shapes produce a compile-time error; assign the intermediate value to a local variable first.

Operations that trigger CoW

Type	Mutating operations
List	`append()`, `pop()`, `insert()`, `remove()`, `remove_at()`, `sort!()`, `reverse!()`, index assignment (`xs[i] = val`)
Map	`remove()`, index assignment (`m[key] = val`)
Set	`add()`, `remove()`

Non-mutating operations (`appended`, `slice`, `take`, `filter`, `map`, `sort`, `reverse`, `get`, `items`, etc.) create new collections and do not trigger CoW.

Map

Overview

A key-value mapping. Allocated on the heap.

Syntax

```
m = {"a": 1, "b": 2}
m: Map<str, int> = {"a": 1, "b": 2}
```

Equality

Maps support `==` and `!=`. Two maps are equal when they have the same number of entries and every key-value pair in one map exists with an equal value in the other.

```
{"a": 1, "b": 2} == {"a": 1, "b": 2}   # true
{"a": 1}         != {"a": 2}            # true (different values)
{"a": 1}         != {"b": 1}            # true (different keys)
```

Key Access

```
m = {"a": 1, "b": 2}
print(m["a"])   # 1
```

Insert and Update

```
m = {"a": 1}
m["b"] = 2     # Insert new entry
m["a"] = 99    # Update existing entry
```

length

```
m = {"a": 1, "b": 2, "c": 3}
print(length(m))   # 3
```

print

```
m = {"a": 1, "b": 2}
print(m)   # {a: 1, b: 2}
```

has_key

```
m = {"a": 1, "b": 2}
print(m.has_key("a"))   # true
print(m.has_key("z"))   # false
```

keys

Returns a list of all keys in the map.

```
m = {"a": 1, "b": 2, "c": 3}
print(keys(m))   # ["a", "b", "c"]
```

values

Returns a list of all values in the map.

```
m = {"a": 1, "b": 2, "c": 3}
print(values(m))   # [1, 2, 3]
```

items

Returns a list of `(key, value)` tuples for all entries in the map.

```
m = {"a": 1, "b": 2}
pairs = items(m)
# pairs = [("a", 1), ("b", 2)]
```

remove (Map)

Removes the entry with the specified key from the map. Does nothing if the key does not exist.

```
m = {"a": 1, "b": 2}
remove(m, "a")
print(m)   # {b: 2}
```

get

Returns the value for the specified key, or a default value if the key does not exist.

```
m = {"a": 1, "b": 2}
print(get(m, "a", 0))   # 1
print(get(m, "z", 0))   # 0
```

merge

Returns a new map that combines all entries from both maps. When keys overlap, values from the second map take precedence. The original maps are not modified.

```
m1 = {"a": 1, "b": 2}
m2 = {"b": 99, "c": 3}
m3 = merge(m1, m2)
print(m3["a"])   # 1
print(m3["b"])   # 99
print(m3["c"])   # 3
```

Constraints and Errors

Constraint	Details
All keys must be the same type	Mixed key types cause a compile error
All values must be the same type	Mixed value types cause a compile error
Empty map	Type annotation is required (e.g., `m: Map<str, int> = {"a": 1}`)
Accessing a non-existent key	Runtime error (exit(1))
Key lookup	Hash table (O(1) average)
Capacity overflow	Automatically doubles in size

Set

Overview

A collection that holds elements of the same type without duplicates. Allocated on the heap.

Syntax

```
s = {1, 2, 3}
s: Set<int> = {1, 2, 3}
```

Supported Element Types

`int`, `float`, `bool`, `str`

Equality

Sets support `==` and `!=`. Equality is order-independent — two sets are equal when they contain exactly the same elements.

```
{1, 2, 3} == {3, 2, 1}   # true (order does not matter)
{1, 2}    != {1, 2, 3}   # true (different sizes)
```

in Operator (Membership Check)

```
s = {1, 2, 3}
print(2 in s)   # true
print(5 in s)   # false
```

length

```
s = {1, 2, 3}
print(length(s))   # 3
```

print

```
s = {1, 2, 3}
print(s)   # {1, 2, 3}
```

add (Add Element)

Duplicate elements are ignored when added.

```
s = {1, 2, 3}
s.add(4)          # Add
s.add(1)          # Ignored because it already exists
print(length(s))    # 4
```

remove (Remove Element)

```
s = {1, 2, 3}
s.remove(2)
print(2 in s)   # false
```

for Iteration

```
s = {10, 20, 30}
for x in s:
    print(x)
```

Empty Set

An empty set requires a type annotation.

```
s: Set<int> = {}
```

Function Parameters

```
function has_value(s: Set<int>, v: int) -> bool:
    return v in s
```

union

Returns a new set containing all elements from both sets.

```
a = {1, 2, 3}
b = {3, 4, 5}
print(union(a, b))   # {1, 2, 3, 4, 5}
```

intersection

Returns a new set containing only elements present in both sets.

```
a = {1, 2, 3}
b = {2, 3, 4}
print(intersection(a, b))   # {2, 3}
```

difference

Returns a new set containing elements in the first set but not in the second.

```
a = {1, 2, 3}
b = {2, 3, 4}
print(difference(a, b))   # {1}
```

symmetric_difference

Returns a new set containing elements that are in either set but not in both.

```
a = {1, 2, 3}
b = {2, 3, 4}
print(symmetric_difference(a, b))   # {1, 4}
```

is_subset

Returns `true` if all elements of the first set are contained in the second set.

```
a = {1, 2}
b = {1, 2, 3}
print(is_subset(a, b))   # true
print(is_subset(b, a))   # false
```

is_superset

Returns `true` if the first set contains all elements of the second set.

```
a = {1, 2, 3}
b = {1, 2}
print(is_superset(a, b))   # true
print(is_superset(b, a))   # false
```

Constraints and Errors

Constraint	Details
All elements must be the same type	Mixed types cause a compile error
Empty set `{}`	Type annotation is required
Element lookup	Hash table (O(1) average)
Capacity overflow	Automatically doubles in size

Iterator

Overview

A lazy iterator abstraction that enables efficient data transformation pipelines. Iterators do not copy or materialize intermediate results —each element is processed on demand.

Creating Iterators

Use `iter()` to create an iterator from any collection:

```
xs = [1, 2, 3, 4, 5]
it = xs.iter()            # Iterator<int>

s = {10, 20, 30}
sit = s.iter()            # Iterator<int>

m = {"a": 1, "b": 2}
mit = m.iter()            # Iterator<(str, int)>
```

Lazy Method Chaining

Iterator methods return new iterators, forming a pipeline that is only evaluated when consumed:

Method	Description
`.filter(function)`	Yields only elements where the predicate returns `true`
`.map(function)`	Transforms each element using the given function
`.take(count)`	Yields at most `count` elements

```
result = [1, 2, 3, 4, 5]
    .iter()
    .filter((x: int) => x > 2)
    .map((x: int) => x * 2)
    .take(2)
    .to_list()   # [6, 8]
```

Consuming Iterators

Method	Description
`.to_list()`	Collects all elements into a `List<T>`
`.next()`	Returns the next element as `Option<T>`

```
it = [10, 20].iter()
print(it.next())   # Some(10)
print(it.next())   # Some(20)
print(it.next())   # None
```

For Loop Support

Iterators can be used directly in `for` loops:

```
for x in [1, 2, 3].iter().filter((x: int) => x > 1):
    print(x)
# 2
# 3

for k, v in {"a": 1, "b": 2}.iter():
    print(k)
```

English | 日本語 | 繁體中文

Design by Contract (DbC)

Ry supports Eiffel-style Design by Contract with preconditions (`require`), postconditions (`ensure`), and record invariants (`invariant`). Contract violations terminate the process with `exit(1)`.

Preconditions (`require`)

Preconditions are checked at function entry. They specify what must be true for the function to be called correctly.

```
function deposit(amount: int, balance: int) -> int:
    require:
        amount > 0
        balance >= 0
    new_balance: int = balance + amount
    return new_balance
```

If any precondition fails, the program terminates with:

```
Contract violation: require failed in deposit()
```

Postconditions (`ensure`)

Postconditions are checked before every `return`. They specify what the function guarantees about its result.

Variable binding

`ensure` requires a variable name that binds the return value. This variable can be used in the postcondition expressions.

```
function abs(x: int) -> int:
    ensure v:
        v >= 0
    if x < 0:
        return -x
    return x
```

Since function arguments are immutable in Ry, you can reference them directly in `ensure` blocks to compare with entry values:

```
function increment(x: int) -> int:
    ensure v:
        v == x + 1
    return x + 1
```

Tuple destructuring

For functions that return tuples, multiple variable names can be specified, separated by commas:

```
function divide(a: int, b: int) -> (int, int):
    ensure q, r:
        q >= 0
        r >= 0
    return (a // b, a % b)
```

The number of binding variables must match the number of tuple elements.

Combined Example

```
function deposit(amount: int, balance: int) -> int:
    require:
        amount > 0
        balance >= 0
    ensure v:
        v >= 0
        v == balance + amount
    new_balance: int = balance + amount
    return new_balance
```

Record Invariants (`invariant`)

Invariants are conditions that must always hold for a record instance. They are checked: - After construction - After every field assignment

```
record BankAccount:
    balance: int
    min_balance: int
    invariant:
        balance >= min_balance

a = BankAccount(100, 0)     # OK: 100 >= 0
a.balance = -1                  # Contract violation: invariant failed
```

Rules

- `require` and `ensure` blocks are optional and appear before the function body.
- `require` must come before `ensure` when both are present.
- `ensure` requires a variable binding (e.g., `ensure v:`) to name the return value.
- For tuple returns, multiple bindings can be specified (e.g., `ensure q, r:`).
- `invariant` appears at the end of a `record` definition, after all field declarations.
- All contract violations terminate with `exit(1)` and print a diagnostic message.

English | 日本語 | 繁體中文

Control Flow Reference

if / else

Statement Syntax

```
if condition:
    # then block
else:
    # else block (optional)
```

Expression Forms

`if` can also be used as an expression that produces a value. Two forms are supported:

Single-expression form (`=>`):

```
x = if condition => true_value else false_value
```

Examples:

```
abs_val = if x > 0 => x else -x
label = if score >= 90 => "A" else "B"
```

The `else` branch in the single-expression form takes a value directly (without `=>`). Both branches must produce the same type, and `else` is required.

Block form (`:`):

```
x = if condition:
    compute_something()
else:
    compute_other()
```

In the block form, each block must end with an expression statement (tail-expression semantics). The `else` branch is required, and both branches must produce the same type.

For multi-branch conditionals with values, use `case:` instead (see below).

Condition Types

Type	Falsy Value	Truthy Value
`bool`	`false`	`true`
`int`	`0`	non-zero
`float`	`0.0`	non-zero

Only `bool`, integer, and `float` types may appear in a condition. `str`, `List`, `Map`, `Set`, iterators, closures, records, `Option`, and `Result` cannot be used directly as conditions. For collections and strings, write the length check explicitly:

```
xs = [1, 2, 3]
# ☒ error: value of this type cannot be used as a boolean condition
# if xs:
#     print("non-empty")
# ✓ explicit length check
if length(xs) > 0:
    print("non-empty")
# ✓ equivalent using is_empty
```

```
if not is_empty(xs):
    print("non-empty")
```

For `Option` and `Result`, pattern-match the variants explicitly with `case` instead of using them as conditions. These rules apply equally to `while`, `case` arms, and the unary `not` operator.

Example

```
x = 10

if x > 5:
    print("big")
else:
    print("small or equal")
```

Scope Rules

- Each `if` / `else` block has its own independent block scope.
- Variables declared inside a block are not accessible outside the block.

```
if true:
    y = 42
# y is not accessible here
```

---

while

Syntax

```
while condition:
    # loop body
```

Repeats the loop body while the condition is `true`.

Example

```
i = 0
while i < 5:
    print(i)
    i += 1
```

Combining with break / continue

```
i = 0
while true:
    if i >= 3:
        break
    i += 1
```

---

for

Syntax

```
# List / set iteration
for x in iterable_expr:
    # x is assigned each element
```

```
# range (starting from 0)
for i in range(n):
    # i = 0, 1, ..., n-1

# range (with start and end)
for i in range(start, end):
    # i = start, start+1, ..., end-1

# range (with step)
for i in range(start, end, step):
    # i = start, start+step, start+2*step, ...
```

String Iteration

A `for` loop over a `str` yields each **Unicode code point** as a single-character `str`. Multi-byte UTF-8 sequences (including CJK characters and emoji) are decoded correctly; bytes within a multi-byte character are never split.

This is **code-point** iteration, not **grapheme-cluster** iteration: user-perceived characters that span multiple code points —combining-mark sequences (e.g., base letter + U+0301) and ZWJ emoji sequences (e.g., family or skin-tone compositions) — are yielded as several iterations, one per code point. If you need grapheme-cluster-aware iteration, decompose the string with a future segmentation helper rather than relying on `for c in s:`.

```
for c in "hello":
    print(c)                # h, e, l, l, o

for c in "こんにちは":
    print(c)                # こ, ん, に, ち, は  (not individual bytes)

for c in "a b":
    print(c)                # a,  , b
```

The loop variable is typed as `str`, so you can pass it to other string functions:

```
for c in "abc":
    print(to_upper(c))      # A, B, C
```

Iterating an empty string runs the loop body zero times. `enumerate` and `zip` also accept `str` arguments and yield the same code-point units:

```
for i, c in enumerate("abc"):
    print(i, c)

for a, b in zip("abc", "xyz"):
    print(a + b)            # ax, by, cz
```

Map Key-Value Iteration

```
for k, v in map_expr:
    # k is the key, v is the value for each entry
```

Tuple Destructuring

When iterating over a list of tuples, you can destructure into N variables matching the tuple' s element count. Use `_` to discard a value.

```python
xs = [10, 20, 30]

for i, x in enumerate(xs):
    print(f"{i}: {x}")    # 0: 10, 1: 20, 2: 30

for a, b in zip([1, 2], [10, 20]):
    print(a + b)          # 11, 22

for _, x in enumerate(xs):
    print(x)              # index discarded

# N-element destructuring (3+ variables)
triples = [(1, 2, 3), (4, 5, 6)]
for a, b, c in triples:
    print(a + b + c)      # 6, 15

for a, _, c in triples:
    print(a + c)          # 4, 10 (middle element discarded)
```

Range Operator (..)

The `..` operator creates an inclusive integer range. `1 .. 5` produces `[1, 2, 3, 4, 5]`.

```python
for i in 1 .. 5:
    print(i)     # 1 2 3 4 5
```

Example

```python
xs = [10, 20, 30]
for x in xs:
    print(x)

s = {1, 2, 3}
for x in s:
    print(x)

for i in range(5):
    print(i)     # 0 1 2 3 4

for i in range(2, 6):
    print(i)     # 2 3 4 5

for i in range(0, 10, 2):
    print(i)     # 0 2 4 6 8

for i in range(10, 0, -3):
    print(i)     # 10 7 4 1

# Map iteration
m = {"a": 1, "b": 2}
for k, v in m:
    print(k)
    print(v)

# Range operator
```

```
for i in 1 .. 3:
    print(i)     # 1 2 3
```

---

async / await

`async function` declares a function that runs concurrently. Calling an `async function` returns `Task<T>`. Use `await` inside another `async function` or `block_on()` from synchronous context to wait for the result.

```
async function add(a: int, b: int) -> int:
    return a + b

# From synchronous context, use block_on()
t: Task<int> = add(20, 22)
print(block_on(t))                  # 42
print(block_on(add(1, 2)))          # 3

# Inside async function, use await
async function double_add(a: int, b: int) -> int:
    result = await add(a, b)
    return result * 2
```

Rules

- `async function name(...) -> T:` is declared with the awaited result type `T`.
- Calling an `async function` immediately returns `Task<T>`.
- `await expr` requires `expr` to be `Task<T>` and produces `T`.
- `await` can only be used inside an `async function`. Use `block_on(task)` from synchronous context.
- `block_on(task)` blocks the current thread until the task completes and returns the result.
- `async function ... -> Unit` is supported; `block_on(task)` is the primary way to wait when no value is produced.
- Tasks run on the runtime worker pool; they are not implemented as one OS thread per task.
- `async` lambdas and `async @native function` are not supported in v1.

@parallel for

`@parallel` can be attached only to counted `for` loops over `range(...)` or integer `..` ranges. The loop body runs in parallel chunks on the runtime worker pool.

```
@parallel
for i in range(8):
    print(i)
```

Constraints

- Only `range(...)` and integer `..` loops are supported.
- Destructuring iteration is not supported.
- Assigning to outer mutable bindings is rejected.
- `break` and `continue` are rejected.
- Indexed assignment and field assignment inside the loop body are rejected in v1.

Use `available_parallelism()` to inspect the runtime worker count.

---

break

- Immediately exits the innermost loop (`while` or `for`).
- Using it outside a loop causes a compile error.

```
for i in range(10):
    if i == 5:
        break    # Exits when i == 5
    print(i)     # 0 1 2 3 4
```

Error Example

```
# break outside a loop is a compile error
break   # Error: break outside loop
```

---

continue

- Ends the current iteration of the innermost loop and skips to the next iteration.
- Using it outside a loop causes a compile error.

```
for i in range(5):
    if i == 2:
        continue   # Skip i == 2
    print(i)       # 0 1 3 4
```

... (Ellipsis)

- A no-op statement that does nothing. Used as a placeholder for empty blocks.
- Can be used in any block: function body, `if`/`else`, `while`, `for`, `case` arm, etc.

```
function not_yet():
    ...

if true:
    ...
else:
    ...
```

---

case

`case` unifies multi-branch conditional flow (formerly `when`) and pattern matching (formerly `match`) into a single construct. Two forms are supported:

- `case:` — no subject, each arm is a condition expression (replaces `when:`)
- `case <expr>:` — with a subject, each arm is a pattern (replaces `match`)

Both forms support a block body (`:`) and an expression body (`=>`).

> **Note**: The `when` and `match` keywords were removed in favor of the unified `case` construct. Legacy Ry code using `when` / `match` must be migrated.

case without subject

Use `case:` for multi-branch conditional flow without a subject value.

Syntax

```
case:
    condition:
        # body
    condition:
        # body
    _:
        # fallback body
```

Example

```
x = 0

case:
    x > 0:
        print("positive")
    x < 0:
        print("negative")
    _:
        print("zero")
```

The arms are evaluated from top to bottom and the first arm whose condition is truthy is executed. The wildcard arm `_:` is optional for statements.

For the expression form of `case:`, see the Expression Forms section below.

---

case with subject (pattern matching)

Syntax

```
case expression:
    pattern:
        # body
    pattern if guard_condition:
        # guarded body
    _:
        # wildcard (matches anything)
```

Pattern Types

Pattern	Example	Description
Wildcard	`_`	Matches anything
Literal	`0`, `"hello"`, `true`	Equality comparison
Variable binding	`n`	Matches anything and binds to a variable
enum variant	`Color::Red`	Compares enum tag (simple enum)
ADT enum variant	`Shape::Circle(r)`	Matches an enum variant with associated data and binds it
`Some(x)`	`Some(v)`	When Option has a value, binds the inner value
`None`	`None`	When Option has no value

Pattern	Example	Description
`Ok(x)`	`Ok(v)`	When Result is Ok, binds the inner value
`Err(x)`	`Err(e)`	When Result is Err, binds the error value
OR pattern	`1 | 2 | 3`	Matches if any alternative matches

Guard Clause

A guard condition can be specified in the form `pattern if condition:`. The arm is executed only when the pattern matches and the guard condition is true.

OR Pattern

Multiple patterns can be combined with `|` to match any of them. Variable bindings (`n`, `Some(x)`, `Ok(v)`, `Err(e)`) are not allowed in OR patterns.

```
case x:
    1 | 2 | 3:
        print("small")
    _:
        print("other")

# Enum OR pattern
case color:
    Color::Red | Color::Blue:
        print("warm or cool")
    Color::Green:
        print("green")
```

Exhaustiveness Checking

- enum types: Must cover all variants or include `_`. OR patterns count each alternative individually.
- Option types: Must cover both `Some` and `None` or include `_`.
- bool type: Must cover both `true` and `false` or include `_`.
- int / float / str literals: `_` is required.
- Guarded arms do not count toward exhaustiveness.

Example

```
# enum pattern match
enum Color:
    Red
    Green
    Blue

case color:
    Color::Red:
        print("red")
    Color::Green:
        print("green")
    Color::Blue:
        print("blue")
```

```
# Option pattern match
x: Option<int> = Some(42)
case x:
    Some(v):
        print(v)
    None:
        print("nothing")

# Result pattern match
function divide(a: int, b: int) -> Result<int, Error>:
    if b == 0:
        return Err(Error("division by zero"))
    return Ok(a // b)

case divide(10, 2):
    Ok(v):
        print(v)          # 5
    Err(e):
        print(e.message)

# Literal pattern match
case x:
    0:
        print("zero")
    1:
        print("one")
    _:
        print("other")

# Guard clause
case x:
    n if n > 0:
        print("positive")
    n if n < 0:
        print("negative")
    _:
        print("zero")
```

ADT Enum Pattern Matching

When an enum variant carries associated data, use a binding pattern to extract the value(s).

```
enum Shape:
    Circle(float)
    Rectangle(float, float)
    Point

s = Shape::Circle(3.14)
case s:
    Shape::Circle(r):
        print(r)        # 3.14
    Shape::Rectangle(w, h):
        print(w)
        print(h)
    Shape::Point:
```

```
        print("point")
```

Multi-field variants bind each field to a separate name in declaration order.

Expression Forms

Both `case:` and `case <expr>:` can be used as expressions by replacing `:` with `=>` in each arm. Each arm provides a single expression whose value becomes the result.

```
# case: expression (no subject)
label = case:
    x > 100 => "huge"
    x > 10  => "big"
    x > 0   => "small"
    _       => "non-positive"
```

Pattern-matching expression form:

Syntax

```
result = case expression:
    pattern => value_expression
    pattern if guard => value_expression
    _ => default_value
```

All patterns supported in match statements are also supported in match expressions: literals, variable bindings, enums, ADT enums, `Some`/`None`, `Ok`/`Err`, OR patterns, guards, and wildcards.

Match expressions must be exhaustive (same rules as match statements).

Examples

```
# Option
value = case opt:
    Some(v) => v
    None    => 0

# Enum
label = case direction:
    Direction::North => "N"
    Direction::South => "S"
    Direction::East  => "E"
    Direction::West  => "W"

# Guard
grade = case score:
    n if n >= 90 => "A"
    n if n >= 80 => "B"
    _            => "F"

# OR pattern
kind = case x:
    1 | 2 | 3 => "small"
    _         => "large"

# ADT enum
area = case shape:
    Shape::Circle(r)  => 3.14 * r * r
```

```
    Shape::Rect(w, h) => w * h
    Shape::Point      => 0.0
```

Scope Rules

- Each `case` arm has its own block scope.
- Variables bound by variable binding patterns (`n`), `Some(x)`, `Ok(v)`, or `Err(e)` are only valid within that arm.

Scope Rules

Block Scope

- Each block of `if` / `else` / `while` / `for` / `case` has a block scope.
- Variables declared inside a block go out of scope when the block ends.

```
for i in range(3):
    tmp = i * 2
# tmp is not accessible here

if true:
    a = 1
# a is not accessible here
```

Inner Scope Reassignment

- Assigning to a variable inside an inner scope modifies the outer variable (Python-style scoping).
- There is no shadowing — the inner assignment changes the same variable.

```
x = 10
if true:
    x = 99   # Modifies the outer x
    print(x)     # 99
print(x)         # 99
```

English | 日本語 | 繁體中文

Directives

Directives are compile-time metadata annotations that can be attached to declarations. They use the `@name` syntax, similar to Java annotations.

Syntax

```
@name
@name(key=value, ...)
```

Directives are placed before the target declaration. Multiple directives can be stacked.

Supported Targets

Directives can be applied to the following declarations:

- `function` - Function definitions (including named test functions decorated with `@it` / `@describe`)
- `record` - Record definitions
- Variable declarations (with or without `@const`)

- Fields within a `record` definition
- `for` - Counted loops only for `@parallel`
- `it` / `describe` calls (legacy lambda form) - Test cases and test groups for `@each` and `@property`

Built-in Directives

`@deprecated`

Marks a declaration as deprecated. When a deprecated entity is used (called, referenced, or accessed), a compile-time warning is emitted.

On functions:

```
@deprecated
function old_function() -> int:
    return 42

print(old_function())   # warning: 'old_function' is deprecated
```

On types:

```
@deprecated
record OldPoint:
    x: int
    y: int

@const
p = OldPoint(1, 2)  # warning: 'OldPoint' is deprecated
```

On variables:

```
@deprecated
@const
old_value = 99

print(old_value)         # warning: 'old_value' is deprecated
```

On fields:

```
record Config:
    @deprecated
    old_setting: int
    new_setting: int

@const
c = Config(1, 2)
print(c.old_setting)     # warning: 'Config.old_setting' is deprecated
print(c.new_setting)     # no warning
```

`@const`

Marks a variable as immutable. Variables declared with `@const` cannot be reassigned after initialization. Without `@const`, variables are mutable by default.

```
@const
x = 42
# x = 10   # Error: cannot reassign @const variable
```

With type annotation:

```
@const
name: str = "hello"
```

Tuple destructuring:

```
@const
a, b = (1, 2)
```

Top-level `@const` and functions. A top-level `@const` declaration is visible from any top-level function defined after it in the same source file, and the immutability is enforced for every reference — including field mutations through a top-level `@const` record. See the "Top-Level Variables and `@const` in Function Bodies" section in functions.md for details.

`@native`

Declares a function whose implementation is provided by the runtime. The function must not have a body.

An optional string argument specifies the shared library module name. When a `@native("libname")` function is called, the JIT dynamically loads the corresponding shared library (`libry_<libname>.dylib` on macOS, `libry_<libname>.so` on Linux) and resolves the runtime symbol from it:

```
@native              # built-in (statically linked into the process)
@native("base64")    # dynamically loaded from libry_base64.dylib/.so
```

Basic syntax:

```
@native
function contains(string: str, substring: str) -> bool

print(contains("hello world", "world"))  # true
```

Operator overloads:

```
@native
function operator+(a: str, b: str) -> str

print("hello" + " world")  # hello world
```

UFCS-compatible:

```
@native
function to_upper(string: str) -> str

print("hello".to_upper())  # HELLO
```

Argument count validation:

When a `@native` declaration includes a type signature, the compiler validates the number of arguments at call sites. Overloaded functions (e.g., `range` with 1, 2, or 3 arguments) are supported — any matching overload passes validation.

```
@native
function range(n: int) -> List<int>
@native
function range(start: int, end: int) -> List<int>

print(length(range(5)))       # OK: matches 1-arg overload
print(length(range(1, 10)))   # OK: matches 2-arg overload
print(length(range()))        # Error: expects 1 or 2 argument(s), but got 0
```

Standard library declarations (`core/`):

The `core/` directory contains `@native` declarations for all built-in functions, organized by category:

File	Contents
`core/builtins.ry`	`print`, `length`, `range`, `enumerate`, `zip`, `exit`, `args`, `available_parallelism`, `sleep`
`core/str.ry`	`contains`, `starts_with`, `ends_with`, `find`, `substring`, `char_at`, `replace`, `to_upper`, `to_lower`, `trim`, `trim_start`, `trim_end`, `repeat`, `reverse`, `split`, `join`
`core/convert.ry`	`to_int`, `to_float`, `to_str`
`core/list.ry`	`append`, `pop`, `insert`, `remove_at`, `slice`, `distinct`, `flatten`, `sort`, `first`, `last`, `is_empty`
`core/map.ry`	`keys`, `values`, `items`, `has_key`, `get`, `merge`
`core/set.ry`	`add`, `remove`, `union`, `intersection`, `difference`, `symmetric_difference`, `is_subset`, `is_superset`
`core/higher_order.ry`	`filter`, `map`, `reduce`, `fold`, `any`, `all`, `sum`, `min`, `max`

These files are automatically loaded as a prelude when the `core/` directory is found relative to the `ry` executable. The prelude enables argument count validation for built-in function calls.

Constraints: - `@native` functions must not have a body (no `:` after the signature). - Providing a body causes a parse error: `@native function must not have a body`. - For bare `@native`, the declared function must correspond to an existing built-in; otherwise the call will fail at compile time. For `@native("libname")`, the function is compiled based on the declared signature and will fail at JIT link time if the symbol cannot be resolved from the loaded library.

Library specification: - `@native("libname")` specifies that the native function lives in a shared library named `libry_<libname>.dylib` (macOS) or `libry_<libname>.so` (Linux). At JIT startup, the required shared libraries are loaded from the following search paths (in order): 1. `exe/../lib/` — installed layout 2. `exe/lib/` — development/build layout 3. `$RY_HOME/lib/` — user-installed environment - Both `@native` (static) and `@native("libname")` (dynamic) declarations register for argument-count validation and call resolution. The difference is only in how the runtime symbol is provided to the JIT. - The runtime function name follows the convention `__ry_<libname>_<fn_name>` (e.g., `@native("base64") fn encode(...)` → `__ry_base64_encode`). This works for both stdlib packages and user-defined native libraries.

`@parallel`

Marks a counted `for` loop for parallel execution.

```
@parallel
for i in range(8):
    work(i)
```

Supported target:

- `for` statements only

Constraints:

- Only a single `@parallel` directive is allowed on a `for` statement.
- The iterable must be `range(...)` or an integer `..` range.
- Destructuring iteration is not supported.
- Assigning to outer mutable variables is rejected.
- `break`, `continue`, indexed assignment, and field assignment inside the loop body are rejected in v1.

`@each`

Enables parameterized testing by running a test multiple times with different parameters.

Syntax (on a named function, preferred):

```
@each([(arg1, arg2, ...), ...])
@it("should handle {0} and {1}")
function test_handle(param1: type, param2: type):
    # test body
```

Syntax (on a legacy `it` lambda):

```
@each([(arg1, arg2, ...), ...])
it("should handle {0} and {1}", (param1: type, param2: type):
    # test body
)
```

The argument can be any expression that evaluates to a list of tuples, including a function call:

```
@each(make_inputs())
@it("should handle {0}")
function test_handle(x: int):
    # test body
```

Supported targets: functions decorated with `@it`, or legacy `it` calls.

Constraints: - The argument must evaluate to a list of tuples - Tuple arity must match the function parameter count - Placeholders `{0}`, `{1}`, ⋯ in the description string are replaced with stringified values

`@property`

Enables property-based testing by generating random inputs for a test.

Syntax (on a named function, preferred):

```
@property(count=100)
@it("should verify property name")
function test_property(a: int, b: int):
    # test body with random values
```

Syntax (on a legacy `it` lambda):

```
@property(count=100)
it("should verify property name", (a: int, b: int):
    # test body with random values
)
```

Supported targets: functions decorated with `@it`, or legacy `it` calls.

Parameters:

Parameter	Type	Default	Description
`count`	int	100	Number of random trials

Supported parameter types:

Type	Range
`int`	-1000 to 1000
`float`	-1000.0 to 1000.0

Type	Range
`bool`	true or false
`str`	Random ASCII, 0-20 characters

On failure, the counterexample (parameter values that caused the failure) is printed.

`@it`

Declares a test case by decorating a named function. The function body becomes the test body and is executed by `ry test`. See Testing Reference for the full specification.

Syntax:

```
@it("description")
function test_name():
    # assertions
```

Basic example:

```
@it("should add 1 + 2 = 3")
function test_add():
    expect(1 + 2).to_eq(3)
```

Composed with `@each` or `@property`:

```
@each([(1, 2, 3), (0, 0, 0), (-1, 1, 0)])
@it("should add {0} + {1} = {2}")
function test_add_each(a: int, b: int, expected: int):
    expect(a + b).to_eq(expected)

@property(count=100)
@it("should verify addition is commutative")
function test_commutative(a: int, b: int):
    expect(a + b).to_eq(b + a)
```

Supported target: `function` declarations only. The function must not have a return type annotation.

Constraints: - Only valid in `*.test.ry` files executed with `ry test` - When combined with `@each`, the function' s parameter list must match the tuple arity - When combined with `@property`, each parameter type must be one of the supported generator types (`int`, `float`, `bool`, `str`)

`@describe`

Groups a set of related tests by decorating a named function. Inner `@it` functions declared in the body belong to the group, and variables declared directly in the body act as shared setup captured by every inner `@it`. Unlike the legacy lambda form, `@describe` groups **may be nested**; output is indented proportionally to nesting depth.

Syntax:

```
@describe("group name")
function group_name():
    @it("nested test")
    function test_nested():
        # assertions
```

Basic example:

```
@describe("arithmetic")
function arithmetic_tests():
    @it("should subtract")
    function test_sub():
        expect(10 - 3).to_eq(7)

    @it("should multiply")
    function test_mul():
        expect(4 * 5).to_eq(20)
```

Shared setup:

Variables declared in the outer `@describe` body are automatically captured by every inner `@it` function.

```
@describe("shared setup")
function shared_setup_tests():
    base = 100
    offset = 5

    @it("should use base")
    function test_base():
        expect(base).to_eq(100)

    @it("should use base and offset")
    function test_combined():
        expect(base + offset).to_eq(105)
```

Nested groups:

```
@describe("outer")
function outer():
    @describe("inner")
    function inner():
        @it("should pass deeply nested test")
        function test_deep():
            expect(1 + 1).to_eq(2)
```

Supported target: `function` declarations only. The function must not have parameters or a return type annotation.

`@inline`

Provides inlining hints to the LLVM optimizer. By default, marks the function for aggressive inlining.

Basic usage (always inline):

```
@inline
function add(a: int, b: int) -> int:
    return a + b
```

With mode parameter:

```
@inline(mode="always")
function hot_path(x: int) -> int:
    return x * 2 + 1

@inline(mode="hint")
function medium_path(x: int) -> int:
    return x + 1
```

```
@inline(mode="never")
function cold_error_handler(msg: str):
    print("ERROR: " + msg)
```

Modes:

Mode	LLVM Attribute	Description
`always` (default)	`AlwaysInline`	Always inline this function
`hint`	`InlineHint`	Suggest inlining to the optimizer
`never`	`NoInline`	Never inline this function

Constraints: - `@inline` cannot be used with `@native` (native functions have no body to inline). - An unknown mode value causes a compile error.

Parameters (future extension)

Directives support an optional parameter syntax for future extensions:

```
@deprecated(reason="use new_api instead")
function old_api() -> int:
    return 0
```

Currently, parameters are parsed but not used by the `@deprecated` directive.

Notes

- Deprecated entities still function normally; only a warning is emitted.
- Warnings are emitted at the point of use, not at the definition.
- Defining a deprecated entity without using it produces no warnings.
- Unknown directive names cause a parse error.
- Directives on unsupported targets (e.g., `if`, `while`) cause a parse error. `@parallel` is the only directive supported on `for`.

English | 日本語 | 繁體中文

Error Reference

Error Format

ry displays compile errors in a Rust-inspired rich format that shows the exact location of the error with source context:

```
error: cannot reassign @const variable: x
  --> main.ry:5:1
  |
5 | x = 10
  | ^ cannot reassign @const variable: x
```

Each error message includes: - **Error level and message** (`error: ...`) - **File location** (`--> file:line:col`) - **Source line** with the relevant code - **Caret indicator** (`^`) pointing to the exact column

Compile Errors

Error	Cause	Example
Use of undeclared variable	Referenced a variable that has not been declared	`print(x)` (`x` is undeclared)
Reassignment to @const variable	Reassigned to a variable declared with `@const`	`@const x = 1 -> x = 2`
Redeclaration of same-named variable	Redeclared a variable with the same name in the same scope	`x = 1` -> another declaration of `x`
Type-changing reassignment	Assigned a value of a different type to a variable	`x = 1 -> x = 3.14`
Type annotation mismatch	Declared type and assigned value type differ	`x: int = 3.14`
Overload return type conflict	Defined overloads with the same parameter types but different return types only	Two functions with parameters `(int, int)` returning `int` and `float`
Overload resolution failure	No overload matches the argument types	`function add(a: int, b: int)` called with `add(1.0, 2.0)`
Float in bitwise operation	Passed `float` type to `&`, `|`, `^`, `~`, `<<`, `>>`	`3.14 & 1`
Empty list	Cannot infer type from empty list literal `[]`	`xs = []`
Empty map	Cannot infer type from empty map literal `{}`	`m = {}`
Tuple out-of-range index	Accessed a non-existent index on a tuple	`t = (1, 2) -> t.2`
break/continue outside loop	Used `break` or `continue` outside a `for`/`while` loop	`break` at function top level
Module import inside block	Used `from` statement inside a function or conditional block	`from math` inside a function
Circular import	Modules import each other	`a.ry` imports `b.ry` and `b.ry` imports `a.ry`
Duplicate field name	Defined the same field name twice in a struct	Defining `x` twice in `type T: x: int`
Non-exhaustive match	`case` does not cover all patterns	Some enum variants uncovered, missing `None` for Option, missing `Ok`/`Err` for Result, no `_` for literals
`?` on non-Result type	Applied `?` to an expression that is not a `Result` type	`x = 42 -> x?`
`?` in non-Result function	Used `?` in a function that does not return `Result`	`function foo() -> int: -> bar()?`

Compile Error Examples

```
# Reassignment to @const variable
@const
x = 10
x = 20   # Error
```

```
# Type-changing reassignment
n = 1
n = "hello"   # Error: assigning str to int variable

# Empty list
xs = []   # Error: type cannot be inferred

# break outside loop
break   # Error: outside loop

# Import inside block
function foo():
    from math   # Error: top level only

# Duplicate field name
record Bad:
    x: int
    x: float   # Error: x is duplicated
```

Runtime Errors

Error	Cause	Example
List out-of-range access	List index exceeds bounds	`xs = [1, 2, 3]` -> `xs[5]`
Map non-existent key access	Referenced a key that does not exist in the map	`m = {"a": 1}` -> `m["b"]`
Contract violation	A `require`, `ensure`, or `invariant` condition evaluated to false	See Design by Contract

All runtime errors terminate the process with `exit(1)`.

Runtime Error Examples

```
# List out-of-range access
xs = [1, 2, 3]
print(xs[10])   # Runtime error: exit(1)

# Map non-existent key access
m = {"a": 1}
print(m["z"])   # Runtime error: exit(1)
```

English | 日本語 | 繁體中文

Filesystem Function Reference

File and directory manipulation. All functions require explicit import from `filesystem`.

The `filesystem` package handles operations on files and directories themselves (copy, move, remove, etc.), while the `io` package handles reading and writing file contents.

```
from filesystem import list_dir, walk, glob_files, copy, move, remove, remove_all
from filesystem import make_dir, make_dir_all, file_size, is_file, is_dir, is_symlink
from filesystem import chmod, symlink, read_link
```

Function List

Function	Signature	Description
`list_dir`	`(str) -> Result<List<str>, Error>`	Lists entries in a directory (non-recursive)
`walk`	`(str) -> Result<List<str>, Error>`	Recursively lists all files and directories
`glob_files`	`(str) -> Result<List<str>, Error>`	Finds files matching a glob pattern
`copy`	`(str, str) -> Result<Unit, Error>`	Copies a file
`move`	`(str, str) -> Result<Unit, Error>`	Moves or renames a file or directory
`remove`	`(str) -> Result<Unit, Error>`	Removes a file or empty directory
`remove_all`	`(str) -> Result<Unit, Error>`	Removes a file or directory tree recursively
`make_dir`	`(str) -> Result<Unit, Error>`	Creates a single directory
`make_dir_all`	`(str) -> Result<Unit, Error>`	Creates a directory and all missing parents
`file_size`	`(str) -> Result<int, Error>`	Returns file size in bytes
`is_file`	`(str) -> bool`	Checks if path is a regular file
`is_dir`	`(str) -> bool`	Checks if path is a directory
`is_symlink`	`(str) -> bool`	Checks if path is a symbolic link
`chmod`	`(str, int) -> Result<Unit, Error>`	Changes file permissions (POSIX mode)
`symlink`	`(str, str) -> Result<Unit, Error>`	Creates a symbolic link
`read_link`	`(str) -> Result<str, Error>`	Reads the target of a symbolic link

Examples

Directory Operations

```
from filesystem import make_dir, make_dir_all, list_dir, remove_all

# Create a single directory
case make_dir("/tmp/myapp"):
  Ok(_):
    print("created")
  Err(e):
    print("error: " + e.message)

# Create nested directories (like mkdir -p)
make_dir_all("/tmp/myapp/data/logs")

# List directory contents
case list_dir("/tmp/myapp"):
```

```
  Ok(entries):
    for entry in entries:
      print(entry)
  Err(e):
    print("error: " + e.message)

# Remove a directory tree (like rm -rf)
remove_all("/tmp/myapp")
```

File Operations

```
from filesystem import copy, move, remove, file_size
from io import write_text

write_text("/tmp/hello.txt", "Hello, World!")

# Copy a file
copy("/tmp/hello.txt", "/tmp/hello_copy.txt")

# Get file size
case file_size("/tmp/hello.txt"):
  Ok(sz):
    print("size: " + to_str(sz))
  Err(e):
    print("error: " + e.message)

# Move / rename a file
move("/tmp/hello_copy.txt", "/tmp/renamed.txt")

# Remove a file
remove("/tmp/renamed.txt")
```

Recursive Traversal

```
from filesystem import walk, glob_files

# Walk a directory tree (like find)
case walk("/var/log"):
  Ok(files):
    for f in files:
      print(f)
  Err(e):
    print("error: " + e.message)

# Glob pattern matching
case glob_files("/var/log/*.log"):
  Ok(matches):
    for m in matches:
      print(m)
  Err(e):
    print("error: " + e.message)
```

Path Type Checks

```
from filesystem import is_file, is_dir, is_symlink

if is_file("/etc/hosts"):
  print("regular file")

if is_dir("/tmp"):
  print("directory")

if is_symlink("/usr/local/bin/python"):
  print("symbolic link")
```

Symbolic Links

```
from filesystem import symlink, read_link, is_symlink

# Create a symlink
symlink("/usr/local/bin/ry", "/tmp/ry_link")

# Check and read symlink
if is_symlink("/tmp/ry_link"):
  case read_link("/tmp/ry_link"):
    Ok(target):
      print("points to: " + target)
    Err(e):
      print("error: " + e.message)
```

Permissions

```
from filesystem import chmod

# chmod 755 (rwxr-xr-x) — use decimal value: 0o755 = 493
chmod("/tmp/script.sh", 493)

# chmod 644 (rw-r--r--) — 0o644 = 420
chmod("/tmp/data.txt", 420)
```

Notes

- `is_file`, `is_dir`, and `is_symlink` return `false` on error (e.g., path does not exist)
- `is_file` and `is_dir` follow symlinks; `is_symlink` uses `lstat` to detect links
- `list_dir` returns entry names only (not full paths)
- `walk` returns full paths for all entries (both files and directories)
- `glob_files` returns an empty list (not an error) when no files match the pattern
- `remove` fails on non-empty directories; use `remove_all` for recursive deletion

English | 日本語 | 繁體中文

Function Reference

Function Definition Syntax

```
function function_name(param_name: type, ...) -> return_type:
    # body
    return value
```

- Parameter types are optional. When omitted, the parameter is treated as `any` type.
- Return type is optional. When omitted, the return type is **inferred from the body** (both named functions and lambdas). If the function has no `return` statement, the return type is inferred as `Unit`. Use `-> any` explicitly for functions that should accept any return type.
- The body is an indented block.
- Functions with an explicit return type (other than `Unit` or `any`) must have a `return` statement on all control flow paths. The compiler reports an error if any path is missing a return.
- Functions can have `require` (precondition) and `ensure` (postcondition) clauses. See Design by Contract.

  Naming convention: Function names and parameter names must use snake_case (e.g., `add`, `get_value`, `map_list`). The compiler enforces this convention.

```
function add(a: int, b: int) -> int:
    return a + b

function greet(name: str) -> Unit:
    print("Hello, " + name)    # Return type is Unit (explicit)
```

Parameter and Return Types

Item	Description
Parameter type	Optional. Defaults to `any` when the `: type` annotation is omitted
Return type	Optional. Inferred from the body when omitted (inferred as `Unit` if no `return` statement)
`Unit`	Return type for functions that return no value

> **Note**: Function parameters are **immutable**. You cannot reassign a parameter inside the function body. This ensures that parameter values at entry are always available for postcondition checks (see Design by Contract).

```
function no_return(x: int) -> Unit:  # Return type Unit (explicit)
    print(x)

function get_value() -> int:     # Return type int
    return 42

function identity(x) -> any:    # Parameter type any (omitted)
    return x
```

Type Omission and `any`

When a parameter type annotation is omitted, the parameter is treated as `any` — a dynamic type that accepts any primitive value at runtime. This is similar to Python's untyped parameters.

```
# All parameters default to any
function add(a, b):
    return a + b

add(1, 2)              # 3 (int + int)
add("hello", " world") # "hello world" (str + str)
add(1, 2.0)            # 3.0 (int + float)
```

You can also use `any` explicitly in type annotations:

```
function identity(x: any) -> any:
    return x
```

Return Type Inference

When the return type is omitted, it is inferred from the `return` statements in the body:

```
function double(x: int):      # return type inferred as int
    return x * 2

function greet(name: str):    # return type inferred as Unit (no return)
    print("Hello, " + name)
```

To explicitly allow any return type, use `-> any`:

```
function flexible(x: any) -> any:
    return x    # can return int, float, str, etc.
```

Nested Functions

Functions can be defined inside other functions. A nested function is only visible within its enclosing function's scope — it cannot be called from outside.

```
function outer() -> int:
    function helper() -> int:
        return 42
    return helper()

outer()     # 42
# helper()  # error: undefined function
```

Same-named nested functions in sibling scopes do not collide:

```
function foo() -> int:
    function helper() -> int:
        return 1
    return helper()

function bar() -> int:
    function helper() -> int:
        return 2
    return helper()

foo()   # 1
bar()   # 2
```

Nested functions can be used as values and passed to higher-order functions. Mutual recursion between nested functions in the same scope also works (the compiler forward-declares them).

Closure Capture

Nested named functions can capture variables from enclosing scopes, just like lambdas. When a nested function references an outer variable, it becomes a closure:

```
function make_adder(base: int) -> function(int) -> int:
    function add(x: int) -> int:
        return x + base
```

```
    return add

add10 = make_adder(10)
add10(5)    # 15
```

Capture rules:

- Captures are **by value** (same as lambdas). The value is copied at the point the closure is created.
- Captured variables **cannot be reassigned** inside the nested function body.
- ARC-managed values (strings, lists, etc.) are properly retained and released.
- If a nested function has no captures, it remains a plain function pointer (no overhead).
- Multi-level capture works: a deeply nested function can reference variables from any enclosing scope.

Recursion

Functions can call themselves.

```
function factorial(n: int) -> int:
    if n <= 1:
        return 1
    return n * factorial(n - 1)
```

Mutual Recursion

Functions can call each other regardless of definition order. The compiler forward-declares top-level functions with explicit return types before processing function bodies, provided all referenced types are already known (primitive types are always available; record/enum types must be defined earlier in the file).

```
function is_even(n: int) -> bool:
    if n == 0:
        return true
    return is_odd(n - 1)       # calls is_odd defined below

function is_odd(n: int) -> bool:
    if n == 0:
        return false
    return is_even(n - 1)      # calls is_even defined above
```

Requirements for forward references:

- The function must have an **explicit return type** annotation (`-> type`). Functions with inferred return types cannot be forward-referenced.
- The function must be defined at the **top level** or inside another function body. Forward references work within the same scope level.
- All parameter and return types must be resolvable at the point of forward declaration (e.g., record types must be defined before the functions that use them).

Top-Level Variables and `@const` in Function Bodies

Top-level `let` bindings and `@const` declarations are visible from any top-level function — including nested functions and lambdas inside those functions — as long as the declaration appears **textually before** the referencing function in the same source file.

```
@const
PI: float = 3.14

@const
```

```
MAX_RETRIES: int = 5

counter: int = 0

function area(radius: float) -> float:
    return PI * radius * radius            # reads top-level @const

function clamp_retries(n: int) -> int:
    if n > MAX_RETRIES:
        return MAX_RETRIES
    return n

function bump():
    counter = counter + 1                  # writes top-level mutable `let`
```

Rules:

- **Source-order strict.** A function body cannot reference a top-level binding declared after it in the same file. Move the binding above the function, or wrap the binding in a helper function called lazily.
- **`@const` is read-only.** Reassignment or field mutation (`P.x = 99` for a top-level `@const P: Point`) is rejected at compile time.
- **Mutable `let` writes are write-through.** Assigning to a top-level mutable variable from inside a function actually mutates the top-level binding — it does not create a local with the same name.
- **Nested blocks are not module-level.** A `let` inside a top-level `if`, `while`, or `for` block is local to that block and is not visible from functions.

Limitations (v0.0.8):

- A parallel `for` block cannot assign to a top-level mutable variable (data-race avoidance).
- Top-level `weak` references and resource-typed bindings (file/regex handles) cannot yet be accessed from function bodies —track these use cases in follow-up issues if you need them.

Tail Call Optimization

The compiler automatically detects self-recursive tail calls — where the last action in a function is a call to itself — and applies LLVM' s `musttail` optimization. This guarantees that tail-recursive functions use constant stack space, preventing stack overflow for deep recursion.

```
function sum_to(n: int, acc: int) -> int:
    if n <= 0:
        return acc
    return sum_to(n - 1, acc + n)     # tail call → optimized

sum_to(1000000, 0)    # works without stack overflow
```

Conditions for TCO:

- The function calls itself directly in a `return` statement (`return f(args)`)
- The call result is returned without any further computation (`return n * f(n-1)` is NOT a tail call)
- The function has no `ensure` (postcondition) clauses

Mutual recursion (A calls B, B calls A) is not currently optimized for tail calls.

Overloading

Multiple functions with the same name can be defined if they differ in the number or types of parameters.

Rules

- Functions with the same name can be defined if the number or types of parameters differ.
- The appropriate function is selected at the call site based on the argument types and count.
- Overloading by return type alone is not allowed.

```
function area(side: int) -> int:
    return side * side

function area(w: int, h: int) -> int:
    return w * h

a = area(5)       # 25
b = area(3, 4)    # 12
```

Resolution Priority

When multiple overloads case a call, the compiler selects the most specific one using the following priority (highest first):

1. **Exact type match** — argument type matches parameter type exactly
2. **Implicit widening** — safe widening conversion (`u8` → `int`, `u8` → `float`, `int` → `float`)
3. **Union type match** — argument type is a member of a union parameter type
4. **`any` type match** — parameter type is `any` (accepts anything)

The overload with the most exact matches wins. If two or more overloads have equal specificity, the compiler reports an ambiguity error.

Low-level numeric types (`i8`, `i16`, `i32`, `i64`, `u8` – `u64`, `f32`) do **not** participate in implicit widening — they require explicit `as` casts.

```
function process(x: int) -> str:
    return "int"

function process(x) -> str:          # x: any
    return "any"

process(42)       # "int" — exact match (int) beats any
process("hello")  # "any" — no exact match for str, falls back to any
```

```
function double(x: float) -> float:
    return x * 2.0

double(5)         # OK — int is implicitly widened to float, returns 10.0
```

Default Arguments

Parameters can have default values, allowing callers to omit trailing arguments.

Syntax

```
function connect(host: str, port: int = 8080, timeout: int = 30):
    # ...

connect("localhost")                    # port=8080, timeout=30
connect("localhost", 3000)              # port=3000, timeout=30
connect("localhost", 3000, 10000)       # port=3000, timeout=10000
```

Rules

- Default parameters must come after all non-default parameters.
- A parameter with a default value **must** have an explicit type annotation (e.g., `x: int = 10`; `x = 10` is a compile error).
- Default values must be compile-time constant expressions (literals and `@const` variables).
- If a function with default arguments creates an ambiguous overload (overlapping arity with matching types), the compiler reports an error.

```
# Error: ambiguous overload
function calc(x: int, y: int = 0) -> int:
    return x + y
function calc(x: int) -> int:      # conflicts with calc(int) from above
    return x * 2
```

Limitations

- Default arguments are not supported in **generic functions** or **lambda expressions**.

Unit Type Functions

Functions without a return value return `Unit`. The return type can be omitted (inferred as `Unit`) or explicitly specified with `-> Unit`.

```
function log(msg: str) -> Unit:
    print(msg)
```

Tasks And Async Functions

`Task<T>` is the built-in handle type for concurrent work. `async function` returns `Task<T>`, `await` extracts `T` inside another `async function`, and `block_on(task)` blocks from synchronous context until the task completes.

```
async function add(a: int, b: int) -> int:
    return a + b

# From synchronous context, use block_on()
t: Task<int> = add(20, 22)
print(block_on(t))                  # 42
block_on(add(1, 2))                 # waits and discards the result

# Inside async function, use await
async function double_add(a: int, b: int) -> int:
    return (await add(a, b)) * 2
```

Rules

- `async function name(...) -> T:` is declared with the awaited result type `T`.
- Calling an `async function` immediately returns `Task<T>`.
- `await expr` requires `expr` to be `Task<T>` and produces `T`.
- `await` can only be used inside an `async function`. Use `block_on(task)` from synchronous context.
- `block_on(task)` blocks the current thread until the task completes and returns the result.
- `async function ... -> Unit` is supported; `block_on(task)` is the primary way to wait when no value is produced.

- Tasks run on the runtime worker pool; they are not implemented as one OS thread per task.
- `async` lambdas and `async @native function` are not supported in v1.

Lambda Functions

Anonymous functions can be defined inline.

Syntax

```
# Single expression (return type inferred from expression)
 (param_name: type, ...) => expression

# Parameter type can be omitted (defaults to any)
 (param_name, ...) => expression

# Multi-line block
(param_name: type, ...):
    # multiple statements
    return value

# With explicit return type (optional)
 (param_name: type, ...) -> return_type => expression
```

Example

```
double = (x: int) => x * 2
result = double(5)   # 10

add = (a: int, b: int) => a + b
sum = add(3, 4)      # 7

# Multi-line lambda
abs = (x: int):
    if x < 0:
        return -x
    return x
```

Closures

Lambda functions **capture by value** the variables from the outer scope at the time of definition. The closure receives its own independent copy at capture time, and the captured variable cannot be reassigned inside the closure.

Outer changes do not affect the closure

Because the closure holds a copy, reassigning the original variable after the closure is defined has no effect on the captured value:

```
base = 10
add_base = (x: int) => x + base   # Captures base by value (copy of 10)

base = 99          # Does not affect the captured value
r = add_base(5)    # 15 (uses base = 10 from capture time)
```

Captured variables are effectively final

Captured variables **cannot be reassigned** inside the closure. Attempting to do so produces a compile error:

```
counter = 0
inc = ():
    counter += 1    # Compile error: cannot modify captured variable 'counter' inside closure

inc()
```

Field assignment on captured records is allowed, since it modifies the copy's internal state rather than reassigning the variable itself:

```
record Point:
    x: int
    y: int

p = Point(0, 0)
move = ():
    p.x = p.x + 1    # OK — modifies the captured copy's field
```

Note: Field modifications apply to the closure's copy only —the outer variable is unaffected.

Capture Rules

Item	Details
Capture method	Capture by value (copy)
Capture timing	At lambda definition time
Reassignment of captured variables	Not allowed (compile error)
Field assignment on captured records	Allowed (modifies the copy only)
Effect of outer variable changes	None (the closure holds its own copy)

> **Note for Python/JavaScript users**: In JavaScript, closures capture variables by reference, so changes to a captured variable are reflected outside the closure. In Python, closures can access outer variables, and rebinding an outer name (such as `counter += x`) requires declaring it `nonlocal`. In Ry, closures always capture by value, and captured variables are effectively final — they cannot be reassigned inside the closure. This is intentional — it ensures safety and predictability, especially in concurrent or higher-order contexts.

Function Type

A type for treating functions as values.

Syntax

```
function(param_type1, param_type2, ...) -> return_type
```

Example

```
f: function(int) -> int = (x: int) => x * 2
g: function(int, int) -> int = (a: int, b: int) => a + b

function apply(func: function(int) -> int, x: int) -> int:
    return func(x)
```

```
result = apply(f, 5)   # 10
```

String Representation

`print()`, `to_str()`, and f-string interpolation all produce `"<closure>"` for function values:

```
f = (x: int) => x + 1
print(f)              # <closure>
s = to_str(f)         # "<closure>"
msg = f"fn={f}"       # "fn=<closure>"
```

> **Note**: Equality comparison (`==` / `!=`) between closures is not supported and produces a compile-time error.

Higher-Order Functions

Functions can accept functions as arguments or return them as values.

```
function map_list(xs: List<int>, f: function(int) -> int) -> List<int>:
    result: List<int> = []
    for x in xs:
        result += [f(x)]
    return result

doubled = map_list([1, 2, 3], (x: int) => x * 2)
# [2, 4, 6]
```

Generic Functions

Functions can have type parameters, enabling type-safe reuse without code duplication.

Syntax

```
function name<T, U>(param1: T, param2: U) -> T:
    # body using T, U as types
```

Example

```
function identity<T>(x: T) -> T:
    return x

# Explicit type argument
result = identity[int](42)      # 42
result = identity[str]("hello") # "hello"

# Type inference (type argument deduced from actual argument)
result = identity(42)           # T = int, result = 42
result = identity("hello")     # T = str, result = "hello"
```

Multiple Type Parameters

```
function pick_first<T, U>(a: T, b: U) -> T:
    return a
```

```
result = pick_first(1, "x")       # T = int, U = str, result = 1
result = pick_first("hello", 42)  # T = str, U = int, result = "hello"
```

Type Parameters Inside Container Types

Type parameters can appear inside generic container types (`List<T>`, `Map<K, V>`, `Set<T>`), tuples `(T, T)`, and function types `function(T) -> T`. Inference walks the declared parameter type structurally against the actual argument, so explicit type annotations are not required when the shape is unambiguous.

```
function first_of<T>(xs: List<T>) -> T:
    return xs[0]

first_of([1, 2, 3])            # T = int  → 1
first_of(["hello", "world"])   # T = str  → "hello"
first_of([[1, 2], [3, 4]])     # T = List<int>  → [1, 2]

function map_lookup<K, V>(m: Map<K, V>, k: K) -> V:
    return m[k]

map_lookup({1: "a", 2: "b"}, 1)      # K = int, V = str → "a"
map_lookup({"x": 10, "y": 20}, "y") # K = str, V = int → 20

function pair_first<T>(p: (T, T)) -> T:
    return p.0

pair_first((42, 7))       # T = int → 42
pair_first(("a", "b"))    # T = str → "a"
```

A type parameter referenced across multiple parameter positions is unified — both occurrences must resolve to the same concrete type:

```
function apply_list<T>(xs: List<T>, f: function(T) -> T) -> T:
    return f(xs[0])

apply_list([10, 20, 30], (x: int) => x + 1)  # T = int → 11
```

If inference cannot determine a type parameter (for example, from an empty container literal), use the explicit `name[Type](args)` syntax:

```
first_of[int]([])   # empty list: tell the compiler T = int explicitly
```

Conflicting inferences across arguments produce a clear compile error naming the type parameter and function rather than an opaque type mismatch:

```
function same<T>(a: T, b: T) -> T:
    return a

same(1, "x")  # error: conflicting type inference for 'T' in call to 'same'
```

Type Constraints (Bounds)

Type parameters can be constrained with record types using `: RecordName` syntax. The concrete type must be the bound type itself or a subtype of it.

```
record Animal:
    name: str
    legs: int
```

```
record Dog < Animal:
    breed: str

function get_name<T: Animal>(a: T) -> str:
    return a.name

get_name(Dog("Rex", 4, "Lab"))  # OK — Dog is a subtype of Animal
get_name(Animal("Cat", 4))      # OK — exact type match
```

Bounded and unbounded type parameters can be mixed:

```
function pair_name<T: Animal, U>(a: T, x: U) -> str:
    return a.name
```

How It Works

Generic functions use **monomorphization**: a specialized version of the function is generated for each unique combination of type arguments. The same instantiation is cached and reused across multiple calls. When type constraints are present, they are validated at instantiation time.

UFCS (Uniform Function Call Syntax)

`a.f(b)` can be used to call `f(a, b)`. Useful for method chaining.

Syntax

```
# Normal call
f(a, b)

# UFCS call (equivalent)
a.f(b)
```

Chaining

```
function double(x: int) -> int:
    return x * 2

function add_one(x: int) -> int:
    return x + 1

result = 5.double().add_one()   # double(5) -> 10, add_one(10) -> 11
```

Mixing with Field Access

Field access (`.field`) and UFCS (`.method()`) use the same dot notation but are distinguished by the presence of arguments.

```
p = Point(3, 4)
length = p.x.to_float()   # Field access + UFCS
```

Operator Overloading

You can define operator behavior for user-defined types.

Syntax

```
# Binary operator (2 parameters)
function operator<op>(a: type, b: type) -> return_type:
    ...

# Unary operator (1 parameter)
function operator<op>(a: type) -> return_type:
    ...
```

Overloadable Operators

Category	Operators
Arithmetic (binary)	`+ - * / % ** //`
Comparison (binary)	`== != < <= > >=`
Bitwise (binary)	`& | ^ << >>`
Logical (binary)	`and or`
Membership	`in`
Subscript	`[]` (read), `[]=` (write)
Call	`()`
Cast	`as`
Unary	`- ~ not`

Return Type Constraints

Comparison, logical, and membership operators must return `bool`:

Category	Operators	Required Return Type
Comparison	`== != < <= > >=`	`bool`
Logical	`and or not`	`bool`
Membership	`in`	`bool`
Cast	`as`	Required (target type)

```
# OK
function operator==(a: Vec2, b: Vec2) -> bool:
    return a.x == b.x and a.y == b.y

# Error: comparison operator '==' must return 'bool', but returns 'int'
function operator==(a: Vec2, b: Vec2) -> int:
    return 42
```

Arithmetic and bitwise operators have no return type constraint.

Distinguishing Binary and Unary

Distinguished by the number of parameters.

```
record Vec2:
    x: float
    y: float

# Binary +
function operator+(a: Vec2, b: Vec2) -> Vec2:
    return Vec2(a.x + b.x, a.y + b.y)
```

```
# Unary -
function operator-(v: Vec2) -> Vec2:
    return Vec2(-v.x, -v.y)

# Comparison
function operator==(a: Vec2, b: Vec2) -> bool:
    return a.x == b.x and a.y == b.y

v1 = Vec2(1.0, 2.0)
v2 = Vec2(3.0, 4.0)
v3 = v1 + v2    # Vec2(4.0, 6.0)
v4 = -v1        # Vec2(-1.0, -2.0)
```

Checked/Saturating Arithmetic

Built-in functions for explicit overflow control on low-level integer types (`i8`, `i16`, `i32`, `i64`, `u8`, `u16`, `u32`, `u64`). Both arguments must be the same type.

Function	Returns	Behavior
`checked_add(a, b)`	`Result<T, Error>`	Returns `Err` on overflow
`checked_sub(a, b)`	`Result<T, Error>`	Returns `Err` on underflow
`checked_mul(a, b)`	`Result<T, Error>`	Returns `Err` on overflow
`saturating_add(a, b)`	`T`	Clamps to type min/max on overflow
`saturating_sub(a, b)`	`T`	Clamps to type min/max on underflow
`saturating_mul(a, b)`	`T`	Clamps to type min/max on overflow
`wrapping_add(a, b)`	`T`	Explicit wrapping (same as `+`)
`wrapping_sub(a, b)`	`T`	Explicit wrapping (same as `-`)
`wrapping_mul(a, b)`	`T`	Explicit wrapping (same as `*`)

```
# Checked: returns Result, use match or ? to handle
r = checked_add(2147483647i32, 1i32)
case r:
  Ok(v):
    print(v)
  Err(e):
    print("overflow!")    # prints "overflow!"

# Saturating: clamps to bounds
v = saturating_add(2147483647i32, 100i32)
print(v as int)    # 2147483647

# Wrapping: self-documenting wrapping behavior
v = wrapping_add(2147483647i32, 1i32)
print(v as int)    # -2147483648
```

> **Note**: These functions do not support floating-point types (`f32`) or the high-level `int` type. The default `+`, `-`, `*` operators on low-level integers use wrapping behavior (two' s complement for signed, modular for unsigned).

English

GC Reference

The `gc` package provides control over the cycle collector, a safety net that works alongside ARC (Automatic Reference Counting) to detect and reclaim circular reference chains.

Overview

Ry uses ARC for memory management, which handles most deallocations immediately and deterministically. However, ARC alone cannot reclaim circular references (e.g., A → B → A). The cycle collector uses a CPython-style trial deletion algorithm to periodically find and free such unreachable cycles.

Using `weak` references is still the recommended way to avoid cycles, but the cycle collector catches cases the user misses.

Import

```
from gc import collect, enable, disable, set_threshold
```

Functions

Function	Signature	Description
`collect`	`() -> int`	Runs a full collection cycle. Returns the number of objects collected.
`enable`	`() -> Unit`	Enables automatic collection (enabled by default).
`disable`	`() -> Unit`	Disables automatic collection. Useful for performance-critical sections.
`set_threshold`	`(n: int) -> Unit`	Sets the candidate count threshold for automatic collection (default: 700).

How It Works

1. **Candidate tracking**: When `arc_release` decrements an object' s reference count but it remains above zero, the object is added to a candidate set (only for types that can potentially form cycles).
2. **Trial deletion**: The collector tentatively decrements reference counts of all objects referenced by candidates. If an object' s count reaches zero through trial deletion alone, it is only reachable within the candidate set — it is part of a cycle.
3. **Resurrection check**: Objects still reachable from outside the candidate set have their counts restored.
4. **Collection**: Remaining unreachable objects are deallocated.

Static Analysis Optimization

The compiler performs static analysis at compile time to identify which types can potentially form reference cycles (e.g., recursive ADT enums). Only these types participate in GC candidate tracking. Non-cyclic types have zero GC overhead.

Example

```
from gc import collect, disable, enable

# Disable automatic collection for a performance-critical section
```

```
disable()

# ... performance-critical code ...

# Re-enable and force a collection
enable()
collected = collect()
print(f"Collected {collected} cyclic objects")
```

English

HTTP Reference

Types

Type	Description
`HttpRequest`	Opaque handle for an incoming HTTP request
`HttpResponse`	Opaque handle for an outgoing HTTP response
`HttpClientResponse`	Opaque handle for an HTTP client response

`HttpRequest` is provided by the server framework. `HttpResponse` is created via `response()`. `HttpClientResponse` is returned by client functions (`http_get`, `http_post`, `http_request`).

Functions (from `http`)

These functions require explicit import:

```
from http import listen, method, path, header, body, body_bytes, query, query_all, cookie, cookies, form
```

Server

Function	Signature	Description
`listen`	`(host: str, port: int, handler: function(HttpRequest) -> HttpResponse) -> Unit`	Starts an HTTP server on the given address. Blocks in an accept loop, calling `handler` for each request.
`listen`	`(host: str, port: int, handler: function(HttpRequest) -> HttpResponse, max_requests: int) -> Unit`	Starts an HTTP server that stops after processing `max_requests` requests. Enables `async function` + `block_on()` lifecycle management.
`listen`	`(host: str, port: int, handler: function(HttpRequest) -> HttpResponse, max_requests: int, port_callback: function(int) -> Unit) -> Unit`	Same as above, but calls `port_callback` with the actual bound port after `bind` + `listen` succeeds. Use with port `0` for OS-assigned ephemeral ports.

Request Accessors

Function	Signature	Description
`method`	`(req: HttpRequest) -> str`	Returns the HTTP method (e.g., `"GET"`, `"POST"`).
`path`	`(req: HttpRequest) -> str`	Returns the request path without the query string (e.g., `"/search"` for `"/search?q=hello"`).
`header`	`(req: HttpRequest, key: str) -> Option<str>`	Returns the value of a request header (case-insensitive lookup). Returns `None` if not found.
`body`	`(req: HttpRequest) -> str`	Returns the request body as a string. Truncated at the first NUL byte; use `body_bytes` for binary data.
`body_bytes`	`(req: HttpRequest) -> List<u8>`	Returns the request body as a byte list. Binary-safe — preserves all bytes including NUL.
`query`	`(req: HttpRequest, key: str) -> Option<str>`	Returns the value of a query parameter. Returns `None` if not found. Values are automatically URL-decoded.
`query_all`	`(req: HttpRequest) -> Map<str, str>`	Returns all query parameters as a map. Keys and values are automatically URL-decoded.
`cookie`	`(req: HttpRequest, name: str) -> Option<str>`	Returns the value of a cookie by name. Returns `None` if not found.
`cookies`	`(req: HttpRequest) -> Map<str, str>`	Returns all cookies as a map.
`form_field`	`(req: HttpRequest, name: str) -> Option<str>`	Returns the value of a multipart form text field. Returns `None` if not found.
`form_file`	`(req: HttpRequest, name: str) -> Option<Map<str, str>>`	Returns file upload info as an `Option`. `Some(map)` contains keys `"filename"`, `"content_type"`, `"data"`. Returns `None` if not found.
`form_fields`	`(req: HttpRequest) -> Map<str, str>`	Returns all multipart form text fields as a map.

Response Builder

Function	Signature	Description
`response`	`(status: int, headers: Map<str, str>, body: str) -> HttpResponse`	Creates an HTTP response with the given status code, headers, and body.

Usage Example

Basic HTTP Server

```
from http import listen, method, path, header, body, response

listen("127.0.0.1", 8080, (req: HttpRequest) -> HttpResponse:
    m = method(req)
    p = path(req)
    if p == "/hello":
        return response(200, {"Content-Type": "text/plain"}, "Hello, World!")
```

```
    if p == "/echo":
        b = body(req)
        return response(200, {"Content-Type": "text/plain"}, b)
    return response(404, {"Content-Type": "text/plain"}, "Not Found")
)
```

Non-blocking Server with `async function`

```
from http import listen, path, response

async function start_server(port: int) -> str:
    listen("127.0.0.1", port, (req: HttpRequest) -> HttpResponse:
        p = path(req)
        if p == "/api/health":
            return response(200, {"Content-Type": "application/json"}, "{\"status\": \"ok\"}")
        return response(404, {"Content-Type": "text/plain"}, "Not Found")
    )
    return "done"

t = start_server(8080)
# Server runs in background task
```

Server with Request Limit (`max_requests`)

```
from http import listen, path, response, http_get, status, body

port_holder = [0]
function on_port(p: int) -> Unit:
    port_holder[0] = p

async function start_server() -> str:
    listen("127.0.0.1", 0, (req: HttpRequest) -> HttpResponse:
        return response(200, {"Content-Type": "text/plain"}, "Hello!")
    , 1, on_port)  # Stop after 1 request; call on_port with bound port
    return "done"

t = start_server()
sleep(100)  # Wait for server to start
port = port_holder[0]

case http_get("http://127.0.0.1:" + to_str(port) + "/"):
    Ok(resp):
        print(body(resp))  # "Hello!"
    Err(e):
        print("error")

result = block_on(t)  # Server exits after 1 request; block_on completes
```

Reading Query Parameters

```
from http import listen, path, query, query_all, response

listen("127.0.0.1", 8080, (req: HttpRequest) -> HttpResponse:
    p = path(req)
    if p == "/search":
```

```
        case query(req, "q"):
            Some(q):
                return response(200, {"Content-Type": "text/plain"}, "Search: " + q)
            None:
                return response(400, {"Content-Type": "text/plain"}, "Missing query parameter: q")
    return response(404, {"Content-Type": "text/plain"}, "Not Found")
)
```

Reading Headers

```
from http import listen, header, response

listen("127.0.0.1", 8080, (req: HttpRequest) -> HttpResponse:
    case header(req, "Authorization"):
        Some(token):
            return response(200, {"Content-Type": "text/plain"}, "Authenticated: " + token)
        None:
            return response(401, {"Content-Type": "text/plain"}, "Unauthorized")
)
```

Handling Form Submissions

```
from http import listen, form_field, form_file, response

listen("127.0.0.1", 8080, (req: HttpRequest) -> HttpResponse:
    case form_field(req, "username"):
        Some(name):
            case form_file(req, "avatar"):
                Some(file_info):
                    filename = file_info["filename"]
                    return response(200, {"Content-Type": "text/plain"}, "Hello " + name + ", file: " +
                None:
                    return response(400, {"Content-Type": "text/plain"}, "No file uploaded")
        None:
            return response(400, {"Content-Type": "text/plain"}, "Missing username")
)
```

Reading Cookies

```
from http import listen, cookie, cookies, response

listen("127.0.0.1", 8080, (req: HttpRequest) -> HttpResponse:
    case cookie(req, "session_id"):
        Some(sid):
            return response(200, {"Content-Type": "text/plain"}, "Session: " + sid)
        None:
            return response(401, {"Content-Type": "text/plain"}, "No session")
)
```

Behavior

- `listen()` binds to the address, starts listening, and enters an accept loop.
- When called with 3 arguments, the accept loop runs indefinitely.
- When called with 4 arguments (`max_requests`), the server stops after processing the specified number of requests. `max_requests` must be a positive integer. This enables `async function` + `block_on()`

lifecycle management. Malformed requests (silently skipped) do not count toward the limit.

- When called with 5 arguments (`max_requests`, `port_callback`), `port_callback` is called synchronously with the actual bound port after `bind` + `listen` succeeds. This allows safe use of port `0` (OS-assigned ephemeral port) to avoid port conflicts in parallel tests.
- The server supports HTTP/1.1 keep-alive by default. Multiple requests can be processed on a single connection. The server checks the `Connection` header on each request: if `Connection: close` is sent, the connection is closed after the response; otherwise the connection stays open for subsequent requests. Idle connections are closed after a 5-second timeout.
- `Content-Length` is automatically added to the response if not provided in the headers map.
- The server supports HTTP/1.1 with `Content-Length`-based body reading and `Transfer-Encoding: chunked` decoding.
- When `Transfer-Encoding: chunked` is present in a request, the body is automatically decoded and concatenated — Ry code receives the full body transparently.
- If both `Transfer-Encoding: chunked` and `Content-Length` are present, the request is rejected as malformed (per RFC 9112).
- To send a chunked response, include `"Transfer-Encoding": "chunked"` in the headers map passed to `response()`. The body will be encoded in chunked format automatically.
- Header lookup via `header()` is case-insensitive.
- `path()` returns the path without the query string. Query parameters are accessed separately via `query()` or `query_all()`.
- Query parameter values are automatically URL-decoded (`%20` → space, `+` → space).
- For duplicate query parameter keys, the first value is returned.
- `cookie()` and `cookies()` parse the `Cookie` header by splitting on `;`, then splitting each pair on the first `=`. Leading and trailing whitespace is trimmed from names and values.
- For duplicate cookie names, the first value is returned.
- Cookie values may contain `=` characters (only the first `=` separates the name from the value).
- `form_field()`, `form_file()`, and `form_fields()` parse `multipart/form-data` request bodies. Parsing is lazy — the body is parsed on the first call and cached.
- The `boundary` parameter is extracted from the `Content-Type` header and supports both quoted and unquoted values.
- Parts with a `filename` in `Content-Disposition` are treated as file uploads; parts without are treated as text fields.
- For duplicate field/file names, the first value is returned.
- `form_file()` returns `Some(map)` with keys `"filename"`, `"content_type"`, and `"data"`, or `None` if the field is not found. If no `Content-Type` is specified for the part, it defaults to `"application/octet-stream"`.
- For non-multipart requests, form functions return `None` (for `form_field`, `form_file`) or an empty map (for `form_fields`).

Supported Status Codes

The following status codes have standard reason phrases (RFC 9110):

Code	Reason
100	Continue
101	Switching Protocols
200	OK
201	Created
202	Accepted
203	Non-Authoritative Information
204	No Content
205	Reset Content
206	Partial Content
300	Multiple Choices

Code	Reason
301	Moved Permanently
302	Found
303	See Other
304	Not Modified
307	Temporary Redirect
308	Permanent Redirect
400	Bad Request
401	Unauthorized
403	Forbidden
404	Not Found
405	Method Not Allowed
406	Not Acceptable
408	Request Timeout
409	Conflict
410	Gone
411	Length Required
413	Content Too Large
414	URI Too Long
415	Unsupported Media Type
416	Range Not Satisfiable
417	Expectation Failed
422	Unprocessable Content
426	Upgrade Required
429	Too Many Requests
500	Internal Server Error
501	Not Implemented
502	Bad Gateway
503	Service Unavailable
504	Gateway Timeout
505	HTTP Version Not Supported

Other status codes use `"Unknown"` as the reason phrase.

HTTP Client

Client Functions

Function	Signature	Description
`http_get`	`(url: str) -> Result<HttpClientResponse, Error>`	Sends an HTTP GET request to the given URL.
`http_post`	`(url: str, body: str, headers: Map<str, str>) -> Result<HttpClientResponse, Error>`	Sends an HTTP POST request with body and headers.
`http_request`	`(method: str, url: str, headers: Map<str, str>, body: str) -> Result<HttpClientResponse, Error>`	Sends an HTTP request with a custom method.

Response Accessors

Function	Signature	Description
`status`	`(resp: HttpClientResponse) -> int`	Returns the HTTP status code.
`body`	`(resp: HttpClientResponse) -> str`	Returns the response body as a string. Truncated at the first NUL byte; use `body_bytes` for binary data.
`body_bytes`	`(resp: HttpClientResponse) -> List<u8>`	Returns the response body as a byte list. Binary-safe — preserves all bytes including NUL.
`header`	`(resp: HttpClientResponse, key: str) -> Option<str>`	Returns the value of a response header (case-insensitive). Returns `None` if not found.
`http_client_response_free`	`(resp: HttpClientResponse) -> Unit`	Frees the response and its associated memory. Call when done with the response.

Client Usage Example

```
from http import http_get, http_post, status, body, header

# Simple GET request
case http_get("http://example.com/api/data"):
    Ok(resp):
        s = status(resp)
        b = body(resp)
        print(to_str(s) + ": " + b)
    Err(e):
        print("Request failed")

# POST request with body and headers
headers: Map<str, str> = {"Content-Type": "application/json"}
case http_post("http://example.com/api/data", "{\"key\": \"value\"}", headers):
    Ok(resp):
        print(body(resp))
    Err(e):
        print("Request failed")
```

Client Behavior

- Both `http://` and `https://` URLs are supported. HTTPS uses TLS with system CA bundle certificate validation.
- The `Host` header is automatically added based on the URL.
- `Connection: close` is always sent; each request uses a separate TCP connection.
- `Content-Length` is always added automatically (including `0` for empty bodies). User-provided `Content-Length` headers are overridden with the correct value.
- Response body reading supports `Content-Length`, `Transfer-Encoding: chunked`, or read-until-close.
- Response header lookup is case-insensitive.
- Connection timeout is 5 seconds; receive timeout is 30 seconds.
- Returns `Err` on connection failure, invalid URL, or malformed response.
- `HttpClientResponse` owns allocated memory (headers, body). Call `http_client_response_free()` when done to avoid memory leaks.

Redirect Behavior

HTTP client functions automatically follow redirect responses (3xx with `Location` header).

- **Supported status codes**: 301, 302, 303, 307, 308
- **Maximum redirects**: 10 (returns `Err` if exceeded)
- **Method conversion** (per RFC 9110):
  - 301, 302: `POST` is changed to `GET` (body is dropped); other methods are preserved
  - 303: Method is always changed to `GET` (body is dropped)
  - 307, 308: Method and body are preserved
- **URL resolution**: Absolute URLs, protocol-relative (`//...`), absolute paths (`/...`), and relative paths in `Location` headers are supported
- User-provided headers are re-sent on each redirect hop, except sensitive headers (`Authorization`, `Proxy-Authorization`, `Cookie`) which are stripped on cross-origin redirects (different host or port)
- If the `Location` header is missing or empty, the redirect response is returned as-is
- The caller receives only the final response after all redirects are followed

Error Handling

- `listen()` raises a runtime error if `bind()` fails (e.g., port already in use).
- Malformed requests or idle timeouts on a keep-alive connection cause the connection to be closed. The server then resumes accepting new connections.
- The handler function must always return an `HttpResponse` —there is no default response.
- Client functions return `Result<HttpClientResponse, Error>` —use `case` to handle success and failure.

English | 日本語| 繁體中文

I/O Function Reference

Standard I/O and file operations. All functions require explicit import from `io`.

```
from io import read_text, write_text, exists
```

Function List

Standard Input

Function	Signature	Description
`read_line`	`() -> str`	Reads one line from stdin (trailing newline removed)
`read_all`	`() -> str`	Reads all of stdin until EOF

File I/O

Function	Signature	Description
`read_text`	`(str) -> Result<str, Error>`	Reads entire file as a string
`write_text`	`(str, str) -> Result<Unit, Error>`	Writes a string to a file (overwrites)
`append_text`	`(str, str) -> Result<Unit, Error>`	Appends a string to the end of a file
`exists`	`(str) -> bool`	Checks if a file exists
`delete_file`	`(str) -> Result<Unit, Error>`	Deletes a file

Function	Signature	Description
`read_bytes`	`(str) -> Result<List<u8>, Error>`	Reads a file as a byte list
`write_bytes`	`(str, List<u8>) -> Result<Unit, Error>`	Writes a byte list to a file

Byte Conversions

Function	Signature	Description
`to_bytes`	`(str) -> List<u8>`	Converts a string to UTF-8 bytes
`bytes_to_str`	`(List<u8>) -> Result<str, Error>`	Converts a byte list to a string

Examples

Reading and Writing Files

```
from io import read_text, write_text, append_text, exists, delete_file

case write_text("hello.txt", "Hello, World!"):
    Ok(_):
        case read_text("hello.txt"):
            Ok(content):
                print(content)    # Hello, World!
            Err(e):
                print(e.message)
    Err(e):
        print(e.message)

print(exists("hello.txt"))    # true

case delete_file("hello.txt"):
    Ok(_):
        print(exists("hello.txt"))    # false
    Err(e):
        print(e.message)
```

Byte Operations

```
from io import to_bytes, bytes_to_str, write_bytes, read_bytes

bs = to_bytes("ABC")
print(length(bs))    # 3

case write_bytes("data.bin", bs):
    Ok(_):
        case read_bytes("data.bin"):
            Ok(rb):
                case bytes_to_str(rb):
                    Ok(s):
                        print(s)          # ABC
                    Err(e):
```

```
                    print(e.message)
            Err(e):
                print(e.message)
    Err(e):
        print(e.message)
```

Reading from Standard Input

```
from io import read_line

name = read_line()
print(f"Hello, {name}!")
```

Error Handling

File operations return `Result<T, Error>` instead of terminating on failure. Use `case` with `Ok`/`Err` patterns to handle errors:

```
case read_text("missing.txt"):
    Ok(content):
        print(content)
    Err(e):
        print(e.message)    # cannot open file 'missing.txt' for reading
```

Operation	Error Condition
`read_text` / `read_bytes`	File does not exist or cannot be opened
`write_text` / `write_bytes` / `append_text`	File cannot be opened for writing
`delete_file`	File cannot be deleted
`bytes_to_str`	Input contains NUL byte

Notes

- `List<u8>` is used as the buffer type. Standard list operations (`length()`, `append()`, `slice()`, index access) all work with byte lists.
- File paths are relative to the current working directory unless absolute paths are specified.
- `write_text` and `write_bytes` overwrite existing files. Use `append_text` to add content to existing files.

English | 日本語 | 繁體中文

JSON Function Reference

JSON parsing and serialization. All functions require explicit import from `json`.

```
from json import parse, stringify, kind, get, at, to_str, to_int, to_float, to_bool, length, keys, json_
```

Overview

The `json` package provides functions to parse JSON text into an opaque `JsonValue` type, access its contents via accessor functions, and serialize it back to text. Since JSON values can be heterogeneous (objects can contain strings, numbers, booleans, arrays, and nested objects), the package uses an opaque pointer type with typed accessor functions.

Function List

Parse / Stringify

Function	Signature	Description
`parse`	`(str) -> Result<JsonValue, Error>`	Parses a JSON string into a JsonValue
`stringify`	`(JsonValue) -> str`	Serializes a JsonValue to compact JSON text
`stringify`	`(JsonValue, int) -> str`	Serializes with pretty printing (indent = number of spaces)

Type Query

Function	Signature	Description
`kind`	`(JsonValue) -> str`	Returns the JSON type: `"object"`, `"array"`, `"string"`, `"number"`, `"boolean"`, or `"null"`

Object / Array Access

Function	Signature	Description
`get`	`(JsonValue, str) -> Result<JsonValue, Error>`	Gets a field from an object by key
`at`	`(JsonValue, int) -> Result<JsonValue, Error>`	Gets an element from an array by index

Value Extraction

Function	Signature	Description
`to_str`	`(JsonValue) -> Result<str, Error>`	Extracts a string value
`to_int`	`(JsonValue) -> Result<int, Error>`	Extracts an integer value
`to_float`	`(JsonValue) -> Result<float, Error>`	Extracts a float value
`to_bool`	`(JsonValue) -> Result<bool, Error>`	Extracts a boolean value

Collection Info

Function	Signature	Description
`length`	`(JsonValue) -> int`	Returns the length of an array or number of keys in an object
`keys`	`(JsonValue) -> Result<List<str>, Error>`	Returns the keys of an object, or an error if the value is not an object

Memory Management

Function	Signature	Description
`json_free`	`(JsonValue) -> Unit`	Frees a JsonValue and all its children

Unwrapping `Result<JsonValue, Error>`

`parse`, `get`, and `at` return `Result<JsonValue, Error>`. You must unwrap the `Result` before passing the inner value to another json function — passing the `Result` directly is rejected at compile time:

```
case parse(text):
  Ok(doc):
    # ☒ error: kind() requires a JsonValue argument
    # kind(get(doc, "name"))
    # ✓ unwrap first
    case get(doc, "name"):
      Ok(name_val):
        print(kind(name_val))
      Err(e):
        print("no name")
  Err(e):
    print("parse error")
```

Generic stringification (`to_str(result)`, `print(result)`, f-string interpolation) still works on a `Result` and formats as `Ok(...)` / `Err(...)`, matching the behavior for any other `Result` value.

Usage Examples

Parsing and accessing fields

```
from json import parse, get, to_str, to_int, json_free

case parse("{\"name\": \"Alice\", \"age\": 30}"):
  Ok(data):
    case get(data, "name"):
      Ok(val):
        case to_str(val):
          Ok(name):
            print(name)   # "Alice"
          Err(e):
            print("error")
      Err(e):
        print("error")
    json_free(data)
  Err(e):
    print("parse error: " + e.message)
```

Working with arrays

```
from json import parse, at, to_int, length, json_free

case parse("[10, 20, 30]"):
  Ok(data):
    print(to_str(length(data)))   # 3
    case at(data, 0):
```

```
      Ok(elem):
        case to_int(elem):
          Ok(n):
            print(to_str(n))    # 10
          Err(e):
            print("error")
      Err(e):
        print("error")
    json_free(data)
  Err(e):
    print("parse error")
```

Stringify with pretty printing

```
from json import parse, stringify, json_free

case parse("{\"key\":\"value\",\"count\":42}"):
  Ok(data):
    print(stringify(data, 2))
    # {
    #   "key": "value",
    #   "count": 42
    # }
    json_free(data)
  Err(e):
    print("error")
```

Notes

- `to_int` accepts both JSON integers and floats that are whole numbers (e.g., `42.0` → `42`)
- `to_float` accepts both JSON floats and integers (e.g., `42` → `42.0`)
- `get` and `at` return references to child values within the parsed tree — do not call `json_free` on child values, only on the root value returned by `parse`
- The `kind` function returns `"number"` for both integers and floats

English | 日本語 | 繁體中文

Math Functions (`math`)

Overview

The `math` package provides mathematical constants and functions. Unlike the `std` package, it is not automatically imported. Use explicit import to access the functions.

```
from math import sqrt, PI, sin
```

Constants

Constant	Type	Description
`PI`	`float`	Pi (3.141592653589793)
`E`	`float`	Euler' s number (2.718281828459045)
`Inf`	`float`	Positive infinity
`NaN`	`float`	Not a Number

```
from math import PI, E, Inf, NaN

circumference = 2.0 * PI * radius
```

Absolute Value

Function	Signature	Description
`abs(x)`	`(int) -> int`	Absolute value of integer
`abs(x)`	`(float) -> float`	Absolute value of float

```
from math import abs

abs(-5)      # 5
abs(-3.14)   # 3.14
```

Rounding

Function	Signature	Description
`floor(x)`	`(float) -> int`	Round down to nearest integer
`floor(x, digits)`	`(float, int) -> float`	Round down to given decimal places
`ceil(x)`	`(float) -> int`	Round up to nearest integer
`ceil(x, digits)`	`(float, int) -> float`	Round up to given decimal places
`round(x)`	`(float) -> int`	Round to nearest integer (half away from zero)
`round(x, digits)`	`(float, int) -> float`	Round to given decimal places (half away from zero)

```
from math import floor, ceil, round

floor(3.7)           # 3
ceil(3.2)            # 4
round(3.5)           # 4

round(3.14159, 2)    # 3.14
floor(3.789, 1)      # 3.7
ceil(3.123, 1)       # 3.2
```

The two-argument forms accept negative `digits` for rounding to powers of ten:

```
round(1234.5, -2)    # 1200.0
round(1750.0, -3)    # 2000.0
```

Rounding uses C99 half-away-from-zero semantics (via `round(x * 10^digits) / 10^digits`), matching the one-argument form. This differs from Python's banker's rounding — for example, `round(2.675, 2) == 2.68`, not `2.67`. `NaN` and `±Inf` pass through unchanged.

Power and Root

Function	Signature	Description
`sqrt(x)`	`(float) -> float`	Square root
`pow(x, y)`	`(float, float) -> float`	x raised to the power of y
`pow(x, y)`	`(int, int) -> int`	Integer exponentiation via fast-exponentiation

```
from math import sqrt, pow

sqrt(9.0)       # 3.0
pow(2.0, 3.0)   # 8.0
pow(2, 10)      # 1024
pow(-2, 3)      # -8
```

The integer overload raises a runtime error when the exponent is negative (`pow(2, -1)` aborts with `pow() integer exponent must be non-negative`). Overflow wraps silently, matching Ry' s existing integer arithmetic model.

Logarithm

Function	Signature	Description
`log(x)`	`(float) -> float`	Natural logarithm (base e)
`log(x, base)`	`(float, float) -> float`	Logarithm with arbitrary base
`log2(x)`	`(float) -> float`	Base-2 logarithm
`log10(x)`	`(float) -> float`	Base-10 logarithm

```
from math import log

log(8.0, 2.0)      # 3.0
log(100.0, 10.0)   # 2.0
```

`log(x, base)` is computed as `log(x) / log(base)`, so domain errors on either argument propagate as `NaN` or `-Inf`.

Exponential

Function	Signature	Description
`exp(x)`	`(float) -> float`	e raised to the power of x

Trigonometric

Function	Signature	Description
`sin(x)`	`(float) -> float`	Sine (x in radians)
`cos(x)`	`(float) -> float`	Cosine (x in radians)
`tan(x)`	`(float) -> float`	Tangent (x in radians)

Inverse Trigonometric

Function	Signature	Description
`asin(x)`	`(float) -> float`	Arc sine (result in radians)
`acos(x)`	`(float) -> float`	Arc cosine (result in radians)
`atan(x)`	`(float) -> float`	Arc tangent (result in radians)

Two-argument Functions

Function	Signature	Description
`atan2(y, x)`	`(float, float) -> float`	Arc tangent of y/x (result in radians)
`hypot(x, y)`	`(float, float) -> float`	Hypotenuse: sqrt(x^2 + y^2)

Predicates

Function	Signature	Description
`is_nan(x)`	`(float) -> bool`	True if x is NaN
`is_inf(x)`	`(float) -> bool`	True if x is positive or negative infinity

```
from math import is_nan, is_inf, NaN, Inf

is_nan(NaN)   # true
is_inf(Inf)   # true
is_nan(1.0)   # false
```

English

Network (TCP) Reference

Types

Type	Description
`TcpListener`	Opaque handle for a TCP server socket
`TcpStream`	Opaque handle for a TCP connection
`TlsStream`	Opaque handle for a TLS-encrypted TCP connection

All types are opaque pointers managed by ARC (Automatic Reference Counting). They cannot be constructed directly; use `bind()` or `connect()` to obtain `TcpListener`/`TcpStream`, and `tls_connect()` for `TlsStream`. Resources are automatically closed when no longer referenced — explicit `close()` calls are optional but supported for immediate cleanup.

Functions (from `net`)

These functions require explicit import:

```
from net import bind, listen, accept, connect, listener_port, shutdown, set_timeout, set_receive_timeout
```

Function	Signature	Description
`bind`	`(host: str, port: int) -> Result<TcpListener, Error>`	Creates a TCP server socket bound to the given address. Returns `Err` on failure. Use port `0` for dynamic allocation.
`listen`	`(listener: TcpListener, backlog: int) -> Result<Unit, Error>`	Starts listening for incoming connections. Returns `Err` on failure.
`accept`	`(listener: TcpListener) -> Result<TcpStream, Error>`	Accepts a new connection. Waits up to 1 second for a client to connect. Returns `Err` on timeout or failure.
`connect`	`(host: str, port: int) -> Result<TcpStream, Error>`	Connects to a remote TCP server. Times out after 5 seconds. Returns `Err` on timeout or failure.
`listener_port`	`(listener: TcpListener) -> int`	Returns the actual port number the listener is bound to. Useful when binding to port `0` (OS-assigned port).
`shutdown`	`(listener: TcpListener) -> Unit`	Signals the listener to stop accepting connections. Causes any pending `accept()` to return within at most 1 second.
`tls_connect`	`(host: str, port: int) -> Result<TlsStream, Error>`	Connects to a remote server with TLS encryption. Validates the server certificate against the system CA bundle. Returns `Err` on connection or handshake failure.
`set_timeout`	`(stream: TcpStream|TlsStream, ms: int) -> Unit`	Sets both receive and send timeout in milliseconds.
`set_receive_timeout`	`(stream: TcpStream|TlsStream, ms: int) -> Unit`	Sets the receive timeout in milliseconds. `receive()` returns `Err` if no data arrives within this time.
`set_send_timeout`	`(stream: TcpStream|TlsStream, ms: int) -> Unit`	Sets the send timeout in milliseconds.

Built-in Overloaded Functions

These functions are built-in and work with TCP socket types. No import needed.

Function	Signature	Description
`send`	`(stream: TcpStream|TlsStream, data: List<u8>) -> Result<int, Error>`	Sends bytes through a TCP or TLS connection. Returns `Ok` with the number of bytes sent, or `Err` on failure.

Function	Signature	Description
`receive`	`(stream: TcpStream|TlsStream, max: int) -> Result<List<u8>, Error>`	Receives up to `max` bytes. Returns `Ok` with an empty list on connection close, or `Err` on error.
`close`	`(handle: TcpStream|TlsStream) -> Unit`	Closes a TCP or TLS stream.
`close`	`(handle: TcpListener) -> Unit`	Closes a TCP listener.

Usage Example

Echo Server

```
from net import bind, listen, accept, connect
from io import to_bytes, bytes_to_str

# Server
case bind("127.0.0.1", 8080):
    Ok(server):
        case listen(server, 128):
            Ok(_):
                case accept(server):
                    Ok(conn):
                        case receive(conn, 4096):
                            Ok(data):
                                case send(conn, data):
                                    Ok(_):
                                        ...
                                    Err(e):
                                        print(e.message)
                            Err(e):
                                print(e.message)
                        close(conn)
                    Err(e):
                        ...
            Err(e):
                print("listen failed")
        close(server)
    Err(e):
        print("bind failed")
```

Client

```
case connect("127.0.0.1", 8080):
    Ok(conn):
        case send(conn, to_bytes("hello")):
            Ok(_):
                ...
            Err(e):
                print(e.message)
        case receive(conn, 4096):
            Ok(resp):
```

```
                case bytes_to_str(resp):
                    Ok(s):
                        print(s)
                    Err(e):
                        print(e.message)
            Err(e):
                print(e.message)
        close(conn)
    Err(e):
        print("connect failed")
```

Concurrent Echo Server with `async function`

```
from net import bind, listen, accept, connect, listener_port
from io import to_bytes, bytes_to_str

async function echo_server(server: TcpListener) -> str:
    case accept(server):
        Ok(conn):
            case receive(conn, 4096):
                Ok(data):
                    case send(conn, data):
                        Ok(_):
                            ...
                        Err(e):
                            ...
                Err(e):
                    ...
            close(conn)
        Err(e):
            ...
    close(server)
    return "done"

case bind("127.0.0.1", 0):
    Ok(server):
        case listen(server, 1):
            Ok(_):
                port = listener_port(server)
                t = echo_server(server)
                # ... client code using port ...
                block_on(t)
            Err(e):
                ...
    Err(e):
        ...
```

Timeout Configuration

By default, `receive()` uses a 30-second timeout if no custom timeout is set. Use `set_timeout()`, `set_receive_timeout()`, or `set_send_timeout()` to override the default:

```
from net import connect, set_receive_timeout

case connect("127.0.0.1", 8080):
```

```
    Ok(conn):
        set_receive_timeout(conn, 5000)  # 5-second timeout
        case receive(conn, 4096):
            Ok(data):
                ...
            Err(e):
                print("timeout or error")
        close(conn)
    Err(e):
        print("connect failed")
```

Pass `0` to disable the timeout (wait indefinitely).

Error Handling

- All TCP functions except `close()` return `Result<T, Error>` — use `case` with `Ok`/`Err` to handle failure.
- `receive()` returns `Ok` with an empty `List<u8>` when the connection is closed by the peer, and `Err` on actual errors (timeout, socket error).
- `close()` closes the socket and frees the handle. Using a handle after close is undefined behavior.

Byte Conversion

TCP operations work with `List<u8>`. Use `to_bytes()` and `bytes_to_str()` from `io` to convert between strings and byte lists.

English | 日本語 | 繁體中文

Operator Reference

Precedence Table

Lower numbers indicate higher precedence (evaluated first).

Precedence	Operator	Description	Associativity
0	`? !!`	Error propagation (postfix)	Left
1	`()`	Grouping	–
2	`+x -x ~x`	Unary plus, unary minus, bitwise NOT	Right
3	`**`	Exponentiation	Right
3.5	`as`	Type cast	Left
4	`* / % //`	Multiplication, division, modulo, integer division	Left
5	`+ -`	Addition, subtraction	Left
6	`<< >> >>>`	Bit shift	Left
7	`&`	Bitwise AND	Left
8	`^`	Bitwise XOR	Left
9	`|`	Bitwise OR	Left
10	`== != < <= > >= in not in`	Comparison, membership	Left
11	`not`	Logical NOT	Right
12	`and`	Logical AND	Left
13	`or`	Logical OR	Left
13.5	`??`	Null coalescing	Left

Arithmetic Operators

Operator	Description	Example
`+`	Addition / string concatenation	`1 + 2 -> 3`, `"a" + "b" -> "ab"`, `"x" + 1 -> "x1"`
`-`	Subtraction	`5 - 3 -> 2`
`*`	Multiplication / string repetition	`4 * 3 -> 12`, `"ab" * 3 -> "ababab"`
`/`	Division (always float)	`7 / 2 -> 3.5`
`//`	Floor division (toward -∞)	`7 // 2 -> 3`, `-7 // 2 -> -4`
`%`	Modulo	`7 % 3 -> 1`
`**`	Exponentiation (always float)	`2 ** 10 -> 1024.0`
`-x`	Unary minus	`-5`, `-3.14`
`+x`	Unary plus	`+5` (no sign change)

```
a = 10 // 3    # 3 (int)
b = 10 / 3     # 3.3333... (float)
c = 2 ** 8     # 256.0 (float)
s = "foo" + "bar"  # "foobar"
t = "val=" + 42    # "val=42"
u = 3.14 + "!"     # "3.14!"
```

When one operand of `+` is `str` and the other is `int`, `float`, or `bool`, the non-`str` operand is automatically converted to its string representation and concatenated.

Comparison Operators

All return `bool`.

Operator	Description
`==`	Equal
`!=`	Not equal
`<`	Less than
`<=`	Less than or equal
`>`	Greater than
`>=`	Greater than or equal

- Can be used with numeric types (int / float) and bool.
- `str` values are compared lexicographically (byte order).
- Record types support `==` and `!=` with auto-generated field-by-field comparison (see Struct Reference).
- The `in` operator is used for membership checks on sets, lists, and maps (`x in s`).
- The `not in` operator is the negation of `in` (`x not in s`).
- For maps, `in` checks whether the key exists.

```
x = 3 < 5        # true
y = "abc" < "abd"  # true (lexicographic)
s = {1, 2, 3}
z = 2 in s       # true
w = 4 not in s   # true
xs = [1, 2, 3]
a = 2 in xs      # true (list linear search)
m = {"a": 1}
b = "a" in m     # true (map key lookup)
```

Logical Operators

Operator	Description	Type
`and`	Logical AND	`bool x bool -> bool`
`or`	Logical OR	`bool x bool -> bool`
`not`	Logical NOT	`bool -> bool`

```
a = true and false    # false
b = true or false     # true
c = not true          # false
```

Bitwise Operators

Only available for `int` type. Applying to `float` or `bool` causes a compile error.

Operator	Description	Example
`&`	Bitwise AND	`0b1100 & 0b1010 -> 0b1000`
`|`	Bitwise OR	`0b1100 | 0b1010 -> 0b1110`
`^`	Bitwise XOR	`0b1100 ^ 0b1010 -> 0b0110`
`~`	Bitwise NOT (unary)	`~0 -> -1`
`<<`	Left shift	`1 << 4 -> 16`
`>>`	Arithmetic right shift	`16 >> 2 -> 4`
`>>>`	Logical right shift	`-1 >>> 1 -> 9223372036854775807`

```
flags = 0b0001 | 0b0010    # 3
masked = flags & 0b0011    # 3
shifted = 1 << 8           # 256
```

Error Propagation Operator (`?` / `!!`)

The postfix `?` operator unwraps a `Result` or `Option` value on the happy path and short-circuits on the unhappy path. The `!!` operator is an alias for `?` with identical semantics.

Operand	Happy path	Unhappy path
`Result<T, E>`	evaluates to the `Ok` inner value `v`	the enclosing function returns `Err(e)`
`Option<T>`	evaluates to the `Some` inner value `v`	the enclosing function returns `None`

When used inside a function, the operand type must match the enclosing function' s return type:

- `?` on a `Result` value requires the enclosing function to return `Result`.
- `?` on an `Option` value requires the enclosing function to return `Option`.

```
function safe_divide(a: int, b: int) -> Result<int, Error>:
    if b == 0:
        return Err(Error("division by zero"))
    return Ok(a // b)

function compute(a: int, b: int, c: int) -> Result<int, Error>:
    x = safe_divide(a, b)?    # returns Err early if b == 0
    y = safe_divide(x, c)!!
```

```
    return Ok(y + 1)

function safe_get(xs: List<int>, i: int) -> Option<int>:
    if i < 0 or i >= xs.length():
        return none
    return Some(xs[i])

function first_plus_second(xs: List<int>) -> Option<int>:
    a = safe_get(xs, 0)?    # returns None early if out of range
    b = safe_get(xs, 1)?
    return Some(a + b)
```

At the top level

`?` and `!!` can also be used directly at the top level of a script. There, `Err(e)` and `None` are treated as fatal errors: the error message is written to stderr and the process exits with status `1`.

```
function mk() -> Result<int, Error>:
    return Err(Error("something broke"))

v = mk()?   # prints "error: something broke" to stderr and exits with 1

x: int? = none
y = x?      # prints "error: unexpected None" to stderr and exits with 1
```

At the top level, a `Result`'s `Err` type must be `Error` (so its `message` field can be printed).

`case`: Conditional Expression

```
x = case:
    condition => true_value
    _ => false_value
```

Evaluates conditions from top to bottom and returns the expression from the first truthy arm. All result expressions must have the same type. The `_ =>` wildcard arm is required, so the expression always produces a value.

```
x = case:
    3 > 2 => 10
    _ => 20     # 10

s = case:
    false => "yes"
    _ => "no"  # "no"

# Nested ternaries flatten into multiple arms
y = case:
    true => 2
    false => 1
    _ => 3     # 2
```

Range Operator

The `..` operator creates an inclusive integer range.

```
xs = 1 .. 5    # [1, 2, 3, 4, 5]

for i in 1 .. 3:
    print(i)       # 1 2 3
```

The result is a `List<int>` containing all integers from the left operand to the right operand (inclusive).

Null Coalescing Operator (??)

```
x = optional_val ?? default_val
```

The `??` operator accepts either an `Option<T>` or a `Result<T, E>` on the left-hand side:

Left-hand side	Result
`Some(v)`	`v`
`None`	`default_val`
`Ok(v)`	`v`
`Err(_)`	`default_val` (the error value is discarded)

The right-hand operand must have the same type as the `Option`'s inner type (or the `Result`'s `Ok` type).

```
a: int? = Some(10)
b: int? = none

print(a ?? 0)    # 10
print(b ?? 0)    # 0

function parse_int(s: str) -> Result<int, Error>:
    # ...

i = parse_int("42") ?? -1      # 42 on success, -1 on Err
j = parse_int("nope") ?? -1    # -1 — the Err value is discarded
```

Compound Assignment Operators

Shorthand for updating a variable. `x op= y` is equivalent to `x = x op y`.

Operator	Equivalent Expression
`x += y`	`x = x + y`
`x -= y`	`x = x - y`
`x *= y`	`x = x * y`
`x /= y`	`x = x / y`
`x %= y`	`x = x % y`
`x //= y`	`x = x // y`
`x **= y`	`x = x ** y`
`x &= y`	`x = x & y`
`x |= y`	`x = x | y`
`x ^= y`	`x = x ^ y`
`x <<= y`	`x = x << y`
`x >>= y`	`x = x >> y`

```
x = 10
x += 5     # x = 15
x -= 3     # x = 12
x *= 2     # x = 24
x //= 3   # x = 8
x &= 6    # x = 0
```

Compound assignment is allowed on any lvalue — plain variables, list or map elements, record fields, and arbitrarily nested chains:

```
xs = [1, 2, 3]
xs[0] += 10              # list element

record Point:
  x: int
  y: int
p = Point(1, 2)
p.x *= 5                 # record field

pts = [Point(1, 2), Point(3, 4)]
pts[0].x -= 1            # chained: list-of-records field
```

Each index expression on a chained LHS is evaluated exactly once. Compound assignment to a missing map key (`m["absent"] += 1`) is a runtime error.

Increment / Decrement Operators

Postfix-only, statement-level operators for incrementing or decrementing a variable by 1. These are desugared to `x = x + 1` and `x = x - 1` respectively.

Operator	Equivalent Expression
`x++`	`x = x + 1`
`x--`	`x = x - 1`

```
count = 0
count++        # count = 1
count++        # count = 2
count--        # count = 1

f = 1.5
f++            # f = 2.5 (int 1 is promoted to float)
```

> **Note**: `++` / `--` can only be used as statements, not as expressions. `@const` variables cannot be incremented/decremented (immutability is enforced).

Type Rules for Operations

Operation	Left Type	Right Type	Result Type
`+ - *`	int	int	int
`+ - *`	float	int / float	float
`+ - *`	int	float	float
`/`	any numeric	any numeric	float
`//`	int	int	int

Operation	Left Type	Right Type	Result Type
`//`	float or int (one is float)	–	float
`**`	any numeric	any numeric	float
`%`	int	int	int
`%`	float or int (one is float)	–	float
`+`	str	str	str
`+`	str	int / float / bool	str
`+`	int / float / bool	str	str
`== != < <= > >=`	numeric / bool / str	same type	bool
`*`	str	int	str
`in`	any	Set / List / Map<K, V>	bool
`not in`	any	Set / List / Map<K, V>	bool
`& | ^ ~ << >> >>>`	int	int	int
`and or not`	bool	bool	bool

Operator Overloading

You can define operator behavior for user-defined types.

Syntax

```
# Binary operator (2 parameters)
function operator+(a: MyType, b: MyType) -> MyType:
    ...

# Unary operator (1 parameter)
function operator-(a: MyType) -> MyType:
    ...
```

Overloadable Operators

Category	Operators
Arithmetic (binary)	`+ - * / % ** //`
Comparison (binary)	`== != < <= > >=`
Bitwise (binary)	`& | ^ << >> >>>`
Logical (binary)	`and or`
Membership	`in`
Subscript	`[]` (read), `[]=` (write)
Call	`()`
Cast	`as`
Unary	`- ~ not`
Compound assignment	`+= -= *= /= %= //= **= &= |= ^= <<= >>=`

Return Type Constraints

Comparison and logical operators must return `bool`:

Category	Operators	Required Return Type
Comparison	`== != < <= > >=`	`bool`
Logical	`and or not`	`bool`
Membership	`in`	`bool`

Category	Operators	Required Return Type
Cast	`as`	Required (target type)

```
# OK
function operator==(a: Vec2, b: Vec2) -> bool:
    return a.x == b.x and a.y == b.y

# Error: comparison operator '==' must return 'bool', but returns 'int'
function operator==(a: Vec2, b: Vec2) -> int:
    return 42
```

Arithmetic and bitwise operators have no return type constraint.

Distinguishing Binary and Unary

Distinguished by the number of parameters.

```
# Binary -
function operator-(a: Vec2, b: Vec2) -> Vec2:
    return Vec2(a.x - b.x, a.y - b.y)

# Unary -
function operator-(v: Vec2) -> Vec2:
    return Vec2(-v.x, -v.y)
```

Compound Assignment Operator Overloading

Compound assignment operators (`+=`, `-=`, etc.) can be independently overloaded. This enables in-place optimization for large data structures.

```
record Matrix:
    data: List
    rows: int
    cols: int

function operator+=(a: Matrix, b: Matrix) -> Matrix:
    for i in range(len(a.data)):
        a.data[i] = a.data[i] + b.data[i]
    return a
```

Resolution Priority When `x += y` is evaluated:

1. If `operator+=` is defined for the types → call it directly
2. If `operator+=` is not defined but `operator+` is → fall back to `x = x + y`
3. If neither is defined (for non-builtin types) → compile error

```
record Vec2:
    x: float
    y: float

function operator+=(a: Vec2, b: Vec2) -> Vec2:
    return Vec2(a.x + b.x, a.y + b.y)

v = Vec2(1.0, 2.0)
v += Vec2(3.0, 4.0)  # calls operator+= directly
# v.x == 4.0, v.y == 6.0
```

Compound assignment operators require exactly 2 parameters and have no return type constraint.

Subscript Operator Overloading

The `[]` (read) and `[]=` (write) operators enable custom subscript behavior for user-defined types. Multi-index access (e.g., `m[row, col]`) is supported.

```
record Grid:
    a: int
    b: int
    c: int
    d: int

# Read: requires 2+ parameters (object + indices)
function operator[](g: Grid, row: int, col: int) -> int:
    if row == 0 and col == 0:
        return g.a
    if row == 0 and col == 1:
        return g.b
    if row == 1 and col == 0:
        return g.c
    return g.d

# Write: requires 3+ parameters (object + indices + value)
function operator[]=(g: Grid, row: int, col: int, value: int):
    ...

g = Grid(1, 2, 3, 4)
print(g[0, 1])    # 2
g[1, 0] = 99
```

User-defined subscript operators are tried first; if no match is found, built-in subscript behavior (for lists, maps, and arrays) is used as a fallback.

Membership Operator Overloading

The `in` operator can be overloaded to define custom membership tests. Must return `bool`.

```
record Range:
    lo: int
    hi: int

function operator in(value: int, r: Range) -> bool:
    return value >= r.lo and value < r.hi

r = Range(1, 10)
print(5 in r)       # true
print(15 not in r)  # true
```

User-defined `in` operators are tried first; if no match is found, built-in behavior (for sets, maps, and lists) is used as a fallback. `not in` is automatically supported when `in` is defined.

Call Operator Overloading

The `()` operator enables records to behave as callable objects. Requires at least 2 parameters (object + arguments).

```
record Adder:
    base: int

function operator()(a: Adder, x: int) -> int:
    return a.base + x

add5 = Adder(5)
print(add5(10))    # 15
```

When a variable holding a record value is called like a function, the compiler tries `operator()` overloads first. If no match is found, other call resolution strategies (functions, constructors, lambdas) take precedence.

Cast Operator Overloading

The `as` operator can be overloaded to define custom type conversions. Takes exactly 1 parameter (the source value) and must specify a return type (the target type). Dispatch matches by source type and return type.

```
record Celsius:
    value: int

record Fahrenheit:
    value: int

function operator as(c: Celsius) -> Fahrenheit:
    return Fahrenheit(c.value * 9 // 5 + 32)

c = Celsius(100)
f = c as Fahrenheit    # Fahrenheit(212)
```

The target type can be any type the compiler can resolve, including generic types:

```
record Temperature:
    value: int

function operator as(t: Temperature) -> int?:
    return Some(t.value)

t = Temperature(42)
result: int? = t as int?    # Some(42)
```

User-defined `as` operators are tried first; if no match is found, built-in casts (int, float, bool, str, etc.) are used as a fallback.

English | 日本語 | 繁體中文

Package Reference

Overview

Ry uses a package system to organize code. A **package** can be either a single `.ry` file or a directory containing multiple `.ry` files. Use the `from` statement to import packages.

The `std` package (standard library) is automatically imported into every program.

Import Syntax

Import All Definitions

```
from math
```

Imports all functions and types from the package.

Selective Import

```
from math import sqrt
```

Imports only the specified definition.

Multiple Selective Import

```
from math import sqrt, PI
```

Imports multiple definitions separated by commas.

Relative Import

```
from .helper import greet
```

Imports from a module relative to the current file's directory. The `.` prefix restricts resolution to the current directory only (standard library and other search paths are not searched).

Relative Import from Subdirectory

```
from .utils import helper_fn
from .utils.calc import add
```

Imports from a subdirectory relative to the current file's directory.

Relative Import All from Current Directory

```
from . import add, sub
```

Imports specific symbols from the current directory package (all `.ry` files in the directory, excluding `_`-prefixed and `.test.ry` files).

Package Resolution

Packages are resolved using dot-separated notation:

Import Statement	Resolution
`from math`	`math/` directory (package) or `math.ry` file
`from utils.math`	`utils/math/` directory or `utils/math.ry` file
`from str`	`str/` directory or `str.ry` file

Resolution Order

For each search path, the system checks: 1. **Directory** (`{path}/`) — if exists, all `.ry` files in the directory are loaded (package) 2. **File** (`{path}.ry`) —single file (backward compatible)

Directory Packages

When a package resolves to a directory: - All `.ry` files in the directory are automatically loaded - Files starting with `_` are excluded - Test files (`.test.ry`) are excluded - No special entry file (like `__init__.py`) is needed - All functions and types defined in the directory' s files are exported

Private Symbols

Definitions whose names start with `_` (underscore) are private to the package and cannot be imported:

- Wildcard imports (`from pkg`) automatically exclude `_`-prefixed symbols
- Named imports (`from pkg import _helper`) produce a compile error

```
# mylib/internal.ry
function _helper() -> int:     # private — not importable
    return 42
function public_api() -> int:  # public — importable
    return _helper()
```

```
mypackage/
  calc.ry      # function add(), function sub()
  string.ry    # function concat()
```

```
from mypackage          # imports add, sub, concat
from mypackage import add   # imports only add
```

Standard Library (`std`)

The `std` package is automatically imported into every program. It provides: - Built-in functions (`print`, `length`, `range`, etc.) - String functions (`contains`, `find`, `replace`, etc.) - Type conversion functions (`to_int`, `to_float`, `to_str`) - Collection functions (`map`, `filter`, `sort`, etc.)

Sub-packages

The following sub-packages require explicit import:

Package	Description
`math`	Mathematical constants and functions
`io`	File I/O, standard input, and byte conversions
`path`	File path operations (join, basename, dirname, etc.)

```
from math import sqrt, PI, sin
```

You can also explicitly import specific definitions from standard library packages:

```
from str import contains
```

RY_HOME

The standard library is installed at `$RY_HOME/share/std/`. The default value of `RY_HOME` is `~/.ry`.

```
export RY_HOME="$HOME/.ry"    # default
```

RY_ENV

The `RY_ENV` environment variable controls the runtime environment mode. You can also use the `--env=<value>` CLI flag.

Value	Alias	`.env` loading	Lib search
`prod`	`production`	Disabled	Project override for repo builds → `$RY_HOME/share` (fallback: `lib`) → `exe/../share` (fallback: `lib`) → `exe/share` (fallback: `lib`)
`dev`	`development`	`.env.dev` → `.env`	Same as `prod`
`test`	—	`.env.test` → `.env`	Same as `prod`
`staging`	—	`.env.staging` → `.env`	Same as `prod`
`internal`	—	`.env.internal` → `.env`	Project override for repo builds → `exe/../share` (fallback: `lib`) → `exe/share` (fallback: `lib`) (`$RY_HOME` skipped)
(unset) (default)	—	`.env` only	Same as `prod`

Aliases are automatically resolved to their canonical form. For example, `RY_ENV=production` is normalized to `prod`.

In `prod` mode, `.env` files are not loaded for security —production secrets should be managed via infrastructure-level environment variables (CI/CD, secret managers, etc.).

For other modes, `.env.<env>` is loaded first (if it exists), then `.env`. Environment-specific values take precedence because existing variables are not overwritten.

```
# Short form (recommended)
RY_ENV=dev ./build/ry app.ry

# Long form (backward compatible)
RY_ENV=development ./build/ry app.ry

# CLI flag
./build/ry --env=dev test

# prod mode: .env is NOT loaded
RY_ENV=prod ./build/ry app.ry

# Additional isolation when developing Ry itself
RY_ENV=internal ./build/ry test
```

When a `ry` executable is built inside the Ry source tree, it can use a repo-local stdlib override from the project's `package.toml`. This keeps repo builds aligned with the checked-out `share/std` even if `~/.ry/share/std` is older. Installed `ry` binaries ignore that override and continue to use `$RY_HOME/share/std`.

Search Path Priority

1. The directory of the importing file
2. Repo-local stdlib override from the current Ry checkout, when using a repo-built `ry`

3. `$RY_HOME/share` (standard library location, falls back to `$RY_HOME/lib` for legacy installs)
4. Executable-relative `share/` directories (falls back to `lib/` for legacy layouts)
5. Paths specified in the `RY_PATH` environment variable (colon-separated)

RY_PATH Environment Variable

Specifying colon-separated directories in `RY_PATH` adds them to the package search path.

```
export RY_PATH="/usr/local/ry/lib:/home/user/ry-packages"
```

Constraints

Constraint	Details
Allowed location	Top level only (not inside functions or blocks)
Duplicate imports	Automatically skipped (no error)
Circular imports	Compile error
Relative imports	`from .` and `from .pkg` resolve only against the current file' s directory
Parent directory imports	`from ..` is not supported
Package names	Only letters, digits, and underscores are allowed (no hyphens)

```
# Error example: Import inside a block
function main():
    from math   # Error: imports only allowed at top level

# OK: Importing the same package multiple times does not cause an error
from math
from math   # Skipped
```

Creating Package Files

Single File Package

```
# calc.ry
function add(a: int, b: int) -> int:
    return a + b

function sub(a: int, b: int) -> int:
    return a - b
```

```
# main.ry
from calc import add, sub

print(add(1, 2))   # 3
print(sub(5, 3))   # 2
```

Directory Package

```
mylib/
```

```
  calc.ry
  string.ry

# main.ry
from mylib import add, concat
```

Native Function Naming Convention

Stdlib package functions that are implemented as C runtime functions follow the `__ry_<package>_<function_name>` convention.

> **Note**: This convention applies to stdlib package functions (e.g., `base64`, `filesystem`, `path`). Built-in functions (e.g., `print`, `length`) and math functions use varied implementations (inline LLVM IR, libc calls) and do not follow this naming pattern.

Format

`__ry_<package>_<function_name>`

Rules

1. **Prefix**: `__ry_`
2. **Package**: The package name (e.g., `base64` from `from base64 import encode`)
3. **Function name**: The snake_case function name as declared in Ry
4. **Overloads**: When a function has multiple overloads with different arities, append the argument count as a suffix (e.g., `__ry_path_join2`, `__ry_path_join3`)
5. **Error getter**: Each package that returns `Result` types provides `__ry_<pkg>_get_last_error`

Examples

Ry declaration	C runtime function name
`base64::encode(data: str) -> str`	`__ry_base64_encode`
`filesystem::list_dir(path: str) -> Result<List<str>, Error>`	`__ry_filesystem_list_dir`
`path::join(a: str, b: str) -> str`	`__ry_path_join2`
`path::join(a: str, b: str, c: str) -> str`	`__ry_path_join3`

English | 日本語 | 繁體中文

Path Function Reference

File path operations. All functions require explicit import from `path`.

```
from path import join, basename, dirname, extension, resolve, is_absolute
```

Function List

Function	Signature	Description
`join`	`(str, str) -> str`	Joins two path segments
`join`	`(str, str, str) -> str`	Joins three path segments
`join`	`(str, str, str, str) -> str`	Joins four path segments

Function	Signature	Description
`basename`	`(str) -> str`	Extracts the filename component
`dirname`	`(str) -> str`	Extracts the directory component
`extension`	`(str) -> str`	Extracts the file extension (including dot)
`resolve`	`(str) -> Result<str, Error>`	Resolves a path to its absolute canonical form
`is_absolute`	`(str) -> bool`	Returns whether a path is absolute

Examples

Joining Paths

```
from path import join

p = join("/tmp", "data", "file.txt")
print(p)  # /tmp/data/file.txt

# Absolute second argument replaces the first
print(join("/tmp", "/usr"))  # /usr
```

Extracting Path Components

```
from path import basename, dirname, extension

p = "/home/user/docs/report.pdf"

print(basename(p))    # report.pdf
print(dirname(p))     # /home/user/docs
print(extension(p))   # .pdf
```

Extension Edge Cases

```
from path import extension

print(extension("archive.tar.gz"))  # .gz
print(extension(".gitignore"))      # (empty string — hidden file with no extension)
print(extension(".config.json"))    # .json
print(extension("Makefile"))        # (empty string)
```

Checking Absolute Paths

```
from path import is_absolute

print(is_absolute("/usr/local"))  # true
print(is_absolute("src/main.ry")) # false
```

Resolving Paths

```
from path import resolve

case resolve("/tmp"):
  Ok(p):
    print(p)  # /private/tmp (on macOS) or /tmp
  Err(e):
```

```
    print(e.message)

case resolve("/nonexistent"):
  Ok(p):
    print(p)
  Err(e):
    print(e.message)  # cannot resolve path '/nonexistent': No such file or directory
```

English | 日本語 | 繁體中文

Project Management

CLI Overview

```
ry <file.ry> [args...]              # Run a Ry script
echo '<code>' | ry -c               # Run code from stdin
ry test [options] [<file> | <dir>]  # Run tests
ry init                             # Initialize a project
ry new <project-name>               # Create a new project
ry run [<script-name>]              # Run a project script
ry fmt [options] [<file> | <dir>]   # Format source files
ry self-update [options]            # Update ry itself
```

Global Options

Option	Description
`-c`	Read and execute code from stdin
`-h`, `--help`	Show help
`-v`, `--version`	Show version
`--env=<env>`	Set environment (`production`|`development`|`internal`). Overrides the `RY_ENV` environment variable.

Entry Point Execution

When no file argument is given, `ry` looks for a `package.toml` in the current directory (or parent directories) and runs the file specified by the `entry` field:

```
ry                      # runs entry file (e.g. src/main.ry)
ry -- arg1 arg2         # runs entry file with arguments
```

If no `package.toml` is found or no `entry` field is set, `ry` prints help and exits.

Bare filename

If the first argument is a **single path component** whose name ends with `.ry` (for example `main.ry`) and no file with that name exists in the current working directory, `ry` searches the nearest `package.toml` project in order: **first** the project root directory (the directory containing `package.toml`), **then** each directory listed under `[paths]` (keys sorted alphabetically; keys starting with `_` are reserved and ignored for this search, except `_dev_stdlib` which is handled separately). The **first** existing regular file wins (for example `foo.ry` next to `package.toml` is chosen over `src/foo.ry` when both exist). If none match, `ry` reports that the file does not exist and lists the paths that were tried. Tokens without a `.ry` suffix are not resolved this way (so mistyped subcommands still show "unknown command").

If the argument is a **path with more than one component** (for example `src/foo.ry` or `./foo.ry`) and that path does not exist, `ry` reports **no such file** instead of treating it as an unknown subcommand.

The same rules apply to `ry test <file.ry>` when `<file.ry>` is a basename and does not exist relative to the current directory.

Stdin Execution

Use the `-c` flag to read and execute code from stdin:

```
echo 'print("hello")' | ry -c
```

`ry init` - Project Initialization

Initializes the current directory as a Ry project.

```
ry init
```

Generated Files and Directories

```
my-project/
  package.toml          # Project configuration file
  src/
    main.ry        # Entry point (sample code)
```

Behavior

1. Exits with an error if `package.toml` already exists
2. Creates the `src/` directory (if it doesn' t exist)
3. Generates `package.toml` (`name` is set to the current directory name)
4. Generates `src/main.ry` (skipped if it already exists)

`ry new` - Create a New Project

Creates a new directory and initializes it as a Ry project.

```
ry new my-project
```

Generated Files and Directories

```
my-project/
  package.toml          # Project configuration file
  src/
    main.ry        # Entry point (sample code)
```

Behavior

1. Exits with an error if no project name is given
2. Exits with an error if the directory already exists
3. Creates the `<project-name>/` directory
4. Creates the `src/` directory inside it
5. Generates `package.toml` (`name` is set to the given project name)
6. Generates `src/main.ry`

`ry run` - Run Project Scripts

Executes a script defined in the `[scripts]` section of `package.toml`.

```
ry run              # List all available scripts
ry run build        # Run the "build" script
ry run test         # Run the "test" script
```

Behavior

1. Searches for `package.toml` from the current directory upward
2. Without arguments, lists all available scripts and their commands
3. With a script name, executes the corresponding shell command via `/bin/sh -c`
4. The exit code of the executed command is propagated
5. If the script name is not found, shows an error with a list of available scripts

Notes

- Does not require LLVM initialization (fast startup)
- Commands are executed in the current working directory
- Shell features (pipes, redirects, etc.) are supported since commands run through the shell

---

`ry fmt` - Code Formatter

Formats `.ry` source files with consistent 2-space indentation and canonical style.

```
ry fmt                    # Format all .ry files in the project (requires package.toml)
ry fmt src/main.ry        # Format a single file
ry fmt src/               # Format all .ry files in a directory (recursive)
ry fmt --check            # Check if files are formatted (exit 1 if not)
ry fmt --check src/       # Check specific directory
```

Formatting Rules

- 2-space indentation per block level
- Spaces around binary operators (`a + b`, not `a+b`)
- Space after comma (`f(a, b)`, not `f(a,b)`)
- Blank line between top-level definitions (functions, records, enums)
- Comments are preserved

Behavior

1. Reads the source file, parses it into an AST, and re-emits with canonical formatting
2. Writes the formatted result back to the file (in-place)
3. With `--check`, only reports unformatted files and exits with code 1 if any are found (useful for CI)
4. Skips `.git/`, `build/`, and `node_modules/` directories during recursive formatting

Notes

- Does not require LLVM initialization (fast startup)
- Compound assignment operators (`+=`, `-=`, etc.) are represented in their desugared form (`x = x + expr`) after formatting, because the parser desugars them during parsing
- Hex (`0xFF`) and binary (`0b1010`) number literals are converted to decimal notation

---

`ry test` - Run Tests

Discovers and runs test files (`*.test.ry`). See Testing for full test syntax documentation.

```
ry test                          # Auto-discover and run all *.test.ry files
ry test tests/spec               # Run all tests under a directory
ry test test_file.ry             # Run a specific test file
ry test -p                       # Run tests in parallel
ry test -w                       # Watch mode: re-run on file change
ry test --coverage               # Collect line coverage information
```

Options

Option	Description
`-p`, `--parallel`	Run tests in parallel
`-w`, `--watch`	Watch for changes and re-run
`--coverage`, `--cov`	Collect coverage information
`-h`, `--help`	Show help

Behavior

1. Without arguments, searches for `package.toml` to find the project root and recursively discovers `*.test.ry` files (skipping `.git`, `build`, `node_modules`)
2. Exit code is 0 if all tests passed, 1 if any failed
3. `--coverage` with `--parallel` falls back to sequential execution

`ry self-update` - Self Update

Updates ry itself to the latest version. Downloads a binary from GitHub Releases and replaces the current executable.

```
ry self-update               # Update to the latest stable version
ry self-update --nightly     # Update to the latest nightly pre-release
ry self-update v0.0.1        # Update to a specified version
```

Behavior

1. Displays the current version
2. Resolves the target version based on arguments
   - No arguments: Latest stable release from GitHub (`/releases/latest`)
   - `--nightly`: Latest pre-release
   - Version specified: Release with the specified tag
3. If the current version is the same, exits with `"Already up to date."`
4. Downloads the binary and replaces the current executable

Security

Release archives are verified in two steps:

1. **Authenticity**: The `checksums.txt.sig` file is verified against the embedded Ed25519 public key.
   - If the signature file is **missing**, the update is aborted unless `RY_SKIP_SIGNATURE=1` is set.
   - If the signature file is **present but invalid**, the update is aborted regardless of `RY_SKIP_SIGNATURE`.
2. **Integrity**: The archive' s SHA-256 hash is compared against `checksums.txt`.

To bypass the failure caused by a missing signature file (not recommended), set `RY_SKIP_SIGNATURE=1`. This does **not** bypass verification when a signature file is present but invalid.

Notes

- Requires `curl` and `tar` commands
- If replacing the binary fails due to insufficient permissions, a message suggesting `sudo` is displayed (sudo is not invoked automatically)
- Downloads are performed to a temporary directory first; however, if the cross-filesystem `cp` fallback is interrupted, the destination binary may be left in a partial state

---

`package.toml` Configuration File

Describes project metadata and path settings in TOML format.

```toml
[project]
name = "my-project"
version = "0.1.0"
entry = "src/main.ry"

[paths]
src = "src"

[scripts]
build = "cmake --preset default && cmake --build build"
test = "./build/ry_tests"
clean = "rm -rf build"
```

`[project]` Section

Key	Description
`name`	Project name (directory name at initialization)
`version`	Version string
`entry`	Source file serving as the entry point

`[paths]` Section

Key	Description
`src`	Source code directory
(other keys)	Additional project-relative directories. Values must not be absolute and must not contain `..`. Together with `src`, these directories are used to resolve **bare filenames** for `ry <file>` and `ry test <file>` (see **Bare filename** above).
`_dev_stdlib`	Optional; development override for the standard library location (see tooling docs). Not used for bare-filename resolution.

`[scripts]` Section

Defines named scripts that can be executed with `ry run <name>`. Each key is a script name and the value is a shell command string.

Key	Description
`<name>`	Shell command to execute (run with `ry run <name>`)

TOML Subset Specification

`package.toml` supports the following TOML subset.

- Section headers: `[section]`
- Key-value pairs: `key = "value"` (string values only)
- Comments: From `#` to end of line
- Blank lines are ignored

English | 日本語 | 繁體中文

Regular Expression Reference

Regex Literal Syntax

Regex literals use the `/pattern/` syntax and produce a `Regex` type value:

```
from regex import is_match, split, replace

# Regex literals enable type-based overloading
"hello".is_match(/[a-z]+/)        # true
"a1b2c".split(/[0-9]/)         # ["a", "b", "c"]
"abc123".replace(/[0-9]+/, "X") # "abcX"
```

Regex literals can be stored in variables:

```
pat = /[a-z]+/
"hello".is_match(pat)  # true
```

The `/` inside a regex literal can be escaped with `\/`:

```
"a/b".is_match(/a\/b/)  # true
```

Division vs Regex

The lexer uses context to distinguish regex literals from division:

- After value-producing tokens (identifiers, numbers, string literals, `)` or `]`), `/` is parsed as division
- After operators, keywords, or delimiters that expect an expression (`(`, `[`, `,`, `=`), `/` starts a regex literal

```
x = 10 / 2         # division: 5
y = is_match("a", /a/) # regex literal
```

Function List

Regex Literal Functions (text-first, UFCS-compatible)

These functions take a `Regex` type pattern and use text-first argument order for UFCS:

Function	Signature	Description
`is_match`	`(str, Regex) -> bool`	Returns whether the entire text matches the pattern
`search`	`(str, Regex) -> int`	Returns the start position of the first match (-1 if not found)

Function	Signature	Description
`replace`	`(str, Regex, str) -> str`	Replaces all matches with a replacement string
`split`	`(str, Regex) -> List<str>`	Splits text by pattern matches
`find_all`	`(str, Regex) -> List<str>`	Returns all non-overlapping matches

```
from regex import is_match, search, replace, split, find_all

# Direct call
print(is_match("hello", /[a-z]+/))       # true

# UFCS (text.function(pattern))
print("abc123".search(/[0-9]+/))          # 3
print("abc123".replace(/[0-9]+/, "X"))    # abcX
parts = "hello world".split(/\s+/)
nums = "a1b2c3".find_all(/[0-9]/)
```

Legacy Functions (text-first)

The original `regex_*` functions remain available for backward compatibility. They take pattern strings (not regex literals) with text-first argument order, consistent with the regex literal API:

Function	Signature	Description
`regex_match`	`(text: str, pattern: str) -> bool`	Returns whether the entire text matches the pattern
`regex_search`	`(text: str, pattern: str) -> int`	Returns the start position of the first match (-1 if not found)
`regex_replace`	`(text: str, pattern: str, replacement: str) -> str`	Replaces all matches with a replacement string
`regex_split`	`(text: str, pattern: str) -> List<str>`	Splits text by pattern matches
`regex_find_all`	`(text: str, pattern: str) -> List<str>`	Returns all non-overlapping matches

```
print(regex_match("hello", "[a-z]+"))   # true
pos = regex_search("abc123", "[0-9]+")  # 3
```

Supported Pattern Syntax

Syntax	Description	Example
`abc`	Literal characters	`"hello"`
`.`	Any character (except newline)	`"a.c"` matches `"abc"`, `"aXc"`
`|`	Alternation	"cat|dog" matches `"cat"` or `"dog"`
`*`	Zero or more	`"a*"` matches `""`, `"a"`, `"aaa"`
`+`	One or more	`"a+"` matches `"a"`, `"aaa"`
`?`	Zero or one	`"a?"` matches `""` or `"a"`
`{n}`	Exactly n times	`"a{3}"` matches `"aaa"`
`{n,m}`	Between n and m times	`"a{2,4}"` matches `"aa"` to `"aaaa"`

Syntax	Description	Example
`{n,}`	At least n times	`"a{2,}"` matches `"aa"`, `"aaa"`, …
`*?`	Zero or more (non-greedy)	`".*?"` matches shortest
`+?`	One or more (non-greedy)	`".+?"` matches shortest
`??`	Zero or one (non-greedy)	`"a??"` prefers zero
`{n,m}?`	Range (non-greedy)	`"a{2,4}?"` prefers n times
`(...)`	Group	`"(ab)+"` matches `"abab"`
`[abc]`	Character class	`"[aeiou]"` matches vowels
`[a-z]`	Character range	`"[a-z]+"` matches lowercase words
`[^...]`	Negated character class	`"[^0-9]"` matches non-digits
`^`	Start of string anchor	`"^hello"`
`$`	End of string anchor	`"world$"`
`\d`	Digit `[0-9]`	`"\d+"` matches numbers
`\D`	Non-digit `[^0-9]`	
`\w`	Word character `[a-zA-Z0-9_]`	`"\w+"` matches identifiers
`\W`	Non-word character	
`\s`	Whitespace	`"\s+"` matches spaces/tabs
`\S`	Non-whitespace	
`\b`	Word boundary	`"\bword\b"` matches whole word
`\B`	Non-word boundary	`"\Bword"` matches inside a word
`(?i)`	Case-insensitive flag	`"(?i)hello"` matches `"HELLO"`
`\.`	Escaped special character	`"\."` matches literal `.`

Usage Examples

Range Quantifiers

```
print(regex_match("123-4567", "\\d{3}-\\d{4}"))  # true
print(regex_match("aaa", "a{2,4}"))               # true
print(regex_match("ababab", "(ab){2,}"))          # true
```

Non-Greedy (Lazy) Match

```
# Greedy: matches longest
g = regex_replace("\"a\" and \"b\"", "\".*\"", "X")
print(g)  # X

# Non-greedy: matches shortest
l = regex_replace("\"a\" and \"b\"", "\".*?\"", "X")
print(l)  # X and X

# Find individual HTML-like tags
tags = regex_find_all("<a> <bb> <ccc>", "<.*?>")
print(length(tags))  # 3
```

> **Note:** Non-greedy matching controls the overall match length. Without support for extracting parenthesized groups, mixed greedy/lazy patterns may behave differently from PCRE-style engines.

Word Boundary

```
# Match whole words only
pos = regex_search("hello world", "\\bworld\\b")
print(pos)  # 6

# Find all words
words = regex_find_all("hello world foo", "\\b\\w+\\b")
print(length(words))  # 3
```

Case-Insensitive Matching

```
# (?i) at the start of pattern enables case-insensitive matching
print(regex_match("HELLO", "(?i)hello"))  # true
print(regex_match("Hello", "(?i)hello"))  # true
```

> **Note:** `(?i)` must appear at the beginning of the pattern and applies to the entire pattern. Partial case-insensitive matching (e.g., `(?i:sub)pattern`) is not supported.

English | 日本語 | 繁體中文

Record Reference

Overview

Structs are value types allocated on the stack. They are defined with the `record` keyword. Structs can have `invariant` clauses for Design by Contract. See Design by Contract.

> **Naming convention**: Struct names must use PascalCase (e.g., `Point`, `Rectangle`). Field names must use snake_case. The compiler enforces these conventions.

Definition Syntax

```
record TypeName:
    field_name: type
    field_name: type
```

Example

```
record Point:
    x: int
    y: int

record Rectangle:
    width: float
    height: float
```

Constructor

Arguments are passed in the order of field definitions. Named arguments are not supported.

```
p = Point(10, 20)
r = Rectangle(3.0, 4.5)
```

Field Access

Fields are read using dot notation.

```
p = Point(10, 20)
print(p.x)   # 10
print(p.y)   # 20
```

Field Assignment

Variable Declaration	Field Assignment
Mutable (no `@const`)	Allowed
`@const`	Compile error

```
p = Point(10, 20)
p.x = 100    # OK: mutable variable

@const
q = Point(10, 20)
q.x = 100    # Error: fields of @const variables cannot be modified
```

Chained and Deep Field Assignment

Field assignment composes across nested records, collection elements, and compound operators. The left-hand side can be any postfix chain rooted at a mutable variable.

```
record Inner:
    val: int

record Outer:
    inner: Inner
    tag: str

o = Outer(Inner(1), "t")
o.inner.val = 42              # deep field write
o.inner.val += 1              # compound form
print(o.inner.val)            # 43

pts = [Point(1, 2), Point(3, 4)]
pts[0].x = 99                 # list-of-records field update
pts[0].x *= 2
print(pts[0].x)               # 198
```

See collections → Chained Index and Field Assignment for the full matrix and the aliasing caveat for nested collections inside records.

Usage as Function Parameters and Return Values

```
function distance(p: Point) -> float:
    return (p.x * p.x + p.y * p.y) as float

function make_point(x: int, y: int) -> Point:
    return Point(x, y)
```

Nested Structs

Structs can be used as fields of other structs.

```
record Point:
    x: int
    y: int

record Circle:
    center: Point
    radius: float

c = Circle(Point(0, 0), 1.0)
print(c.center.x)   # 0
```

Comparison (== / !=)

Record types automatically support `==` and `!=` operators. Comparison is performed field-by-field (structural equality).

```
record Point:
    x: int
    y: int

p1 = Point(10, 20)
p2 = Point(10, 20)
p3 = Point(30, 40)

print(p1 == p2)  # true
print(p1 != p3)  # true
```

- All fields are compared in order. For `==`, all fields must be equal. For `!=`, at least one field must differ.
- Nested records are compared recursively.
- If a user-defined `operator==` or `operator!=` is provided, it takes precedence over the auto-generated version.

Record Subtyping (Inheritance)

Records support single inheritance using the `<` syntax. A child record inherits all fields from its parent.

Syntax

```
record ChildName < ParentName:
    child_field: type
```

Example

```
record HttpError < Error:
    status: int
    url: str
```

Field Inheritance

- The child record inherits all parent fields at the beginning of its layout.
- The constructor takes parent fields first, then child-specific fields.

```
err = HttpError("not found", 404, 404, "/api")
print(err.message)  # "not found" (inherited from Error)
print(err.status)   # 404 (own field)
```

Subtype Coercion

A child value can be passed where the parent type is expected. The child is automatically sliced to extract the parent-prefix fields (value-type slicing).

```
function handle(e: Error) -> str:
    return e.message

err = HttpError("fail", 500, 500, "/api")
handle(err)  # OK — HttpError coerced to Error
```

Deep Inheritance

Records can form inheritance chains. Each level inherits all ancestor fields.

```
record DetailedHttpError < HttpError:
    detail: str

# Constructor: Error fields + HttpError fields + own fields
derr = DetailedHttpError("fail", 500, 500, "/x", "server crash")
handle(derr)  # OK — coerced to Error (grandparent)
```

Rules

Rule	Details
Single inheritance only	`record A < B:` — one parent only
Deep inheritance	`record C < B:` where `record B < A:` — allowed
Name collision	Child field with same name as parent field → compile error
Auto `==` / `to_str`	Includes all inherited fields
Invariant inheritance	Parent `invariant:` clauses are checked when constructing or modifying child records
Subtype coercion	Applies to: function args, return, `Err()`, field assignment, `?` operator
Generic bounds	`<T: RecordName>` constrains type parameter to subtypes of the record
`@const`	Applies to all fields including inherited

Constraints and Errors

Constraint	Details
Duplicate field names	Compile error
Field assignment on `@const` variables	Compile error

```
# Error example: Duplicate field names
record Bad:
    x: int
    x: int   # Error
```

Enumerations (enum)

Overview

Enumerations are a set of named constants. By default, they are represented as sequential i64 integers (0, 1, 2, ⋯). Explicit integer values can also be assigned.

Definition Syntax

```
enum TypeName:
    VariantName
    VariantName
    ...
```

Example

```
enum Color:
    Red
    Green
    Blue
```

Variant Access

Variants are accessed using the `::` operator.

```
c = Color::Red
print(c)   # Red
```

Comparison

Since enum values are integers, they can be compared directly with `==` / `!=`.

```
print(Color::Red == Color::Red)    # true
print(Color::Red != Color::Green)  # true
```

Usage in if Statements

```
c = Color::Green
case:
    c == Color::Red:
        print("red")
    c == Color::Green:
        print("green")
    _:
        print("blue")
```

Function Parameters

Use the enum name as the type name.

```
function is_red(c: Color) -> bool:
    return c == Color::Red

print(is_red(Color::Red))    # true
print(is_red(Color::Green))  # false
```

print

`print()` outputs the variant name.

```
c = Color::Blue
print(c)   # Blue
```

Explicit Values

Simple enum variants can be assigned explicit integer values for use cases like HTTP status codes or bitmask patterns.

```
enum HttpStatus:
    Ok = 200
    NotFound = 404
    InternalError = 500

s = HttpStatus::NotFound
print(s)                              # NotFound
print(s == HttpStatus::NotFound)      # true
```

```
enum Permission:
    Read = 1
    Write = 2
    Execute = 4
```

Rules: - Only simple enums (no ADT variants with associated data) support explicit values. - Values must be integer literals (negative values are allowed). - If any variant has an explicit value, all variants must have explicit values (no mixing auto and manual). - Duplicate values are a compile error. - `print()` displays the variant name, not the integer value.

Named Fields in ADT Variants

ADT variant fields can optionally include names for documentation purposes. Named fields make definitions self-describing without changing construction or pattern matching semantics.

```
enum Shape:
    Circle(radius: float)
    Rect(width: float, height: float)
    Point
```

- Construction is always positional: `Shape::Circle(3.14)`, not `Shape::Circle(radius: 3.14)`.
- Pattern matching binds user-chosen variable names: `case Shape::Circle(r):`.
- Field names must be `snake_case`. Mixing named and unnamed fields within a single variant is not allowed.
- Unnamed syntax (`Circle(float)`) remains valid.

Constraints and Errors

Constraint	Details
Variant access requires `EnumName::VariantName`	The `::` operator is required

Constraint	Details
Variant values	Auto-assigned (0, 1, 2, ⋯) by default, or explicitly specified with `= value`
Comparison uses integer comparison	`==`, `!=` can be used
Named field names	Must be `snake_case`; no duplicates within a variant; no mixing named/unnamed

English | 日本語 | 繁體中文

Testing

Ry has a built-in RSpec-style test syntax. Test files are executed using the `ry test` subcommand.

Running Tests

```
ry test               # Auto-discover and run all *.test.ry files in the project
ry test tests/spec    # Run all *.test.ry files under a directory (recursive)
ry test test_file.ry # Run a specific test file
ry test -p            # Run all tests in parallel (-p or --parallel)
ry test -p tests/     # Run tests in a directory in parallel
ry test -w            # Watch mode: re-run tests on file change (-w or --watch)
ry test -w -p         # Watch mode with parallel execution
ry test -w tests/     # Watch a specific directory
ry test --coverage    # Run all tests with line coverage summary
ry test --cov         # Short alias for --coverage
ry test --outline     # Print describe/it structure without running tests
```

The exit code is 0 if all tests passed, 1 if any test failed.

Auto-Discovery Mode

When `ry test` is run without arguments, it:

1. Searches for `package.toml` to find the project root
2. Recursively discovers all `*.test.ry` files under the project root (`.git`, `build`, `node_modules` are skipped)
3. Runs each file and aggregates results

Syntax

Function-Based Syntax (recommended)

Use the `@it` and `@describe` directives to define test cases as ordinary named functions:

```
@it("test case name")
function test_add():
    expect(1 + 2).to_eq(3)
```

Group related tests using `@describe`:

```
@describe("Arithmetic")
function arithmetic_tests():
    @it("should add integers")
    function test_add():
```

```
        expect(1 + 2).to_eq(3)

    @it("should subtract integers")
    function test_sub():
        expect(5 - 3).to_eq(2)
```

- The function name is used for code navigation and symbol identity
- The description string (in the directive) is used for test output and reporting
- `@it` functions must have no parameters unless combined with `@each` or `@property`
- Both `@it` and `@describe` are only available with `ry test`

Shared Setup Variables declared in the `@describe` function body are automatically captured by inner `@it` functions:

```
@describe("User validation")
function user_validation_tests():
    min_length = 8
    max_length = 64

    @it("should reject short passwords")
    function test_short():
        expect(min_length).to_be_greater_than(0)

    @it("should accept passwords within length limits")
    function test_range():
        expect(max_length).to_be_greater_than(min_length)
```

Nested `@describe` `@describe` functions can be nested to create multi-level groupings. Output is indented to reflect the nesting depth:

```
@describe("API")
function api_tests():
    @describe("GET /users")
    function get_users_tests():
        @it("should return 200 OK")
        function test_ok():
            expect(true).to_be_true()
```

Output:

```
API
  GET /users
    + should return 200 OK
```

Lambda Syntax (deprecated)

> **Deprecated**: The `describe()` and `it()` lambda call syntax is deprecated. Use `@describe` and `@it` directives on named functions instead. The lambda syntax will be removed in a future release.
>
> Migration:

Lambda syntax	Directive syntax
`it("name", (): ...)`	`@it("name") function name(): ...`
`describe("name", (): ...)`	`@describe("name") function name(): ...`

```
describe("description", ():
```

```
    it("test case name", ():
        # test body
        expect(actual_value).to_eq(expected_value)
    )
)
```

- `describe` and `it` take a description string and a **lambda argument** `():` as the second parameter
- `it` blocks and other statements (e.g., variable declarations) can be written inside a `describe` block
- Each `it` block is an independent test case
- `describe` / `expect` are only available with `ry test` (compile error with normal `ry` execution)

Trailing Block Syntax

Any function call (except `describe`/`it`/`mock`) can use trailing block syntax. A colon after `()` causes the indented block to be passed as a no-argument lambda in the last argument position:

```
# These are equivalent:
foo("arg"):
    bar()

foo("arg", ():
    bar()
)
```

expect / Matchers

Matcher	Description	Supported Types
`to_eq(expected)`	Equality comparison	int, float, bool, str
`to_not_eq(expected)`	Asserts not equal	int, float, bool, str
`to_be_true()`	Asserts `true`	bool
`to_be_false()`	Asserts `false`	bool
`to_be_none()`	Asserts `None`	Option
`to_be_some()`	Asserts Option is `Some`	Option
`to_be_ok()`	Asserts Result is `Ok`	Result
`to_be_err()`	Asserts Result is `Err`	Result
`to_contain(val)`	Asserts container includes value	List, Set, Map, str
`to_not_contain(val)`	Asserts container does not include value	List, Set, Map, str
`to_be_greater_than(v)`	Asserts `actual > v`	int, float
`to_be_less_than(v)`	Asserts `actual < v`	int, float
`to_be_greater_than_or_eq(v)`	Asserts `actual >= v`	int, float
`to_be_less_than_or_eq(v)`	Asserts `actual <= v`	int, float
`to_have_length(n)`	Asserts length equals `n`	List, Set, Map, str
`to_be_empty()`	Asserts length is 0	List, Set, Map, str
`to_start_with(prefix)`	Asserts string starts with prefix	str
`to_end_with(suffix)`	Asserts string ends with suffix	str

fail

Immediately marks the current test as failed.

```
it("should not reach here", ():
    fail("unexpected error")
)
```

- `fail()` — marks the test as failed with a generic message
- `fail(msg)` — marks the test as failed with a custom message
- Execution continues after `fail()` (does not abort the test)
- Only available in `ry test` mode

Output Format

```
Calculator
  + should add numbers
  + should subtract
  - should fail
    line 10: expected 3, got 2

2 passed, 1 failed
```

- `+` indicates pass (green), `-` indicates failure (red)
- On failure, the line number and expected/actual values are displayed

---

Example

```
describe("Arithmetic", ():
    it("should add integers", ():
        expect(1 + 2).to_eq(3)

    )
    it("should compare strings", ():
        expect("hello").to_eq("hello")

    )
    it("should check booleans", ():
        expect(3 > 1).to_be_true()

    )
)
describe("Booleans", ():
    it("should return false", ():
        expect(1 > 2).to_be_false()
    )
)
```

Mocking

mock(fn_name, replacement)

Replaces a function with a mock implementation for the current `it` block. The mock is automatically cleared when the `it` block ends.

```
function fetch_data() -> str:
    return "real data"

describe("mocking", ():
```

```
    it("should replace function", ():
        mock(fetch_data, () => "fake")
        expect(fetch_data()).to_eq("fake")

    )
    it("should auto-restore after it block", ():
        expect(fetch_data()).to_eq("real data")
    )
)
```

- The first argument is the function name (identifier, not a string)
- The second argument is a replacement lambda
- The replacement must have the same parameter types and return type as the original function
- `require` and `ensure` contracts on the original function are still enforced when the mock is called
- Mocks are automatically restored at the end of each `it` block

verify(fn_name)

Returns the number of times a mocked function was called (as `int`).

```
describe("verify", ():
    it("should count calls", ():
        mock(fetch_data, () => "fake")
        fetch_data()
        fetch_data()
        expect(verify(fetch_data)).to_eq(2)
    )
)
```

Limitations

- Overloaded functions cannot be mocked
- Capture-based closures cannot be used as replacements (use plain lambdas)
- `@native function` functions cannot be mocked

Parameterized Tests (@each)

`@each` runs the same test with multiple sets of parameters.

Function-based syntax (recommended):

```
@each([
    (1, 2, 3),
    (0, 0, 0),
    (-1, 1, 0)
])
@it("should add {0} + {1} = {2}")
function test_add(a: int, b: int, expected: int):
    expect(a + b).to_eq(expected)
```

Lambda syntax:

```
@each([
    (1, 2, 3),
    (0, 0, 0),
    (-1, 1, 0)
```

```
])
it("should add {0} + {1} = {2}", (a: int, b: int, expected: int):
    expect(a + b).to_eq(expected)
)
```

- The list must contain tuples whose arity matches the parameter count
- Placeholders `{0}`, `{1}`, ⋯ in the description are replaced with the parameter values
- Each tuple generates an independent test case
- Supported parameter types: `int`, `float`, `bool`, `str`

Property-Based Tests (@property)

`@property` generates random inputs and runs the test multiple times.

Function-based syntax (recommended):

```
@property(count=100)
@it("should verify addition is commutative")
function test_commutative(a: int, b: int):
    expect(a + b).to_eq(b + a)
```

Lambda syntax:

```
@property(count=100)
it("should verify addition is commutative", (a: int, b: int):
    expect(a + b).to_eq(b + a)
)
```

- `count=N` specifies the number of random trials (default: 100)
- On failure, the counterexample (failing inputs) is printed
- The test stops at the first failure
- Supported parameter types: `int` ([-1000, 1000]), `float` ([-1000.0, 1000.0]), `bool`, `str` (random ASCII, 0-20 chars)

Test Coverage

Run tests with the `--coverage` (or `--cov`) flag to measure line coverage:

```
ry test --coverage                    # All tests with coverage summary
ry test --cov tests/spec/math.test.ry # Single file
ry test --coverage tests/spec/        # Directory
```

Output

```
Test Coverage Summary:
  tests/spec/math.test.ry    100.0%  (74/74 lines)
  tests/spec/strings.test.ry  92.3%  (24/26 lines)
  ------------------------------------------------
  Total                       95.1%  (98/100 lines)
```

- Only user code is reported; standard library files are excluded
- `--coverage` with `--parallel` falls back to sequential execution

Test Outline

Use `--outline` to display the `describe`/`it` structure of test files without executing any test bodies:

```
ry test --outline tests/spec/mock.test.ry
```

Output:

```
describe mock
  it should replace function
  it should auto-restore after it block
  it should mock with arguments
describe verify
  it should count calls
  it should count zero calls
```

- Works with individual files, directories, and `-p` (all test files)
- `@each` parameterized tests show the format template with an `(@each)` suffix
- `@property` tests show the label with a `(@property)` suffix

Test Description Style

`it` descriptions should start with `should` so they read naturally as complete sentences in test output:

```
it should add integers
it should reject invalid input
it should return error when file is missing
```

Preferred:

Description	Notes
`"should add integers"`	verb in base form
`"should reject short passwords"`	verb in base form
`"should return error for missing file"`	verb in base form
`"should add {0} + {1} = {2}"`	parameterized: verb in base form
`"should verify addition is commutative"`	property-based

Avoid:

Description	Reason
`"adds integers"`	third-person verb, reads awkwardly as "it adds"
`"integer addition"`	noun phrase, not a sentence
`"handles error"`	third-person verb

`describe` blocks use noun or topic phrases (e.g., `"Arithmetic"`, `"List"`, `"GET /users"`) — they do not need `should`.

Limitations

- Nesting of `describe` (lambda syntax) is not supported; use the function-based `@describe` directive syntax for nested grouping
- `before_each` / `after_each` are not supported

English

Thread Reference

The `thread` package provides OS-level threading primitives for CPU-bound parallel workloads and fine-grained concurrency control.

Types

Type	Description
`Thread`	Opaque handle for an OS thread
`Lock`	Opaque handle for a mutex (mutual exclusion lock)
`RWLock`	Opaque handle for a read-write lock
`Semaphore`	Opaque handle for a counting semaphore
`Barrier`	Opaque handle for a thread barrier
`AtomicInt`	Opaque handle for an atomic 64-bit integer
`AtomicBool`	Opaque handle for an atomic boolean

All types are opaque pointers managed by ARC (Automatic Reference Counting). Use the corresponding `*_new()` functions to create instances. Resources are automatically cleaned up when no longer referenced — explicit `*_free()` calls are optional but supported for immediate cleanup.

Import

```
from thread import thread_spawn, thread_join, lock_new, lock_acquire, lock_release
```

Thread

Function	Signature	Description
`thread_spawn`	`(body: function() -> T) -> Thread`	Creates and starts a new OS thread executing `body`. Captured variables are copied by value. `T` may be `Unit`, `int`, `float`, or `bool`; see the limitations section below.
`thread_join`	`(thread: Thread) -> Result<T, Error>`	Waits for the thread to finish and returns the worker' s value in `Ok(value)`. Joining an already-joined `Thread` returns `Err("thread already joined")`.

Example — side-effect worker (`Unit`)

```
from thread import thread_spawn, thread_join, atomic_int_new, atomic_int_load, atomic_int_add

counter = atomic_int_new(0)
t = thread_spawn():
  atomic_int_add(counter, 1)
)
thread_join(t)
print(atomic_int_load(counter))  # 1
```

Example — value-returning worker (`int` / `float` / `bool`)

```
from thread import thread_spawn, thread_join

t = thread_spawn(() => 42)
case thread_join(t):
  Ok(v):
    print(v)       # 42
  Err(e):
    print(e.message)

# Captures work with value-returning workers too:
x = 10
t2 = thread_spawn(() => x * x)
case thread_join(t2):
  Ok(v):
    print(v)       # 100
  Err(_):
    print("error")
```

> **Note:** Captured variables are copied into the thread's environment. For primitive types (int, float, bool, str), this produces an independent copy. For opaque handle types (Lock, AtomicInt, etc.), the pointer is copied, sharing the underlying resource — which is the intended behavior for synchronization primitives.

Limitations (MVP)

- **Return type.** Workers may return `Unit`, `int`, `float`, or `bool`. ARC-managed types (`str`, `List`, `Map`, `Set`, records) and sum types (`Option`, `Result`, enums) are not yet supported; passing such a worker produces a codegen error pointing at a follow-up issue.
- **Lambda body shape.** Only expression-bodied lambdas (`() => <expr>`) may return a value. Block-bodied lambdas can still be used for `Unit` workers but cannot carry a non-`Unit` return value yet.
- **Variable-reference workers.** `thread_spawn(my_fn)` still treats `my_fn` as a `Unit` worker; reading its return value requires the inline-lambda form for now.
- **Panics.** A runtime panic inside the worker (e.g. division by zero, array out-of-bounds, contract violation) terminates the entire process. It does not currently surface as `Err` from `thread_join` — this requires a separate refactor of Ry's panic mechanism and is tracked as a follow-up issue.

Lock (Mutex)

Function	Signature	Description
`lock_new`	`() -> Lock`	Creates a new mutex.
`lock_acquire`	`(lock: Lock) -> Result<Unit, Error>`	Acquires the lock. Blocks until available.
`lock_release`	`(lock: Lock) -> Result<Unit, Error>`	Releases the lock.
`lock_free`	`(lock: Lock) -> Unit`	Immediately frees the lock. Optional — ARC handles cleanup automatically.

RWLock (Read-Write Lock)

Function	Signature	Description
`rwlock_new`	`() -> RWLock`	Creates a new read-write lock.
`rwlock_read_lock`	`(rwlock: RWLock) -> Result<Unit, Error>`	Acquires a shared read lock. Multiple readers allowed.
`rwlock_write_lock`	`(rwlock: RWLock) -> Result<Unit, Error>`	Acquires an exclusive write lock. Blocks until all readers and writers release.
`rwlock_unlock`	`(rwlock: RWLock) -> Result<Unit, Error>`	Releases the lock (shared or exclusive).
`rwlock_free`	`(rwlock: RWLock) -> Unit`	Immediately frees the read-write lock. Optional — ARC handles cleanup automatically.

Semaphore

Function	Signature	Description
`semaphore_new`	`(count: int) -> Result<Semaphore, Error>`	Creates a semaphore with the given initial count. Returns `Err` if count is negative.
`semaphore_acquire`	`(sem: Semaphore) -> Result<Unit, Error>`	Decrements the semaphore. Blocks if count is zero.
`semaphore_release`	`(sem: Semaphore) -> Result<Unit, Error>`	Increments the semaphore and wakes a waiting thread.
`semaphore_free`	`(sem: Semaphore) -> Unit`	Immediately frees the semaphore. Optional — ARC handles cleanup automatically.

Barrier

Function	Signature	Description
`barrier_new`	`(count: int) -> Result<Barrier, Error>`	Creates a barrier that synchronizes `count` threads. Returns `Err` if count is not positive.
`barrier_wait`	`(barrier: Barrier) -> Result<Unit, Error>`	Blocks until all `count` threads have called `barrier_wait`.
`barrier_free`	`(barrier: Barrier) -> Unit`	Immediately frees the barrier. Optional — ARC handles cleanup automatically.

AtomicInt

Function	Signature	Description
`atomic_int_new`	`(value: int) -> AtomicInt`	Creates an atomic integer with the given initial value.
`atomic_int_load`	`(atomic: AtomicInt) -> int`	Atomically reads the value.
`atomic_int_store`	`(atomic: AtomicInt, value: int) -> Unit`	Atomically writes the value.
`atomic_int_add`	`(atomic: AtomicInt, delta: int) -> int`	Atomically adds `delta` and returns the **previous** value.

Function	Signature	Description
`atomic_int_sub`	`(atomic: AtomicInt, delta: int) -> int`	Atomically subtracts `delta` and returns the **previous** value.
`atomic_int_cas`	`(atomic: AtomicInt, expected: int, desired: int) -> bool`	Compare-and-swap: if current value equals `expected`, sets to `desired` and returns `true`; otherwise returns `false`.
`atomic_int_free`	`(atomic: AtomicInt) -> Unit`	Immediately frees the atomic integer. Optional — ARC handles cleanup automatically.

AtomicBool

Function	Signature	Description
`atomic_bool_new`	`(value: bool) -> AtomicBool`	Creates an atomic boolean with the given initial value.
`atomic_bool_load`	`(atomic: AtomicBool) -> bool`	Atomically reads the value.
`atomic_bool_store`	`(atomic: AtomicBool, value: bool) -> Unit`	Atomically writes the value.
`atomic_bool_free`	`(atomic: AtomicBool) -> Unit`	Immediately frees the atomic boolean. Optional — ARC handles cleanup automatically.

Comparison with async/await

Feature	`async`/`await`	`thread` package
Execution model	Work-stealing thread pool	Dedicated OS threads
Best for	I/O-bound tasks, many lightweight tasks	CPU-bound tasks, fine-grained control
Synchronization	Automatic via `await`	Manual via Lock, Semaphore, etc.
Overhead	Low (task scheduling)	Higher (OS thread creation)

English | 日本語 | 繁體中文

Type Reference

Type List

Type	Internal Representation	Literal Examples	Description
`int`	i64	`42`, `-7`, `0xFF`, `0b1010`, `100_000`	64-bit signed integer
`u8`	i8	(no dedicated literal)	Unsigned 8-bit integer (0-255). Used with type annotation `b: u8 = 42`
`float`	f64	`3.14`, `0.5`, `3.14_159`, `1e10`, `1.5e-3`, `2.5E+2`	64-bit floating-point number (scientific notation supported)
`bool`	i1	`true`, `false`	Boolean value

Type	Internal Representation	Literal Examples	Description
`str`	ptr	`"hello"`, `""`, `"a\nb"`	String (immutable byte sequence on the heap)
`Unit`	void	(no return value)	Return type for functions with no return value. Must be specified explicitly with `-> Unit`
`Option<T>`	`{ i1, T }`	`Some(42)`, `None`	A type that may or may not contain a value
`(T1, T2, ...)`	LLVM StructType (literal)	`(1, 3.14)`	Tuple type
`List<T>`	ptr (heap)	`[1, 2, 3]`	Dynamic array
`Map<K, V>`	ptr (heap)	`{"a": 1}`	Hash map
`Set<T>`	ptr (heap)	`{1, 2, 3}`	Set with no duplicates
`function(T1, T2) -> R`	ptr (function pointer)	`(x: int) => x * 2`	Function type
User-defined type	LLVM StructType (named)	`record Point: ...`	Struct defined with the `record` keyword
`enum`	i64 / tagged union	`Color::Red`, `Shape::Circle(3.14)`	Enumeration defined with the `enum` keyword (supports associated data)
`Error`	`{ ptr, i64 }`	`Error("msg")`, `Error("msg", 404)`	Built-in error type
`Type`	`{ i64, ptr }`	`type_of(42)`	Compile-time type identity returned by `type_of`. See Type
`any`	`{ i64, [8 x i8] }`	`x: any = 42`	Tagged union that can hold any primitive value
`T1 | T2`	`{ i64, [N x i8] }`	`int | str`	Union type (holds one of multiple types)
Int literal	i64	`42`, `0 | 1`	Int literal type (value constraint)
String literal	ptr	`"N" | "S"`	String literal type (value constraint)
Range	i64	`1..12`, `-10..10`	Range type (inclusive integer range constraint)
`i8`	i8	`x: i8 = 42`, `x = 42i8`	8-bit signed integer (low-level, no implicit conversion)
`i16`	i16	`x: i16 = 100`, `x = 100i16`	16-bit signed integer (low-level, no implicit conversion)
`i32`	i32	`x: i32 = 42`, `x = 42i32`	32-bit signed integer (low-level, no implicit conversion)
`i64`	i64	`x: i64 = 100`, `x = 100i64`	64-bit signed integer (low-level, no implicit conversion)
`u8`	i8	`x: u8 = 200`, `x = 200u8`	8-bit unsigned integer (low-level, no implicit conversion)

Type	Internal Representation	Literal Examples	Description
`u16`	i16	`x: u16 = 60000`, `x = 60000u16`	16-bit unsigned integer (low-level, no implicit conversion)
`u32`	i32	`x: u32 = 4294967295`, `x = 100u32`	32-bit unsigned integer (low-level, no implicit conversion)
`u64`	i64	`x: u64 = 18446744073709551615`, `x = 0xFFFFFFFFFFFFFFFFu64`	64-bit unsigned integer up to 2^64 − 1 (low-level, no implicit conversion)
`f32`	float	`x: f32 = 3.14`, `x = 1e10f32`	32-bit floating-point (low-level, no implicit conversion)
`weak T`	ptr (header)	`weak s`	Weak reference to an ARC-managed value (does not prevent deallocation)
`Regex`	ptr	`/[a-z]+/`, `/\d{3}/`	Regular expression pattern (created via regex literal syntax)
`Result<T, E>`	`{ i1, T/E }`	`Ok(42)`, `Err(Error("fail"))`	A type representing success (`Ok`) or failure (`Err`)
`Task<T>`	ptr	(returned by async functions)	Asynchronous task handle (used with `await` and `block_on`)
`Iterator<T>`	ptr	(created by `iter()`)	Lazy iterator for sequential element access
`T[N]`	`[N x T]`	`buf: i32[8]`	Fixed-length contiguous array of low-level type T with N elements (stack-allocated)

Type Annotation Syntax

You can explicitly specify the type when declaring a variable. The annotation can be omitted when the type is inferrable.

```
x: int = 42
b: u8 = 255
f: float = 3.14
s: str = "hello"
b: bool = true
opt: Option<int> = Some(10)
t: (int, float) = (1, 3.14)
xs: List<int> = [1, 2, 3]
m: Map<str, int> = {"a": 1}
s: Set<int> = {1, 2, 3}
fn_val: function(int) -> int = (x: int) => x * 2
rx: Regex = /[0-9]+/
u: int | str = 42
a: any = 42
```

Available Type Names

Type Name	Notes
`int`	Built-in scalar type
`u8`	Built-in scalar type (unsigned 0-255)
`float`	Built-in scalar type
`bool`	Built-in scalar type
`str`	Built-in string type
`Unit`	Return type of functions that return no value
`Option<T>`	Generic type (T is any type)
`(T1, T2, ...)`	Tuple type (arbitrary number and combination of element types)
`List<T>`	Generic dynamic array type
`Map<K, V>`	Generic hash map type
`Set<T>`	Generic set type
`function(T1, ...) -> R`	Function type
`Error`	Built-in error type (`message: str`, `code: int`)
`any`	Built-in type that can hold any primitive value (`int`, `float`, `bool`, `str`) or `Unit`. Supports implicit conversion: concrete values are automatically wrapped when assigned to `any`, and `any` values are automatically unwrapped (with runtime type check) when assigned to a concrete type. `any(int)` → `float` auto-promotion is supported. See any Type for details
`T1 | T2 | ...`	Union type (one of multiple types separated by `|`)
`i8`	Low-level 8-bit signed integer (no implicit conversion)
`i16`	Low-level 16-bit signed integer (no implicit conversion)
`i32`	Low-level 32-bit signed integer (no implicit conversion)
`i64`	Low-level 64-bit signed integer (no implicit conversion)
`u8`	Low-level 8-bit unsigned integer (no implicit conversion)
`u16`	Low-level 16-bit unsigned integer (no implicit conversion)
`u32`	Low-level 32-bit unsigned integer (no implicit conversion)
`u64`	Low-level 64-bit unsigned integer (no implicit conversion)
`f32`	Low-level 32-bit floating-point (no implicit conversion)
`T[N]`	Fixed-length array of low-level type `T` with `N` elements. Stack-allocated, contiguous memory. Supports index read/write and `length()`
User-defined type name	Type declared with the `record` or `enum` keyword

Type Aliases

The `type` keyword creates a new name for an existing type. The alias is fully interchangeable with the original type.

```
type Meters = float
type StringList = List<str>

d: Meters = 3.14
names: StringList = ["Alice", "Bob"]
```

> **Naming convention**: Type alias names must use PascalCase (e.g., `Meters`, `StringList`). The compiler enforces this convention.

Type aliases also work with function types, literal types, and range types:

```
type Callback = function(int, int) -> int

add: Callback = function(a: int, b: int) => a + b
print(add(3, 4))    # 7
```

```
type Month = 1..12
type Direction = "N" | "S" | "E" | "W"
type Digit = 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

m: Month = 6
d: Direction = "N"
n: Digit = 5
```

Type aliases can also target union types (including primitive and user-defined types), and the alias behaves identically to the inlined union:

```
type Simple = int | str | bool

x: Simple = 42
y: Simple = "hello"
z: Simple = true

function describe(v: Simple) -> str:
  return to_str(v)
```

Nested aliases whose union components are themselves aliases are flattened transparently, and duplicate members are deduplicated. The following three forms are equivalent:

```
type A = int | str
type B = A | bool          # same as `int | str | bool`
type C = B | int           # same as `int | str | bool` (int is deduplicated)

x: B = 42
y: B = "hello"
z: B = true
```

Numeric Literals

Integer Literals

Decimal, hexadecimal (`0x`/`0X`), and binary (`0b`/`0B`) forms are accepted. Underscores are allowed between digits as a visual separator (`1_000_000`, `0xFFFF_FFFF`).

The accepted magnitude is determined by the target type:

Target	Range
bare `int` / `i64`	`-9_223_372_036_854_775_808 .. 9_223_372_036_854_775_807` (i64)
`i8` / `i16` / `i32`	corresponding signed range
`u8` / `u16` / `u32`	`0 .. 2^N - 1`
`u64`	`0 .. 18_446_744_073_709_551_615` (2^64 − 1)

Large unsigned literals require either a suffix (`18446744073709551615u64`) or a type annotation on the receiving variable (`x: u64 = 18446744073709551615`). Negative literals arrive as a unary minus on a non-negative magnitude, so `-1i8` is accepted while `-1u8` is rejected.

```
max_u64: u64 = 18446744073709551615     # 2^64 - 1
mask:    u64 = 0xFFFF_FFFF_FFFF_FFFF    # same value via hex
word:    u32 = 4294967295               # 2^32 - 1
```

Float Literals

```
FloatLiteral := DecDigits '.' DecDigits Exponent? FloatSuffix?
             |  DecDigits Exponent FloatSuffix?
Exponent     := ('e' | 'E') ('+' | '-')? DecDigits
FloatSuffix  := 'f32' | 'f64'
```

Scientific notation is supported anywhere a float is expected:

```
avogadro  = 6.022e23
planck    = 6.626e-34
light_spd = 2.998E8
big       = 1e10f32
```

Overflowing exponents produce `+Inf`/`-Inf` (not a compile error). Note that the runtime `to_float()` converter is stricter: it returns `Err(Error)` on overflow rather than producing `+Inf`.

Literal Types

A literal type restricts a variable to specific constant values. The compiler checks these constraints at compile time for constant values, and emits runtime checks for dynamic values.

Int Literal Type

```
x: 42 = 42          # single literal type
y: 0 | 1 = 0        # union of int literals
z: 0 | 1 = 0
z = 1                   # OK
# z = 2                 # compile error (constant) or runtime error (dynamic)
```

String Literal Type

```
dir: "N" | "S" | "E" | "W" = "N"
# @const bad: "N" | "S" = "X"    # compile error
```

Constraint Checking

- **Compile time**: If the assigned value is a constant (`ConstantInt` or string literal), the constraint is checked at compile time and a compile error is raised on violation.

- **Runtime**: If the value is dynamic (e.g., from a function call), the constraint is checked at runtime and the program exits with an error on violation.

Range Type

A range type constrains an integer variable to a contiguous range of values (inclusive on both ends).

```
month: 1..12 = 6        # OK
# @const bad: 1..12 = 0       # compile error: out of range
# @const bad: 1..12 = 13      # compile error: out of range

t: -10..10 = -5         # negative ranges are supported
```

With Mutable Variables (Runtime Check)

```
x: 1..12 = 6
x = 12                      # OK
# x = dynamic_value()       # runtime check: exits if out of range
```

In Function Parameters

```
function set_month(m: 1..12) -> int:
    return m

set_month(6)                # OK
# set_month(13)             # compile error (constant argument)
```

`none` Keyword and Option Type Shorthand

The `none` keyword represents the absence of a value for Option types, equivalent to `None`.

The `T?` syntax is a shorthand for `Option<T>`.

```
x: int? = 42        # equivalent to Option<int>
y: int? = none       # equivalent to None

function find(xs: List<int>, val: int) -> int?:
    for x in xs:
        if x == val:
            return Some(x)
    return none
```

Weak References (`weak T`)

A `weak` reference is a non-owning reference to an ARC-managed value. Unlike strong references, weak references do not increment the strong reference count. When the last strong reference is released, the referenced object is deallocated — and any surviving weak references automatically become `None`.

Weak references are the user-facing mechanism for breaking reference cycles.

Creating a Weak Reference

Use the `weak` keyword in both type annotation and expression position:

```
s = "hello"
w: weak str = weak s
```

The type `weak T` is a new type constructor where `T` must be an ARC-managed type (currently `str`, `List<T>`, `Map<K, V>`, `Set<T>`).

Accessing a Weak Reference (Upgrade)

Accessing a weak variable automatically performs an **upgrade** —an atomic check-and-increment of the strong reference count. The result is always `Option<T>`:

- `Some(value)` if the referent is still alive (strong count > 0)
- `None` if the referent has been deallocated (strong count $== 0$)

```
s = "alive"
w: weak str = weak s
case w:
  Some(v):
    print(v)           # "alive"
  None:
    print("deallocated")
```

The coalesce operator (`??`) also works with weak references:

```
w: weak str = weak s
val = w ?? "default"
```

Reassignment

Weak references can be reassigned. The old weak reference is released and the new one is retained:

```
a = "first"
b = "second"
w: weak str = weak a
w = weak b
```

Thread Safety

The upgrade operation uses a compare-and-swap (CAS) loop internally, making it safe to use across threads. This is essential since the strong reference may be released concurrently.

Scope Cleanup

Weak references are automatically released when they go out of scope. If both strong and weak reference counts reach zero, the ARC header is freed.

F-String (String Interpolation)

String interpolation with the `f"..."` syntax. Expressions inside `{}` are evaluated and converted to strings.

```
name = "world"
print(f"Hello {name}")      # Hello world

a = 1
```

```
b = 2
print(f"{a} + {b} = {a + b}")    # 1 + 2 = 3
```

Supported Types in Interpolation

Any expression that evaluates to `int`, `float`, `bool`, `str`, a record type, a tuple, or a collection type (`List`, `Map`, `Set`) can be used inside `{}`.

```
xs = [1, 2, 3]
print(f"items: {xs}")      # items: [1, 2, 3]

t = (1, "hello")
print(f"tuple: {t}")       # tuple: (1, hello)
```

Escape Sequences

Sequence	Output
`{{`	`{` (literal brace)
`}}`	`}` (literal brace)
`\n \r \t \\ \"`	Same as regular strings

```
print(f"{{braces}}")    # {braces}
```

Type Casting (`as`)

Explicit type conversion using the `as` keyword.

```
x = 42 as float      # 42.0
y = 3.14 as int       # 3
z = 1 as bool         # true
s = 42 as str          # "42"
b = 255 as u8          # u8 value 255
```

Supported Casts

From	To	Behavior
`int`	`float`	`SIToFP`
`float`	`int`	Truncation (`FPToSI`)
`int`	`bool`	`0` -> `false`, non-zero -> `true`
`bool`	`int`	`false` -> `0`, `true` -> `1`
`int` / `float` / `bool`	`str`	String representation
`int`	`u8`	Truncation (lower 8 bits)
`u8`	`int`	Zero extension

`int` | `i8` / `i16` / `i32` / `i64` | Truncation (or identity for i64) |
`i8` / `i16` / `i32` / `i64` | `int` | Sign extension (`SExt`) |
`int` | `u8` / `u16` / `u32` / `u64` | Truncation (or identity for u64) |
`u8` / `u16` / `u32` / `u64` | `int` | Zero extension (`ZExt`) |
signed | signed (wider) | Sign extension (`SExt`) |
signed | signed (narrower) | Truncation |
unsigned | unsigned/signed (wider) | Zero extension (`ZExt`) |
unsigned | unsigned/signed (narrower) | Truncation |
signed / unsigned int | `float` | `SIToFP` / `UIToFP` then `f64` |

`float` | signed / unsigned int | `FPToSI` / `FPToUI` |
`float` | `f32` | `FPTrunc` |
`f32` | `float` | `FPExt` |
signed int | `f32` | `SIToFP` |
unsigned int | `f32` | `UIToFP` |
`f32` | signed / unsigned int | `FPToSI` / `FPToUI` |

The target type of `as` supports the full type syntax, including generic types:

```
x = value as Option<int>
y = data as Map<str, int>
```

Any `as` cast (including with generics) must be a built-in cast or have a matching user-defined `operator as`, otherwise it is a compile error. Use `to_int()` / `to_float()` for string-to-number conversions.

Enum with Associated Data (ADT)

Enum variants can carry associated data by specifying types in parentheses after the variant name. Variants without parentheses remain simple tags.

```
enum Shape:
    Circle(float)
    Rectangle(float, float)
    Point
```

Named Fields

Variants can optionally use named fields for documentation clarity. Named fields make variant definitions self-describing but do not change runtime behavior — construction and pattern matching remain positional.

```
enum Shape:
    Circle(radius: float)
    Rectangle(width: float, height: float)
    Point
```

Rules: - Field names must be `snake_case`. - Within a single variant, all fields must be either named or unnamed (no mixing). - Duplicate field names within a variant are a compile error.

Constructor

Use the `EnumName::Variant(value)` syntax to construct a variant with data. Arguments are always positional, even when fields are named.

```
c = Shape::Circle(3.14)
r = Shape::Rectangle(4.0, 5.0)
p = Shape::Point
```

Pattern Matching with Binding

Use `case EnumName::Variant(binding):` to extract the associated data. Bindings use user-chosen variable names, not field names.

```
case c:
    Shape::Circle(r):
        print(r)            # 3.14
    Shape::Rectangle(w, h):
        print(w)
        print(h)
```

```
    Shape::Point:
        print("point")
```

Internal Representation

An ADT enum is stored as a tagged union: `{ i64 tag, [N x i8] data }` where `N` is sized to fit the largest variant' s payload.

Generic Enum

An enum can have type parameters using angle-bracket syntax `<T>`. This allows the same enum shape to hold different payload types.

```
enum MyOption<T>:
    MySome(T)
    MyNone
```

Usage

Instantiate by providing a concrete type argument. The type argument is required when the compiler cannot infer it.

```
a = MyOption<int>::MySome(42)
b = MyOption<int>::MyNone

case a:
    MyOption::MySome(v):
        print(v)      # 42
    MyOption::MyNone:
        print("none")
```

Error Type

A built-in type for error handling. `Error` has two fields: `message` (str) and `code` (int).

```
e = Error("something went wrong")       # code defaults to 0
e2 = Error("not found", 404)            # explicit code

print(e.message)    # something went wrong
print(e2.code)      # 404
print(e2)           # Error: not found (code: 404)
```

Error Handling with Result

Functions that can fail return `Result<V, E>`:

```
function divide(a: int, b: int) -> Result<int, Error>:
    if b == 0:
        return Err(Error("division by zero"))
    return Ok(a // b)

case divide(10, 2):
    Ok(v):
        print(v)            # 5
```

```
    Err(e):
        print(e.message)
```

When the return value is not meaningful, use `Result<Unit, Error>`:

```
function save(path: str, data: str) -> Result<Unit, Error>:
    return Ok(0 as u8)   # Unit placeholder

case save("/tmp/test.txt", "hello"):
    Ok(_):
        print("saved")
    Err(e):
        print(e.message)
```

Result Type

`Result<V, E>` is a built-in parameterized type with two constructors:

- `Ok(value)` — success variant
- `Err(error)` — error variant

It is used with `case` for exhaustive error handling. Both `Ok` and `Err` cases must be covered (or use `_` wildcard).

Equality: `Result<T, E>` supports `==` and `!=`. Two results are equal when both variants match (`Ok`/`Ok` or `Err`/`Err`) and the inner values are equal.

```
function make_ok(v: int) -> Result<int, Error>: return Ok(v)
make_ok(42) == make_ok(42)   # true
make_ok(1)  == make_ok(2)    # false
make_ok(1)  != Err(Error("e"))  # true
```

Test matchers: - `expect(x).to_be_ok()` — asserts the result is `Ok` - `expect(x).to_be_err()` — asserts the result is `Err`

Internal Representation

`Error` is represented as `{ ptr message, i64 code }`. `Result<V, E>` is represented as `{ i1 isOk, V okValue, E errValue }`.

Type

`Type` is the value returned by the built-in `type_of` function. It represents the compile-time identity of a type and allows reflective comparison at run time.

```
print(to_str(type_of(42)))          # int
print(to_str(type_of([1, 2, 3])))   # List

print(type_of(42) == type_of(100))  # true
print(type_of(42) == type_of(3.14)) # false
```

Key properties:

- Each distinct type definition (primitive, collection, record, enum, `Option`, `Result`, `function`, `Type` itself, etc.) receives a unique identity at compile time.
- `==` / `!=` on `Type` values compare identities, not display names. Two different records (or a record and an enum with the same name) are always distinguishable.
- `print` and `to_str` display the human-readable type name (for example, `"int"`, `"List"`, `"Point"`, `"i32"`).
- Low-level numeric types (`i8`, `i16`, ⋯, `f32`) are distinguished from `int` / `float`.
- Collection generics collapse to their base name: `type_of([1, 2])` returns `"List"`, not `"List<int>"`.

- `Type` is reflective: `type_of(type_of(x))` returns the `Type` value that represents `Type` itself.

Internal Representation

`Type` is represented as `{ i64 id, ptr name }`. The `id` field is used for equality and the `name` field is used for display. Both fields are populated at compile time by `type_of`.

Union Type

You can declare a variable that may hold one of multiple types using `|`.

```
x: int | str = 42
x = "hello"     # Reassignment is allowed (any type in the union)
print(x)        # hello
```

Usage in Function Parameters and Return Types

```
function show(x: int | str) -> int:
    print(x)
    return 0

function get_val(flag: bool) -> int | str:
    if flag:
        return 42
    return "hello"
```

Internal Representation

A union type is represented as `{ i64 tag, [N x i8] data }`. The `tag` indicates the index of each component type (sorted alphabetically), and `data` is a byte array sized to the largest component type.

Equality

Union types currently support `==` and `!=` for primitive variants (`int`, `float`, `str`, `bool`). Two union values are equal when they hold the same variant (same tag) and the inner values are equal.

```
x: int | str = 42
y: int | str = 42
x == y   # true

z: int | str = "42"
x == z   # false (different tags: int vs str)
```

Constraints

- Assigning a type not included in the union causes a compile error
- `int | str` and `str | int` are the same type (normalized)
- When printing a union value with `print()`, the value is displayed using the appropriate type based on the runtime tag
- `==` and `!=` support primitive variants (`int`, `float`, `str`, `bool`); closure variants are not supported

any Type

The `any` type is a built-in dynamic type that can hold any primitive value. It follows Python's approach of allowing flexible typing — when you don't need static type guarantees, `any` lets you write code that works with multiple types without explicit generics or union types.

Supported Types

`any` can hold the following types:

Type	Tag	Description
`int`	0	64-bit signed integer
`float`	1	64-bit floating-point number
`bool`	2	Boolean value
`str`	3	String
`Unit`	4	Unit value (for functions with no return value)

`any` **cannot** hold collection types (`List`, `Map`, `Set`), resource types (`TcpListener`, `TcpStream`, etc.), function pointers, or user-defined types (`record`, `enum`).

Internal Representation

`any` is implemented as a tagged union:

```
{ i64 tag, [8 x i8] data }   // 16 bytes total
```

The `tag` field identifies the stored type, and the `data` field holds the value (up to 8 bytes).

Wrapping and Unwrapping

Concrete values are automatically **wrapped** when assigned to `any`, and `any` values are automatically **unwrapped** when assigned to a concrete type.

```
# Wrapping: concrete → any
x: any = 42          # int is wrapped into any
x = "hello"          # reassignment with a different type is allowed

# Unwrapping: any → concrete
function get_value() -> any:
    return 42
n: int = get_value()  # any(int) is unwrapped to int

# int → float auto-promotion during unwrap
f: float = get_value()  # any(int) is unwrapped and promoted to float
```

If the runtime type does not match the target type (e.g., unwrapping `any(str)` into an `int` variable), a **runtime error** occurs.

Reassignment

An `any` variable can be reassigned to a value of any supported type:

```
x: any = 42
x = 3.14        # OK: now holds float
x = "hello"     # OK: now holds str
x = true        # OK: now holds bool
```

Arithmetic Operations

When both operands are `any`, arithmetic operations dispatch at runtime based on the actual types:

Operation	Types	Result
`+`	int + int	int
`+`	float + float	float
`+`	int + float	float
`+`	str + str	str (concatenation)
`-`	numeric	int or float
`*`	numeric	int or float
`*`	str * int / int * str	str (repetition)
`/`	numeric	float (always)
`//`	int // int	int
`//`	with float	float
`%`	numeric	int or float
`**`	numeric	float (always)
unary `-`	int	int
unary `-`	float	float

When one operand is `any` and the other is a concrete type, the concrete value is automatically wrapped before the operation.

```
x: any = 10
y: any = x + 20    # 20 is auto-wrapped; result is any(int) = 30
```

Incompatible type combinations (e.g., `str - int`) cause a **runtime error**.

Comparison Operations

Operation	Behavior
`==`, `!=`	Works for same types; int/float mixing is allowed
`<`, `<=`, `>`, `>=`	Numeric (int/float mixing allowed) and string (lexicographic)

```
x: any = 3
y: any = 3.0
print(x == y)    # true (int/float comparison)
```

Type mismatches in comparison (e.g., `int < str`) cause a **runtime error**.

String Conversion

`any` values support `print()` and f-string interpolation:

```
x: any = 42
print(x)              # 42
print(f"value: {x}")  # value: 42
```

Conversion rules: `int` → decimal string, `float` →`%g` format, `bool` → `"true"`/`"false"`, `str` → as-is, `Unit` → `"Unit"`.

Passing any to Typed Functions

An `any` value can be passed to a function with concrete parameter types. The value is automatically unwrapped with a runtime type check:

```
function add_one(x: int) -> int:
    return x + 1
```

```
v: any = 42
result = add_one(v)   # any(int) is unwrapped to int; result is 43
```

Type Rules (Type Conversion in Operations)

Operation	Left	Right	Result Type	Notes
`+ - *`	int	int	int	
`+ - *`	u8	u8	u8	Low-level type: native-width unsigned operations, no implicit promotion
`+ - *`	float or int	float or int (one is float)	float	Implicit float promotion
`/`	any numeric	any numeric	float	Always float
`//`	any numeric	any numeric	int or float	Floor division (toward -∞); int for int operands, float if either is float
`**`	any numeric	any numeric	float	Uses libm `pow`
`%`	int	int	int	
`%`	float or int	float or int (one is float)	float	
`+`	str	str	str	String concatenation
`== != < <= > >=`	str	str	bool	Lexicographic comparison
`== != < <= > >=`	numeric or bool	numeric or bool	bool	
`in`	any	Set	bool	Whether the element is in the set
`& | ^ ~ << >>`	int	int	int	Error for float
`+ - *`	i32	i32	i32	Low-level types: no implicit conversion, same type required
`/ //`	i32	i32	i32	Signed integer division (`SDiv`)
`/ //`	u32	u32	u32	Unsigned integer division (`UDiv`)
`%`	i32	i32	i32	Signed remainder (`SRem`)
`%`	u32	u32	u32	Unsigned remainder (`URem`)
`+ - * /`	f32	f32	f32	
`== !=`	i32/u32	i32/u32	bool	Sign-agnostic equality
`< <= > >=`	i32	i32	bool	Signed comparison (`ICMP_SLT` etc.)
`< <= > >=`	u32	u32	bool	Unsigned comparison (`ICMP_ULT` etc.)

Operation	Left	Right	Result Type	Notes
`>>`	i32	i32	i32	Arithmetic right shift (sign-preserving)
`>>`	u32	u32	u32	Logical right shift (zero-fill)
`**`	low-level	any	error	Power operator not supported for low-level types
mixed	low-level	different	error	Mixing low-level and high-level types is a compile error

Escape Sequences (in str Literals)

Sequence	Meaning
`\n`	Newline
`\r`	Carriage return
`\t`	Tab
`\\`	Backslash
`\"`	Double quote
`\0`	Null character

Type Safety Constraints

- **Implicit widening conversions** – Safe widening conversions are supported in function calls: `u8` → `int`, `u8` → `float`, `int` → `float`. For binary operators, mixing `int` and `float` triggers float promotion. `u8` is a low-level type that operates at native width with unsigned semantics; mixing `u8` with `int` in binary operators is a compile error. Narrowing conversions (e.g., `float` → `int`) are not allowed implicitly. Narrowing conversion from an `int` literal to `u8` is only allowed with a type annotation `b: u8 = 42`.
- **Variable types are fixed at declaration** – A variable declared as `int` cannot be reassigned a `float` value.
- **Bitwise operations are for `int` only** – Applying bitwise operations to `float` or `bool` causes a compile error.
- **Non-`bool` types can be used in conditions** – `if` conditions accept `int` (0 = false, non-zero = true) and other types besides `bool`.
- **Numeric literal separators** – Underscores can be used as visual separators in numeric literals: `100_000`, `0xFF_FF`, `0b1010_0101`, `3.14_159`. Underscores must appear between digits (no leading, trailing, or consecutive underscores).
- **Numeric literal suffixes** – Low-level types can be specified via literal suffixes: `42i32`, `255u8`, `3.14f32`, `.5f32`, `0xFFu8`, `0b1010u8`. An integer literal with a float suffix (`42f32`) produces a float value. A float literal with an integer suffix (`3.14i32`) is a compile error. Out-of-range values (e.g., `256u8`, `129i8`) are also compile errors.
- **Low-level numeric types (`i8`, `i16`, `i32`, `i64`, `u8`, `u16`, `u32`, `u64`, `f32`) have no implicit conversions** – Mixing low-level types with each other or with high-level types (`int`, `float`) causes a compile error. Use explicit `as` casts. The `/` operator on low-level integers performs integer division (like Rust), not float division. Signed types use `SDiv`/`SRem`, unsigned types use `UDiv`/`URem`.
- **Signed vs unsigned** – Signed types (`i8`, `i16`, `i32`, `i64`) use signed comparison (`ICMP_SLT` etc.) and arithmetic right shift (`AShr`). Unsigned types (`u8`, `u16`, `u32`, `u64`) use unsigned comparison (`ICMP_ULT` etc.) and logical right shift (`LShr`). The `>>>` operator always performs logical shift regardless of signedness.

- **`int` arithmetic overflow is a runtime error** – Arithmetic (`+`, `-`, `*`, unary `-`) on the high-level `int` type raises a runtime error on overflow, similar to Swift's default behavior. This prevents silent data corruption from two's complement wrapping. Constant expressions that overflow are caught at compile time.
- **Low-level integer overflow wraps around** – Arithmetic on low-level integer types uses Ry-defined two's complement wrapping on overflow for signed types and modular arithmetic for unsigned types. For example, `2147483647i32 + 1i32` wraps to `-2147483648`. For explicit overflow control, use `checked_add/sub/mul` (returns `Result<T, Error>`), `saturating_add/sub/mul` (clamps to type bounds), or `wrapping_add/sub/mul` (self-documenting wrapping). See Function Reference.